

CODE WALKTHROUGH

Vol Trading Dashboard

Every file in the repository, in full, with an explanation of what it does and why — generated for personal review before discussing this project with others.

36

FILES

122

PASSING TESTS

4

TABS ADDED THIS SESSION

github.com/nain0005/vol_trading_dashboard

Contents

Overview

README.md

risk_tool/ — the pure quant engine

risk_tool/README.md

risk_tool/config.py

risk_tool/pricing.py

risk_tool/greeks.py

risk_tool/strike_selection.py

risk_tool/sizing.py

risk_tool/risk_manager.py

risk_tool/realized_vol.py

risk_tool/hedge.py

risk_tool/ibkr_client.py

risk_tool/cli.py

risk_tool/__init__.py

app/ — I/O and presentation, isolated from the models

app/auth.py

app/data_fetch.py

app/demo_data.py

app/vol_analysis.py

app/journal.py

app/performance.py

app/colors.py

app/__init__.py

dashboard.py — the Streamlit UI

dashboard.py

tests/ — 122 passing cases, all independent of a live connection

tests/test_pricing.py

tests/test_greeks.py

tests/test_sizing.py

tests/test_risk_manager.py
tests/test_realized_vol.py
tests/test_strike_selection.py
tests/test_performance.py
tests/test_cli.py
tests/test_ibkr_client.py
tests/test_hedge.py
tests/__init__.py

Project configuration

requirements.txt
pytest.ini
.env.example
.gitignore

Overview

Vol Trading Dashboard

A local dashboard for a mixed options + VIX-ETP book, backed directly by your Robinhood account via [robin_stocks](#).

Everything runs on your machine — your credentials go straight to Robinhood and are never sent anywhere else.

This is personal-use software that connects to **your own** brokerage account — it's not a hosted product or a multi-tenant service, and it isn't meant to run against anyone else's account or be exposed beyond `localhost` (see Notes/limitations below).

What this demonstrates

A few things this codebase is meant to show, for anyone skimming it:

- **Options pricing & Greeks from scratch** — Black-Scholes, an American binomial tree, Brent's-method implied vol solving, all independently verified against known textbook values and put-call parity (`risk_tool/pricing.py`, `risk_tool/greeks.py`).
- **Real risk management, not just signals** — Kelly-criterion sizing with a hard cap that always wins regardless of what Kelly suggests (`risk_tool/sizing.py`), pre-committed exit rules evaluated live against open positions (`risk_tool/risk_manager.py`), and portfolio-level delta/vega governors that can halt new entries.
- **Statistics applied correctly, not just called** — GARCH/EGARCH vol forecasting fit by maximum likelihood (`risk_tool/realized_vol.py`), and a beta-hedge calculator (`risk_tool/hedge.py`) that separates price-level correlation (inflated by shared trend) from return correlation (the honest

co-movement signal), sized off delta-adjusted exposure rather than notional.

- **Test discipline** — the entire `risk_tool/` package is pure, dependency-injected, and independently pytest-covered (122 passing cases: `pytest tests/ -v`) — it's also usable as a standalone CLI with no Streamlit/Robinhood dependency at all (`python3 -m risk_tool.cli --help`).
- **Production-adjacent app structure** — Robinhood I/O is fully isolated from presentation (`app/data_fetch.py` returns plain DataFrames, `dashboard.py` is UI-only), so the business logic is testable without a live session or network access.

What it shows

- **Overview** — equity, cash, buying power, day P&L, portfolio value over time.
- **Positions & Greeks** — equity + option positions, with per-contract delta, theta, vega, gamma, and implied volatility, plus a near-expiry warning.
- **Vol Exposure** — book-level net delta/theta/vega/gamma, vega & theta broken down by underlying, and your VIX-ETP holdings (VXX/UVXY/SVXY/etc., configurable) with live quotes.
- **Vol Skew** — live IV-by-strike and bid/ask-spread-by-strike for any underlying/expiration, with your held contracts marked on both charts.
- **Risk Tool** — strike selection by expected value, Kelly-based position sizing with a non-overridable hard cap, pre-committed entry/exit levels, and a live monitor that runs exit rules against your actual open positions. Does not predict direction — see `risk_tool/README.md` for the full model-by-model writeup (math, assumptions, limitations). Also usable standalone: `python3 -m risk_tool.cli --help`.
- **Correlation Explorer** — cumulative-return chart and price-level/return correlation for any two symbols you type in.
- **Hedge Calculator** — sizes a beta-hedge (shares or futures) for a shares

or options position against any correlated instrument, with beta estimated live from price history, plus a movable P&L scenario chart (drag to any bearish or bullish move) for the resulting hedged position.

- **Orders & History** — open orders and recent fills.
- **Win Rate** — realized round-trip trades FIFO-matched from your fill history, with win rate, avg win/loss, profit factor, a cumulative realized-P&L chart, and a breakdown by equity vs. options. Equity matching is exact; options are matched per underlying symbol only (fill history has no per-contract identity) — see the caveat in the tab itself.
- **Journal / Export** — normalized trade log with a CSV download button (and an option to save a timestamped copy into `exports/`).

Setup

```
1 cd vol-dashboard
2 python3 -m venv .venv
3 source .venv/bin/activate
4 pip install -r requirements.txt
5
6 cp .env.example .env
7
8 # edit .env: set ROBINHOOD_USERNAME / ROBINHOOD_PASSWORD, and optionally
9 # ROBINHOOD_TOTP_SECRET if you have app-based 2FA (see below), and
10 # VOL_TICKERS if your vol-ETP watchlist differs from the default.
11
12 streamlit run dashboard.py
```

This opens the dashboard in your browser at `http://localhost:8501`.

Demo mode (no Robinhood account required)

```
DEMO_MODE=true streamlit run dashboard.py
```

Runs the exact same UI on synthetic data (`app/demo_data.py`) — no Robinhood credentials, no login screen, no real account anywhere in the loop. Prices, positions, and history are generated deterministically (seeded per symbol), but option prices/Greeks are computed with the real `risk_tool` Black-Scholes

engine, so that part is genuinely the same math the live version uses.

This is what a **public deployment** should run — e.g. on

[Streamlit Community Cloud](#): point it at this

repo, set the main file to `dashboard.py`, and add `DEMO_MODE = "true"` under

the app's Secrets (not `.env` — that file never leaves your machine and isn't

part of the repo). Never deploy this app publicly connected to a real

Robinhood account — see Notes/limitations below for why.

Login & 2FA

- If Robinhood challenges you for an SMS/app code on first login, the app will show a code input field — enter it and click "Log in" again.
- To skip that prompt on future runs, set `ROBINHOOD_TOTP_SECRET` in `.env` to your authenticator app's base32 secret (the same one you'd scan as a QR code when setting up 2FA) — the app generates the current code automatically.
- `robin_stocks` caches a session token locally (`~/.tokens/robinhood.pickle`) so you generally won't be re-prompted every single run.

Notes / limitations

- `VOL_TICKERS` in `.env` controls which symbols count as "vol ETPs" for the dedicated exposure view — edit it to match what you actually trade.
- Robinhood's API doesn't expose the raw VIX index quote, so vol-ETP tracking uses the ETPs themselves (VXX, UVXY, etc.) as the proxy, not `^VIX` directly.
- The trade journal covers **filled** orders from the last 90 days by default (`data_fetch.get_order_history(days_back=...)`); increase that if you want a longer lookback.
- This is a personal-use tool with no auth layer of its own — don't expose `streamlit run` beyond `localhost` (e.g. don't tunnel it to the public internet) since it holds your live Robinhood session.

Project layout

```
|
|
| dashboard.py          Streamlit endpoint / UI
| pp/auth.py           Robinhood login (+ MFA) handling
| pp/data_fetch.py     All robin stocks calls, returned as plain DataFrame
| pp/vol_analysis.py   Greeks aggregation, vega/theta by underlying, expiry checks, cross-symbol correlation
| pp/journal.py        Trade journal normalization + CSV export
| pp/performance.py    FIFO round-trip trade matching + win-rate stats
| pp/colors.py         Shared chart color tokens
| risk tool/           Pricing, Greeks, strike selection, sizing, exit rules, beta-hedge sizing - see
| risk tool/README.md
| tests/               pytest suite for risk tool (run: pytest tests/ -v)
| exports/             CSV journal exports land here (gitignored)
```

risk_tool/ — the pure quant engine

risk_tool/README.md

The risk_tool package's own written explanation of every model — pricing, EV, sizing, exit rules, realized vol. Read this section first; the code sections below implement exactly what it describes.

risk_tool

An options risk-management and strike-selection tool. It sizes positions, ranks strikes by expected value, and enforces pre-committed exits. ****It does not predict direction.**** You bring the thesis (ticker, direction, timeframe); this tool prices it, sizes it, and holds you to your own exit rules.

Every module is pure Python with no UI/network dependency — `pricing.py`, `greeks.py`, `strike_selection.py`, `sizing.py`, `risk_manager.py`, and `realized_vol.py` all run on manual inputs and are independently unit tested (see `../tests/`). `ibkr_client.py` and the Robinhood-backed dashboard tab are optional data sources layered on top.

Quickstart

```
|
from the vol-dashboard/ directory, with venv activated
python3 -m pytest tests/ -v # 87 tests as of this writing

python3 -m risk_tool_cli
--ticker AAPL --spot 190 --iv 0.32 --dte 30
--direction call --account 50000 --strike-increment 5

add your own vol view for the realized-vs-implied edge check
python3 -m risk_tool_cli
--ticker AAPL --spot 190 --iv 0.32 --dte 30
--direction call --account 50000 --strike-increment 5 --realized-vol 0.45
```

Or use the **Risk Tool** tab in the Streamlit dashboard (``streamlit run dashboard.py``) — same models, plus it can auto-fetch spot/IV from your live Robinhood connection and monitor your actual open positions against the

exit rules in real time.

Models, one by one

1. Pricing & Greeks (`pricing.py`, `greeks.py`)

Black-Scholes-Merton with continuous dividend yield q :

```
1
d1 = (ln(S/K) + (r - q + sigma**2/2) * T) / (sigma * sqrt(T))
d2 = d1 - sigma * sqrt(T)
call = S * e**(-q * T) * N(d1) - K * e**(-r * T) * N(d2)
put = K * e**(-r * T) * N(-d2) - S * e**(-q * T) * N(-d1)
```

$N(d2)$ is the risk-neutral probability of finishing in-the-money; the module exposes raw $N(d1) / N(d2)$ directly. Delta is $e^{-qT} \cdot N(d1)$ for a call — **not** bare $N(d1)$ unless $q=0$, a common source of quietly-wrong deltas on dividend payers.

American options are priced with a **Cox-Ross-Rubinstein binomial tree** (`binomial_crr_price`), which checks early exercise at every node. Two facts anchor its tests: an American call on a non-dividend stock must price identically to European (never optimal to exercise early), while an American put can be worth strictly more (early exercise has real value there).

Implied vol (`implied_volatility`) uses Newton-Raphson first (quadratic convergence via vega), falling back to Brent's method (`scipy.optimize.brentq`) whenever vega is too small for Newton-Raphson to be trusted — deep ITM/OTM contracts, mainly.

Assumptions and where they break: constant σ and r over the option's life (real vol is not constant — that's the entire reason `realized_vol.py` and the GARCH forecast exist), continuous trading with no transaction costs or bid/ask spread, and log-normal terminal prices (no jumps — earnings gaps and other jump risk are not modeled at all).

2. Strike selection & EV (``strike_selection.py``)

Read the module docstring before trusting any number here — the short

version:

- $EV = P_{win} \cdot \text{profit_if_win} - P_{lose} \cdot \text{premium}$, where $P_{win} = N(d2)$ (or $N(-d2)$ for puts).
- `profit_if_win` is your **profit-target rule's** payout (`premium × profit_target_pct`, default +100%), not the option's theoretical (unbounded, for a call) payoff at expiry. This is deliberate: it keeps the EV consistent with how `risk_manager.py` actually makes you exit, instead of an idealized hold-to-expiry payoff nobody following the discipline rules ever takes.
- Consequently, `risk_reward` here is relative to **your own target**, not the option's max theoretical gain — don't read it as "this call can only make 2x."
- `P_win` is risk-neutral: it's what the option's price implies about the odds of finishing ITM if the stock drifted at the risk-free rate and the given σ were the true vol. **It is not a forecast.** Pricing built on market IV is, by construction, close to a zero-edge bet before frictions — ranking strikes by risk-neutral EV mostly tells you about cost-efficiency and convexity trade-offs across strikes, not "which strike is more likely to make you money."

The actual edge: `compare_implied_vs_realized_ev` is the first-class feature this whole module exists to support. It prices the option once, at market IV (that's the real premium you pay), but computes `P_win` **twice** — once with market IV (the "risk-neutral" reference view) and once with your own realized/forecast vol (the "edge" view). If your vol estimate differs meaningfully from the market's, the difference between those two

EVs is a genuine, quantifiable statement about whether you think this option is mispriced — not a probability of the stock going up.

3. Position sizing (`sizing.py`)

Full Kelly: $f^* = (p \cdot b - q) / b$, where `b` is the payout odds implied by your own profit-target/stop-loss rule. **Half-Kelly is the default**

(`config.use_half_kelly`) because `p` and `b` are estimates, not known constants — full Kelly is notorious for compounding an overestimated edge into real drawdown fast.

The hard cap always wins. `size_position()` is the only function meant to be called from outside this module; it computes Kelly, then unconditionally takes `min(kelly_size, cap_size)` as its last step. There is no parameter, flag, or code path elsewhere in this package that can produce a size larger than `config.max_position_pct_of_account` (default 5%) of your account — see `tests/test_sizing.py::test_hard_cap_always_wins_when_kelly_suggests_more`.

Assumption that breaks fastest: Kelly assumes you actually know your edge (`p` and `b`) with some precision. You don't — `p` is either a risk-neutral model output or your own vol guess, and `b` depends on you actually exiting at your stated target. Half-Kelly and the hard cap both exist specifically to blunt the damage when that assumption is wrong, which is most of the time.

4. Risk manager — exit rules (`risk\_manager.py`)

Five mechanical, pre-committed rules, each returning **why** it fired with the actual numbers, not just a boolean:

Rule	Default	Fires when
Profit target	+100%	current P&L ≥ target

Stop loss	-50%	current P&L $\leq$ stop		
Delta exit	0.15	`\`	delta\`	` decays below threshold
Theta exit	DTE $\leq$ 3	position is OTM AND DTE at/under threshold		
IV-crush exit	20 pts	IV has dropped $\geq$ threshold since entry		

Plus portfolio-level governors (`check_portfolio_governors`): a ****daily max-loss halt**** on new entries (default 3% of account) and optional net delta/vega dollar caps (unset by default — there's no universal correct value, set them to what your account can actually tolerate).

Known limitation, dashboard-specific: the dashboard doesn't persist a trade log, so "entry IV" for the IV-crush rule is approximated as whatever IV the dashboard first observed for a position in the current session — not the real entry IV from when you opened the trade. The IV-crush exit is only meaningful once the dashboard has been continuously watching a position since you opened it. The CLI and direct `risk_manager.Position` usage don't have this problem if you pass the real entry IV yourself.

5. Realized volatility (``realized_vol.py``)

Three historical-vol estimators, in increasing order of statistical efficiency (more of each day's price action used per estimate):

- **Close-to-close:** $\sigma = \text{std}(\log \text{ returns}) \times \sqrt{252}$. Simplest, noisiest.
- **Parkinson:** uses the day's high/low range,

$\sigma^2 = (1/(4n \cdot \ln 2)) \cdot \sum \ln(H/L)^2$. More efficient, but assumes no overnight gaps.

- **Garman-Klass:** adds open/close, most efficient of the three under continuous-GBM-with-no-drift — an assumption that breaks around earnings, gaps, and trend days, exactly when you most want a correct vol read.

GARCH(1,1) (`fit_garch_11` , `garch_forecast_vol`): fit by maximum likelihood on demeaned returns, $\sigma^2_t = \omega + \alpha \cdot r^2_{t-1} + \beta \cdot \sigma^2_{t-1}$, constrained to $\alpha + \beta < 1$ for stationarity. The forecast mean-reverts geometrically toward the long-run variance $\omega / (1 - \alpha - \beta)$ as the horizon grows — GARCH's answer to "how volatile eventually" is just the unconditional variance, regardless of today's conditions. This is a genuinely simple GARCH implementation (no leverage/asymmetry terms, no fat-tailed innovations) — treat it as one more vol estimate to feed `strike_selection.py`, not a research-grade volatility model.

Architecture notes

- `config.py` is the only place thresholds live — nothing else hardcodes a number. Build a custom `RiskConfig(...)` and pass it through if you want different discipline rules than the defaults.
- `ibkr_client.py` is optional and mockable (`MockIBKRClient`) — nothing in the models above imports it or requires a live connection. It exists to feed the models live numbers instead of typing them by hand, and isn't fully wired for option Greeks/IV yet (see its docstring) since that needs a live IBKR options market-data subscription this environment couldn't test against.
- The dashboard's Risk Tool tab is a thin UI layer over these same functions — it doesn't duplicate any model logic.

What this tool will never do

It will not forecast price direction, and no probability it prints is a real-world prediction — every `P_win`/ITM-probability value is explicitly risk-neutral (see `strike_selection.py`'s module docstring for exactly what that means and doesn't mean). If you want an edge, it has to come from your

own volatility view being better than the market's, compared explicitly via

`compare_implied_vs_realized_ev` — that's the one lever this tool gives you

beyond bookkeeping and discipline enforcement.

risk_tool/config.py

Every threshold used anywhere in risk_tool lives in one dataclass, RiskConfig, with a frozen DEFAULT_CONFIG instance. Nothing downstream hardcodes a number — sizing.py, risk_manager.py, and strike_selection.py all take a config object as a parameter. This is what makes 'change the stop-loss to -30% everywhere' a one-line change instead of a grep-and-replace across the codebase.

```
"""Central configuration for every threshold used across the risk tool.

Nothing in pricing.py, greeks.py, strike_selection.py, sizing.py, or
risk_manager.py should hardcode a threshold — it comes from a RiskConfig
instance, so the whole system's discipline rules live in one auditable
place you can inspect, log, and change without touching model code.

Defaults marked "pick your own" don't have a universally correct value —
they depend on your account size and risk tolerance, not on any model.
"""

from __future__ import annotations

from dataclasses import dataclass

@dataclass
class RiskConfig:
    # --- Market defaults (override per call whenever you have better inputs) ---
    risk_free_rate: float = 0.05 # annualized, continuously compounded
    dividend_yield: float = 0.0 # annualized continuous dividend yield q

    # --- Strike selection ---
    num_strikes_each_side: int = 4 # candidate strikes around ATM, per side
    min_risk_reward_ratio: float = 2.0 # strikes below this are flagged "poor setup"
    default_profit_target_pct: float = 1.00 # used as "profit_if_win" basis when no explicit target price is
    given

    # --- Position sizing (Kelly) ---
    use_half_kelly: bool = True # edge estimates are noisy; half-Kelly is the safer default
    max_position_pct_of_account: float = 0.05 # HARD cap — always wins, see sizing.py

    # --- Exit rules ---
    profit_target_pct: float = 1.00 # close at +100% gain on premium paid
    stop_loss_pct: float = -0.50 # close at -50% loss on premium paid
    delta_exit_threshold: float = 0.15 # close a long option if |delta| falls below this
    theta_exit_dte_threshold: int = 3 # DTE floor for the OTM theta-burn exit
    iv_crush_threshold_points: float = 0.20 # close if IV has dropped this many vol points since entry

    # --- Portfolio governors ("pick your own" — set to what your account can tolerate) ---
    daily_max_loss_pct: float = 0.03 # halt new entries once today's P&L hits -3% of account
    max_net_delta_dollars: float | None = None # None = unset/unenforced until you set it
    max_net_vega_dollars: float | None = None # None = unset/unenforced until you set it

    # --- Numerics ---
    binomial_steps: int = 500 # CRR tree steps for American pricing
    iv_solver_max_iter: int = 100
    iv_solver_tol: float = 1e-8
```

```
iv_solver_bounds: tuple[float, float] = (1e-4, 5.0) # Brent fallback search range for sigma
```

```
DEFAULT_CONFIG = RiskConfig()
```

risk_tool/pricing.py

Black-Scholes-Merton pricing with continuous dividend yield q , a Cox-Ross-Rubinstein binomial tree for American options (early-exercise-aware at every node), and implied-vol inversion (Newton-Raphson first, falling back to Brent's method when vega is too small for Newton to be numerically trustworthy — deep ITM/OTM contracts). Every function is pure: no I/O, no globals, just math — which is exactly why it's trivial to unit test against known values (see tests/test_pricing.py).

```
"""Black-Scholes pricing, the CRR binomial tree for American options, and
implied-volatility inversion.

Every function here is pure and stateless – no I/O, no Streamlit, no config
object required – so they're directly unit-testable and reusable from a
CLI, a dashboard, or live IBKR data alike.
"""

from __future__ import annotations

import math
from dataclasses import dataclass
from typing import Literal

from scipy.optimize import brentq
from scipy.stats import norm

OptionType = Literal["call", "put"]

def _check_option_type(option_type: str) -> None:
    if option_type not in ("call", "put"):
        raise ValueError(f"option_type must be 'call' or 'put', got {option_type!r}")

@dataclass(frozen=True)
class D1D2:
    d1: float
    d2: float

def compute_d1_d2(S: float, K: float, T: float, r: float, q: float, sigma: float) -> D1D2:
    """The two Black-Scholes standardized distances.

    
$$d1 = \frac{\ln(S/K) + (r - q + \sigma^2/2) * T}{\sigma * \sqrt{T}}$$

    
$$d2 = d1 - \sigma * \sqrt{T}$$


     $d2$  is the risk-neutral z-score of the terminal log-price relative to the
    strike:  $N(d2)$  is the risk-neutral probability the option finishes ITM
    for a call,  $N(-d2)$  for a put.  $d1$  is the analogous quantity that appears
    in delta once you account for the  $S * e^{-qT}$  drift term – see greeks.py.
    Both are RISK-NEUTRAL constructs, not real-world return distributions.
    """
    if T <= 0:
        raise ValueError(f"T must be positive, got {T}")
    if sigma <= 0:
        raise ValueError(f"sigma must be positive, got {sigma}")
```

```

if S <= 0 or K <= 0:
    raise ValueError("S and K must be positive")

sqrt_T = math.sqrt(T)
d1 = (math.log(S / K) + (r - q + 0.5 * sigma**2) * T) / (sigma * sqrt_T)
d2 = d1 - sigma * sqrt_T
return D1D2(d1=d1, d2=d2)

def black_scholes_price(S: float, K: float, T: float, r: float, q: float, sigma: float, option_type:
OptionType) -> float:
    """European option price under Black-Scholes-Merton with continuous
    dividend yield q.

    Call =  $S * e^{-qT} * N(d1) - K * e^{-rT} * N(d2)$ 
    Put =  $K * e^{-rT} * N(-d2) - S * e^{-qT} * N(-d1)$ 

    Assumes: constant sigma and r over the option's life, continuous
    trading, no transaction costs, log-normal terminal price distribution.
    None of these hold exactly in real markets – see README for what that
    means for this tool's outputs.
    """
    _check_option_type(option_type)
    d = compute_d1_d2(S, K, T, r, q, sigma)
    disc_q = math.exp(-q * T)
    disc_r = math.exp(-r * T)

    if option_type == "call":
        return S * disc_q * norm.cdf(d.d1) - K * disc_r * norm.cdf(d.d2)
    return K * disc_r * norm.cdf(-d.d2) - S * disc_q * norm.cdf(-d.d1)

def itm_probability(S: float, K: float, T: float, r: float, q: float, sigma: float, option_type: OptionType)
-> float:
    """Risk-neutral probability of finishing in-the-money: N(d2) for a call,
    N(-d2) for a put.

    THIS IS NOT a real-world / statistical probability. It's the
    probability under the risk-neutral measure implied by treating the
    option's IV as the true vol and the stock as drifting at the
    risk-free rate – an artifact of no-arbitrage pricing, not a forecast
    of where the stock will actually go. Use it to price and rank setups,
    never to justify a directional call on its own.
    """
    _check_option_type(option_type)
    d = compute_d1_d2(S, K, T, r, q, sigma)
    return norm.cdf(d.d2) if option_type == "call" else norm.cdf(-d.d2)

def binomial_crr_price(
    S: float,
    K: float,
    T: float,
    r: float,
    q: float,
    sigma: float,
    option_type: OptionType,
    american: bool = True,
    n_steps: int = 500,
) -> float:

```

```
"""Cox-Ross-Rubinstein binomial tree, early-exercise aware for American options.
```

```
    u = e^(sigma*sqrt(dt)), d = 1/u
    p = (e^((r-q)*dt) - d) / (u - d)      [risk-neutral up probability]
    discount = e^(-r*dt)
```

```
Backward induction: at each node, the American value is
max(intrinsic value, discounted continuation value); the European
value just takes the continuation value and never checks intrinsic
value early. This is exactly where American and European pricing
diverge – see tests/test_pricing.py for the case that proves it
(American calls on non-dividend stock should NEVER early-exercise, so
American == European there; American puts do diverge).
```

```
"""
```

```
_check_option_type(option_type)
```

```
if T <= 0 or sigma <= 0:
```

```
    raise ValueError("T and sigma must be positive")
```

```
if n_steps < 1:
```

```
    raise ValueError("n_steps must be >= 1")
```

```
dt = T / n_steps
```

```
u = math.exp(sigma * math.sqrt(dt))
```

```
d = 1.0 / u
```

```
disc = math.exp(-r * dt)
```

```
p = (math.exp((r - q) * dt) - d) / (u - d)
```

```
if not (0.0 < p < 1.0):
```

```
    raise ValueError(
```

```
        f"Risk-neutral probability p={p:.4f} outside (0,1) – dt is too "
```

```
        "large relative to r, q, sigma. Increase n_steps."
```

```
)
```

```
# Terminal payoffs across all n_steps+1 terminal nodes.
```

```
terminal_prices = [S * (u**j) * (d ** (n_steps - j)) for j in range(n_steps + 1)]
```

```
if option_type == "call":
```

```
    values = [max(px - K, 0.0) for px in terminal_prices]
```

```
else:
```

```
    values = [max(K - px, 0.0) for px in terminal_prices]
```

```
# Roll backward one time step at a time, checking early exercise at
```

```
# every node when american=True.
```

```
for step in range(n_steps - 1, -1, -1):
```

```
    next_values = [0.0] * (step + 1)
```

```
    for j in range(step + 1):
```

```
        continuation = disc * (p * values[j + 1] + (1 - p) * values[j])
```

```
        if american:
```

```
            spot_at_node = S * (u**j) * (d ** (step - j))
```

```
            intrinsic = max(spot_at_node - K, 0.0) if option_type == "call" else max(K - spot_at_node,
```

```
0.0)
```

```
            next_values[j] = max(continuation, intrinsic)
```

```
        else:
```

```
            next_values[j] = continuation
```

```
    values = next_values
```

```
return values[0]
```

```
def implied_volatility(
```

```
    market_price: float,
```

```
    S: float,
```

```
    K: float,
```

```

T: float,
r: float,
q: float,
option_type: OptionType,
initial_guess: float = 0.3,
max_iter: int = 100,
tol: float = 1e-8,
bounds: tuple[float, float] = (1e-4, 5.0),
) -> float:
    """Solve for sigma such that black_scholes_price(..., sigma) == market_price.

    Primary method is Newton-Raphson using vega as the derivative:

        sigma_{n+1} = sigma_n - (BS(sigma_n) - market_price) / vega(sigma_n)

    This converges quadratically near the root but is unstable whenever
    vega is small - which happens for deep ITM/OTM contracts, where price
    is nearly flat in sigma. In that regime we fall back to Brent's method
    (scipy.optimize.brentq), a bracketed bisection-family solver that's
    slower but guaranteed to converge given a sign change across the
    bracket, which is exactly the case where Newton-Raphson can't be
    trusted.
    """
    _check_option_type(option_type)
    from risk_tool.greeks import vega as vega_fn # local import: avoids a pricing<->greeks import cycle

    sigma = initial_guess
    for _ in range(max_iter):
        try:
            price = black_scholes_price(S, K, T, r, q, sigma, option_type)
        except ValueError:
            break # sigma wandered non-positive; drop to Brent's method
        diff = price - market_price
        if abs(diff) < tol:
            return sigma

        v = vega_fn(S, K, T, r, q, sigma)
        if v < 1e-8:
            break # vega too small for Newton-Raphson to make progress
        sigma -= diff / v
        if sigma <= 0:
            break # stepped into an invalid region

    def objective(s: float) -> float:
        return black_scholes_price(S, K, T, r, q, s, option_type) - market_price

    lo, hi = bounds
    try:
        return brentq(objective, lo, hi, xtol=tol, maxiter=max_iter)
    except ValueError as exc:
        raise ValueError(
            f"Could not solve implied vol within bounds {bounds} for "
            f"market_price={market_price}: {exc}. The price may be outside "
            "any value Black-Scholes can produce for this contract "
            "(e.g. below intrinsic value, or a data error)."
        ) from exc

```

risk_tool/greeks.py

Delta, gamma, vega, theta, rho, all derived from the same $d1/d2$ Black-Scholes machinery in pricing.py. The one detail worth knowing cold for an interview: delta is $e^{-qT} * N(d1)$ for a call, not bare $N(d1)$ — the dividend discount factor is easy to forget and silently produces a wrong delta on any dividend-paying underlying.

```
"""Greeks derived from the Black-Scholes  $d1/d2$ , plus the raw  $N(d1)/N(d2)$ 
values the strike-selection model needs directly.

Vega and gamma have the same formula for calls and puts; delta, theta, and
rho differ by sign and by which cumulative-normal term they use. All
include the continuous dividend yield  $q$  — dropping it is a common source of
silently-wrong deltas on dividend-paying names.
"""
from __future__ import annotations

import math

from scipy.stats import norm

from risk_tool.pricing import D1D2, OptionType, _check_option_type, compute_d1_d2

TRADING_DAYS_PER_YEAR = 365.0 # theta-per-day convention; use 252 for a trading-day convention instead

def delta(S: float, K: float, T: float, r: float, q: float, sigma: float, option_type: OptionType) -> float:
    """dPrice/dS.

    Call:  $e^{-qT} * N(d1)$ 
    Put:  $e^{-qT} * (N(d1) - 1) = -e^{-qT} * N(-d1)$ 

    Equals bare  $N(d1)$  only when  $q = 0$  — the dividend discount factor is not
    optional and matters most for high-yield underlyings.
    """
    _check_option_type(option_type)
    d = compute_d1_d2(S, K, T, r, q, sigma)
    disc_q = math.exp(-q * T)
    if option_type == "call":
        return disc_q * norm.cdf(d.d1)
    return disc_q * (norm.cdf(d.d1) - 1.0)

def gamma(S: float, K: float, T: float, r: float, q: float, sigma: float) -> float:
    """d(delta)/dS — identical formula for calls and puts:

     $e^{-qT} * \phi(d1) / (S * sigma * \sqrt{T})$ 
    """
    d = compute_d1_d2(S, K, T, r, q, sigma)
    return math.exp(-q * T) * norm.pdf(d.d1) / (S * sigma * math.sqrt(T))

def vega(S: float, K: float, T: float, r: float, q: float, sigma: float) -> float:
    """dPrice/dsigma, per 1.00 (100 vol points) of sigma — identical formula
```

```

for calls and puts:

    S * e-qT * phi(d1) * sqrt(T)

Divide by 100 for the conventional "price change per 1 vol point" quote.
"""
d = compute_d1_d2(S, K, T, r, q, sigma)
return S * math.exp(-q * T) * norm.pdf(d.d1) * math.sqrt(T)

def theta(S: float, K: float, T: float, r: float, q: float, sigma: float, option_type: OptionType, per_day:
bool = True) -> float:
    """dPrice/dt - time decay, expressed as the option's value change per
    unit of time passing (negative for a long option in the typical case).
    Annualized by the raw formula; divided by 365 when per_day=True.
    """
    _check_option_type(option_type)
    d = compute_d1_d2(S, K, T, r, q, sigma)
    disc_q = math.exp(-q * T)
    disc_r = math.exp(-r * T)

    decay_term = -S * disc_q * norm.pdf(d.d1) * sigma / (2 * math.sqrt(T))
    if option_type == "call":
        raw = decay_term - r * K * disc_r * norm.cdf(d.d2) + q * S * disc_q * norm.cdf(d.d1)
    else:
        raw = decay_term + r * K * disc_r * norm.cdf(-d.d2) - q * S * disc_q * norm.cdf(-d.d1)

    return raw / TRADING_DAYS_PER_YEAR if per_day else raw

def rho(S: float, K: float, T: float, r: float, q: float, sigma: float, option_type: OptionType) -> float:
    """dPrice/dr, per 1.00 (100%) change in r. Divide by 100 for "per 1% rate move"."""
    _check_option_type(option_type)
    d = compute_d1_d2(S, K, T, r, q, sigma)
    disc_r = math.exp(-r * T)
    if option_type == "call":
        return K * T * disc_r * norm.cdf(d.d2)
    return -K * T * disc_r * norm.cdf(-d.d2)

def n_d1_d2(S: float, K: float, T: float, r: float, q: float, sigma: float) -> dict[str, float]:
    """Raw d1, d2, N(d1), N(d2) - exposed explicitly per spec.

    N(d2) is the risk-neutral ITM probability for a CALL (use N(-d2) for a
    put - see pricing.itm_probability). N(d1) is the delta-adjacent term,
    equal to call delta only when q = 0. Both are risk-neutral quantities,
    not real-world probabilities.
    """
    d: D1D2 = compute_d1_d2(S, K, T, r, q, sigma)
    return {"d1": d.d1, "d2": d.d2, "N_d1": norm.cdf(d.d1), "N_d2": norm.cdf(d.d2)}

def all_greeks(S: float, K: float, T: float, r: float, q: float, sigma: float, option_type: OptionType) ->
dict[str, float]:
    """Convenience bundle used by strike_selection.py and the dashboard."""
    return {
        "delta": delta(S, K, T, r, q, sigma, option_type),
        "gamma": gamma(S, K, T, r, q, sigma),
        "vega": vega(S, K, T, r, q, sigma),
        "theta": theta(S, K, T, r, q, sigma, option_type),
    }

```

```
    "rho": rho(S, K, T, r, q, sigma, option_type),  
    **n_d1_d2(S, K, T, r, q, sigma),  
}
```

risk_tool/strike_selection.py

The most conceptually important module in the whole project — read its docstring twice. It ranks candidate strikes by expected value, where EV is NOT the option's textbook payoff at expiry; it's the EV of a specific mechanical plan (pay premium, exit at your own profit-target or stop-loss). The critical distinction it draws: risk-neutral win probability ($N(d_2)$, derived from market IV) is not a real-world forecast — pricing built on market IV is close to a zero-edge bet by construction. The one place actual edge can come from is `compare_implied_vs_realized_ev`, which prices at market IV (what you actually pay) but computes win probability twice — once with market IV, once with your own realized/forecast vol — so the gap between those two EVs is a genuine, quantifiable statement about whether you think the market has mispriced volatility.

```
"""Expected-value strike ranking, and the realized-vs-implied comparison
that's the actual source of edge in this tool.
```

```
IMPORTANT – read this before trusting any number this module produces:
```

```
The "EV" here is NOT the option's true expected payoff at expiry. It's the
expected value of a specific, mechanical plan: pay the premium, and exit
at either your own profit-target or your own stop-loss rule (the same
rule risk_manager.py enforces). That's a deliberate choice – it makes the
EV model consistent with how you actually trade, instead of an idealized
"hold to expiry" payoff nobody following a discipline system actually
takes. The tradeoff: max_gain/risk_reward here are relative to YOUR
profit target, not the option's theoretical (unbounded, for a call)
upside – don't confuse the two.
```

```
P_win = N(d2) (or N(-d2) for puts) is a RISK-NEUTRAL probability: it's
what the option's price implies about the odds of finishing ITM if the
stock drifted at the risk-free rate and the given sigma were the true
vol. It is NOT a real-world forecast, and by itself it doesn't tell you
whether a trade has positive real-world expected value – pricing built on
market IV is, by construction, close to a fair (zero-edge) bet before
frictions.
```

```
The actual, honest edge in this framework can only come from ONE place:
believing your own volatility estimate (realized vol, or a GARCH
forecast) is a better estimate of what will actually happen than the
market's IV. `compare_implied_vs_realized_ev` makes that comparison
explicit and first-class, rather than burying it as a side note.
```

```
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
from risk_tool.config import DEFAULT_CONFIG, RiskConfig
```

```
from risk_tool.greeks import all_greeks
```

```
from risk_tool.pricing import OptionType, black_scholes_price, itm_probability
```

```
@dataclass
```

```
class StrikeEV:
```

```

strike: float
option_type: OptionType
premium: float
p_win_risk_neutral: float #  $N(d2)/N(-d2)$  at the sigma actually passed in – LABEL AS RISK-NEUTRAL, not a
forecast
profit_if_win: float # premium * profit_target_pct – profit AT YOUR OWN TARGET, not at expiry
max_loss: float # = premium (worst case for a long option)
ev: float
risk_reward: float # profit_if_win / max_loss, relative to your own target – see module docstring
is_poor_setup: bool # risk_reward below config.min_risk_reward_ratio
delta: float
gamma: float
vega: float
theta: float

def evaluate_strike(
    S: float,
    K: float,
    T: float,
    r: float,
    q: float,
    sigma: float,
    option_type: OptionType,
    config: RiskConfig = DEFAULT_CONFIG,
    profit_target_pct: float | None = None,
) -> StrikeEV:
    """EV for one strike: EV = P_win * profit_if_win - P_lose * premium.

    `sigma` is whatever vol you pass in – market IV for the risk-neutral
    view, or your own realized/forecast vol for the "fair" view (see
    compare_implied_vs_realized_ev). The premium itself always uses
    whichever sigma is passed in too, so if you want "market premium, my
    probability," call this twice with the same premium held fixed
    manually (compare_implied_vs_realized_ev does exactly that).
    """
    target_pct = profit_target_pct if profit_target_pct is not None else config.default_profit_target_pct

    premium = black_scholes_price(S, K, T, r, q, sigma, option_type)
    p_win = itm_probability(S, K, T, r, q, sigma, option_type)
    p_lose = 1.0 - p_win

    profit_if_win = premium * target_pct
    max_loss = premium
    ev = p_win * profit_if_win - p_lose * max_loss
    risk_reward = profit_if_win / max_loss if max_loss > 0 else float("inf")
    greeks = all_greeks(S, K, T, r, q, sigma, option_type)

    return StrikeEV(
        strike=K,
        option_type=option_type,
        premium=premium,
        p_win_risk_neutral=p_win,
        profit_if_win=profit_if_win,
        max_loss=max_loss,
        ev=ev,
        risk_reward=risk_reward,
        is_poor_setup=bool(risk_reward < config.min_risk_reward_ratio),
        delta=greeks["delta"],
        gamma=greeks["gamma"],

```

```

        vega=greeks["vega"],
        theta=greeks["theta"],
    )

def generate_candidate_strikes(spot: float, strike_increment: float, num_each_side: int) -> list[float]:
    """ATM strike (spot rounded to the nearest increment) plus num_each_side
    strikes above and below it."""
    if strike_increment <= 0:
        raise ValueError("strike_increment must be positive")
    if num_each_side < 0:
        raise ValueError("num_each_side must be >= 0")
    atm = round(spot / strike_increment) * strike_increment
    return sorted(atm + i * strike_increment for i in range(-num_each_side, num_each_side + 1))

def rank_strikes_by_ev(
    S: float,
    T: float,
    r: float,
    q: float,
    sigma: float,
    option_type: OptionType,
    strikes: list[float],
    config: RiskConfig = DEFAULT_CONFIG,
    profit_target_pct: float | None = None,
    max_premium_dollars: float | None = None,
    contract_multiplier: int = 100,
) -> list[StrikeEV]:
    """Evaluate every candidate strike and return them ranked by EV,
    descending. If max_premium_dollars is given (e.g. your position-size
    cap for one contract), strikes whose premium * multiplier exceeds it
    are dropped entirely rather than just flagged – you can't take them at
    that size regardless of how good their EV looks.
    """
    results = []
    for K in strikes:
        strike_ev = evaluate_strike(S, K, T, r, q, sigma, option_type, config, profit_target_pct)
        if max_premium_dollars is not None and strike_ev.premium * contract_multiplier > max_premium_dollars:
            continue
        results.append(strike_ev)
    return sorted(results, key=lambda s: s.ev, reverse=True)

@dataclass
class RealizedVsImpliedEV:
    strike: float
    option_type: OptionType
    market_premium: float # priced at market IV – this is what you actually pay
    market_iv: float
    realized_or_forecast_vol: float
    vol_spread: float # realized_or_forecast_vol – market_iv; positive = you think it'll be MORE volatile
    than priced
    risk_neutral_ev: StrikeEV # EV using market IV for both price and probability (the "textbook", ~zero-
    edge view)
    edge_ev: StrikeEV # EV using YOUR vol for probability, but the market's actual premium – this is the
    edge check

def compare_implied_vs_realized_ev(

```

```

S: float,
K: float,
T: float,
r: float,
q: float,
market_iv: float,
realized_or_forecast_vol: float,
option_type: OptionType,
config: RiskConfig = DEFAULT_CONFIG,
profit_target_pct: float | None = None,
) -> RealizedVsImpliedEV:
    """The first-class realized-vs-implied comparison.

    risk_neutral_ev prices AND assigns probability using market_iv - this
    is close to a fair-value view by construction (options pricing is
    close to zero-edge before frictions when your vol assumption matches
    the market's).

    edge_ev keeps the premium you'd ACTUALLY pay (priced at market_iv,
    since that's the real quote) but assigns win probability using YOUR
    OWN volatility estimate. If your realized/forecast vol differs
    meaningfully from market IV, this is where a real, quantifiable edge
    (or a real, quantifiable reason to avoid the trade) shows up. This is
    the actual edge this tool can offer - everything else is bookkeeping
    and discipline enforcement.
    """
    market_premium = black_scholes_price(S, K, T, r, q, market_iv, option_type)

    risk_neutral_ev = evaluate_strike(S, K, T, r, q, market_iv, option_type, config, profit_target_pct)

    # edge_ev: probability from realized/forecast vol, but priced (premium,
    # profit_if_win, max_loss) off the market's actual premium.
    target_pct = profit_target_pct if profit_target_pct is not None else config.default_profit_target_pct
    p_win_edge = itm_probability(S, K, T, r, q, realized_or_forecast_vol, option_type)
    p_lose_edge = 1.0 - p_win_edge
    profit_if_win = market_premium * target_pct
    edge_ev_value = p_win_edge * profit_if_win - p_lose_edge * market_premium
    edge_greeks = all_greeks(S, K, T, r, q, market_iv, option_type) # Greeks quoted off the tradable
    (market-IV) contract
    edge_ev = StrikeEV(
        strike=K,
        option_type=option_type,
        premium=market_premium,
        p_win_risk_neutral=p_win_edge, # computed with YOUR vol - see field docstring caveat, still risk-
neutral-style math
        profit_if_win=profit_if_win,
        max_loss=market_premium,
        ev=edge_ev_value,
        risk_reward=profit_if_win / market_premium if market_premium > 0 else float("inf"),
        is_poor_setup=bool((profit_if_win / market_premium if market_premium > 0 else float("inf")) <
config.min_risk_reward_ratio),
        delta=edge_greeks["delta"],
        gamma=edge_greeks["gamma"],
        vega=edge_greeks["vega"],
        theta=edge_greeks["theta"],
    )

    return RealizedVsImpliedEV(
        strike=K,
        option_type=option_type,

```

```
market_premium=market_premium,  
market_iv=market_iv,  
realized_or_forecast_vol=realized_or_forecast_vol,  
vol_spread=realized_or_forecast_vol - market_iv,  
risk_neutral_ev=risk_neutral_ev,  
edge_ev=edge_ev,  
)
```

risk_tool/sizing.py

Kelly criterion position sizing: $f^* = (p \cdot b - q) / b$. Half-Kelly is the default because p and b are estimates, not known constants, and full Kelly is notorious for compounding an overestimated edge into real drawdown. The single most important property of this module: the hard %-of-account cap always wins, unconditionally, regardless of what Kelly suggests — `size_position()` computes `min(kelly_size, hard_cap)` as its literal last step, and there is no code path anywhere in the package that can bypass it (see the dedicated test for this guarantee).

```
"""Kelly Criterion position sizing, with a mandatory hard cap.

Kelly answers "how much of my capital should I risk on a bet with edge
(p, b)", ASSUMING p and b are known exactly. In reality p comes from a
model (risk-neutral N(d2), or your own vol forecast) and b comes from your
own profit-target/stop-loss rule – both are estimates, not certainties.
Full Kelly is famously aggressive and compounds an overestimated edge into
real drawdown fast; half-Kelly (the default here) trades some long-run
growth rate for a large cut in variance. The hard %-of-account cap exists
for the same reason, and it always wins regardless of what Kelly says:
size_position() is the only function you should call from outside this
module, and the cap is applied as its last step, unconditionally.
"""

from __future__ import annotations

from dataclasses import dataclass

from risk_tool.config import DEFAULT_CONFIG, RiskConfig


def kelly_fraction(p_win: float, b: float) -> float:
    """ $f^* = (p \cdot b - q) / b$ 

    p = probability of winning, q = 1-p, b = odds received (profit per $1
    risked, if you win). Can come out negative – that means the bet has no
    edge at these odds and Kelly says "don't take it," not "go short."
    Callers should floor the result at 0 before sizing anything.
    """
    if not (0.0 <= p_win <= 1.0):
        raise ValueError(f"p_win must be in [0, 1], got {p_win}")
    if b <= 0:
        raise ValueError(f"b (odds received) must be positive, got {b}")
    q = 1.0 - p_win
    return (p_win * b - q) / b


def half_kelly_fraction(p_win: float, b: float) -> float:
    return kelly_fraction(p_win, b) / 2.0


@dataclass
class PositionSizeResult:
    kelly_fraction_raw: float # full Kelly f*, can be negative (no edge)
    kelly_fraction_used: float # floored at 0, then halved if config.use_half_kelly
```

```

    kelly_dollar_size: float
    hard_cap_dollar_size: float
    recommended_dollar_size: float # min(kelly_dollar_size, hard_cap_dollar_size) – the hard cap always wins
ties
    recommended_contracts: int
    capped_by_hard_limit: bool # True if the hard cap is what actually bound the size (not Kelly)

def size_position(
    account_size: float,
    premium_per_contract: float,
    p_win: float,
    profit_if_win_per_contract: float,
    config: RiskConfig = DEFAULT_CONFIG,
    contract_multiplier: int = 100,
) -> PositionSizeResult:
    """The one function to call from outside this module.

    b (odds received) is derived from your own profit-target/stop-loss
    rule: profit_if_win_per_contract / premium_per_contract, i.e. "if I win,
    how many dollars do I make per dollar risked, given I'll take profit at
    my own rule rather than hold to expiry." This keeps sizing consistent
    with the same discipline risk_manager.py enforces on exit.

    The hard cap (config.max_position_pct_of_account of account_size) is
    computed unconditionally and the final recommendation is always
    min(kelly_size, hard_cap) – there is no parameter here that can make
    Kelly override the cap.
    """
    if account_size <= 0:
        raise ValueError("account_size must be positive")
    if premium_per_contract <= 0:
        raise ValueError("premium_per_contract must be positive")

    b = profit_if_win_per_contract / premium_per_contract
    f_raw = kelly_fraction(p_win, b)
    f_used = max(f_raw, 0.0)
    if config.use_half_kelly:
        f_used /= 2.0

    kelly_dollars = f_used * account_size
    hard_cap_dollars = config.max_position_pct_of_account * account_size
    recommended_dollars = min(kelly_dollars, hard_cap_dollars)

    contract_cost = premium_per_contract * contract_multiplier
    recommended_contracts = int(recommended_dollars // contract_cost) if contract_cost > 0 else 0

    return PositionSizeResult(
        kelly_fraction_raw=f_raw,
        kelly_fraction_used=f_used,
        kelly_dollar_size=kelly_dollars,
        hard_cap_dollar_size=hard_cap_dollars,
        recommended_dollar_size=recommended_dollars,
        recommended_contracts=recommended_contracts,
        capped_by_hard_limit=bool(hard_cap_dollars < kelly_dollars),
    )

```

risk_tool/risk_manager.py

Five mechanical, pre-committed exit rules (profit target, stop loss, delta decay, theta/OTM-near-expiry, IV crush) plus portfolio-level governors (daily max-loss halt, optional net delta/vega caps). Every rule returns not just a boolean but the actual numbers and a human-readable reason — the point of the whole module is to replace in-the-moment rationalization with rules you committed to in advance, and to make every exit decision auditable after the fact.

```
"""Exit-rule engine: pre-committed, mechanical exit conditions evaluated
against every open position on every update, plus portfolio-level
governors that halt new entries.

The entire point of this module is to replace gut-feel exits with rules you
committed to before you had a position-specific reason to rationalize
holding. Every rule reports WHY it fired, with the actual numbers, so the
output is auditable rather than a bare yes/no.
"""
from __future__ import annotations

from dataclasses import dataclass

from risk_tool.config import DEFAULT_CONFIG, RiskConfig


@dataclass
class Position:
    symbol: str
    option_type: str # "call" or "put"
    strike: float
    spot: float
    dte: int
    quantity: int # positive = long contracts; this module reasons about long-premium positions
    entry_premium: float # premium paid per contract at entry
    current_premium: float # current mark price per contract
    entry_iv: float # IV at entry (annualized, e.g. 0.35)
    current_iv: float # current IV
    delta: float # current option delta (signed: positive for calls, negative for puts)


@dataclass
class ExitSignal:
    rule: str
    triggered: bool
    reason: str
    numbers: dict


def pct_pnl(position: Position) -> float:
    """(current - entry) / entry, on premium per contract."""
    if position.entry_premium <= 0:
        raise ValueError("entry_premium must be positive")
    return (position.current_premium - position.entry_premium) / position.entry_premium
```

```

def check_profit_target(position: Position, config: RiskConfig = DEFAULT_CONFIG) -> ExitSignal:
    pnl = pct_pnl(position)
    triggered = pnl >= config.profit_target_pct
    reason = (
        f"P&L {pnl:+.1%} has reached the +{config.profit_target_pct:.0%} profit target "
        f"(entry ${position.entry_premium:.2f} -> current ${position.current_premium:.2f})."
        if triggered
        else f"P&L {pnl:+.1%} is below the +{config.profit_target_pct:.0%} profit target."
    )
    return ExitSignal("profit_target", triggered, reason, {"pct_pnl": pnl, "threshold":
config.profit_target_pct})

def check_stop_loss(position: Position, config: RiskConfig = DEFAULT_CONFIG) -> ExitSignal:
    pnl = pct_pnl(position)
    triggered = pnl <= config.stop_loss_pct
    reason = (
        f"P&L {pnl:+.1%} has breached the {config.stop_loss_pct:.0%} stop loss "
        f"(entry ${position.entry_premium:.2f} -> current ${position.current_premium:.2f})."
        if triggered
        else f"P&L {pnl:+.1%} is above the {config.stop_loss_pct:.0%} stop loss."
    )
    return ExitSignal("stop_loss", triggered, reason, {"pct_pnl": pnl, "threshold": config.stop_loss_pct})

def check_delta_exit(position: Position, config: RiskConfig = DEFAULT_CONFIG) -> ExitSignal:
    """For a long option, delta decaying toward zero means the position has
    become a low-probability lottery ticket rather than the directional
    exposure you originally bought – this closes it out regardless of
    current P&L."""
    abs_delta = abs(position.delta)
    triggered = abs_delta < config.delta_exit_threshold
    reason = (
        f"|delta| {abs_delta:.3f} has fallen below the {config.delta_exit_threshold:.2f} exit threshold – "
        "this option barely resembles the original directional bet anymore."
        if triggered
        else f"|delta| {abs_delta:.3f} is still above the {config.delta_exit_threshold:.2f} exit threshold."
    )
    return ExitSignal("delta_exit", triggered, reason, {"abs_delta": abs_delta, "threshold":
config.delta_exit_threshold})

def check_theta_exit(position: Position, config: RiskConfig = DEFAULT_CONFIG) -> ExitSignal:
    """Close OTM positions once DTE is too low for the thesis to plausibly
    still play out – theta burn accelerates fastest in the final days, and
    an OTM option this close to expiry is fighting decay with no help from
    intrinsic value."""
    is_otm = (position.option_type == "call" and position.spot < position.strike) or (
        position.option_type == "put" and position.spot > position.strike
    )
    triggered = position.dte <= config.theta_exit_dte_threshold and is_otm
    reason = (
        f"DTE {position.dte} <= {config.theta_exit_dte_threshold} and position is OTM "
        f"(spot ${position.spot:.2f} vs strike ${position.strike:.2f}) – theta burn with no help from
intrinsic value."
        if triggered
        else f"DTE {position.dte}, OTM={is_otm} – theta exit not triggered."
    )
    return ExitSignal(
        "theta_exit", triggered, reason, {"dte": position.dte, "threshold": config.theta_exit_dte_threshold,

```

```

"is_otm": is_otm}
    )

def check_iv_crush_exit(position: Position, config: RiskConfig = DEFAULT_CONFIG) -> ExitSignal:
    """If IV has dropped sharply since entry (e.g. post-earnings crush), the
    vega tailwind you may have been counting on is gone even if the
    underlying moved your way – this exit is about vol risk specifically,
    independent of the delta/theta/P&L rules above."""
    iv_drop = position.entry_iv - position.current_iv
    triggered = iv_drop >= config.iv_crush_threshold_points
    reason = (
        f"IV dropped {iv_drop:.1%} since entry (entry {position.entry_iv:.1%} -> current
{position.current_iv:.1%}), "
        f"past the {config.iv_crush_threshold_points:.0%}-point crush threshold."
        if triggered
        else f"IV drop {iv_drop:.1%} is within the {config.iv_crush_threshold_points:.0%}-point crush
threshold."
    )
    return ExitSignal(
        "iv_crush_exit", triggered, reason, {"iv_drop": iv_drop, "threshold":
config.iv_crush_threshold_points}
    )

def evaluate_all_rules(position: Position, config: RiskConfig = DEFAULT_CONFIG) -> list[ExitSignal]:
    return [
        check_profit_target(position, config),
        check_stop_loss(position, config),
        check_delta_exit(position, config),
        check_theta_exit(position, config),
        check_iv_crush_exit(position, config),
    ]

def any_exit_triggered(position: Position, config: RiskConfig = DEFAULT_CONFIG) -> bool:
    return any(signal.triggered for signal in evaluate_all_rules(position, config))

@dataclass
class PortfolioState:
    account_size: float
    daily_pnl: float # today's realized + unrealized P&L in dollars; negative = a loss
    net_delta_dollars: float # sum over positions of delta * spot * quantity * multiplier
    net_vega_dollars: float # sum over positions of vega * quantity * multiplier

@dataclass
class GovernorResult:
    halted: bool
    reasons: list[str]

def check_portfolio_governors(state: PortfolioState, config: RiskConfig = DEFAULT_CONFIG) -> GovernorResult:
    """Portfolio-level circuit breakers, independent of any single
    position's exit rules: a daily max-loss halt on new entries, and
    optional net-Greek exposure caps (unset by default – see config.py;
    there's no universal correct value, it's your own risk tolerance)."""
    reasons: list[str] = []

```

```

daily_loss_limit = -abs(config.daily_max_loss_pct) * state.account_size
if state.daily_pnl <= daily_loss_limit:
    reasons.append(
        f"Daily P&L ${state.daily_pnl:,.2f} has breached the max-loss governor "
        f"(${daily_loss_limit:,.2f}, {config.daily_max_loss_pct:.0%} of account) – halt new entries for
today."
    )

    if config.max_net_delta_dollars is not None and abs(state.net_delta_dollars) >
config.max_net_delta_dollars:
        reasons.append(
            f"Net delta exposure ${state.net_delta_dollars:,.2f} exceeds the "
            f"${config.max_net_delta_dollars:,.2f} cap – reduce directional exposure before adding more."
        )

    if config.max_net_vega_dollars is not None and abs(state.net_vega_dollars) > config.max_net_vega_dollars:
        reasons.append(
            f"Net vega exposure ${state.net_vega_dollars:,.2f} exceeds the "
            f"${config.max_net_vega_dollars:,.2f} cap – reduce vol exposure before adding more."
        )

return GovernorResult(halted=len(reasons) > 0, reasons=reasons)

```

risk_tool/realized_vol.py

Three historical volatility estimators in increasing order of statistical efficiency — close-to-close (simplest, noisiest, closing prices only), Parkinson (adds the day's high/low range), Garman-Klass (adds open/close too, most efficient under continuous-GBM-with-no-drift, an assumption that breaks exactly around earnings gaps and jumps). Also a from-scratch GARCH(1,1) and EGARCH(1,1) fit by maximum likelihood, with the asymmetric EGARCH modeling the empirical fact that equity vol rises more after a down move than an up move of the same size (the leverage effect).

```
"""Realized/historical volatility estimators, plus GARCH(1,1) and
EGARCH(1,1) forecasts — the alternative to market IV that
strike_selection.py compares against to find the actual realized-vs-implied
edge.
```

```
Three close-to-close/range estimators, in increasing order of statistical
efficiency (lower variance for the same sample size), because they use
progressively more of each day's price action:
```

- close-to-close: only closing prices
- Parkinson: adds the day's high/low range
- Garman-Klass: adds open too, and is the most efficient of the three for a pure GBM process with no drift or jumps

```
Plus two conditional-vol models that, unlike the three estimators above,
produce a forward-looking forecast rather than a single backward-looking
number:
```

- GARCH(1,1) (`fit\_garch\_11`): symmetric — a big move up and a big move down of the same size raise forecast vol by the same amount.
- EGARCH(1,1) (`fit\_egarch\_11`): adds the "leverage effect" — equity vol empirically rises more after a down move than after an up move of the same size — via an asymmetry term, and models log-variance directly so it needs no positivity constraints on its parameters.

```
Both accept `dist="normal"` (default) or `dist="t"` innovations. Real
equity returns have fatter tails than a normal distribution — large moves
are more common than a Gaussian would predict — so `dist="t"` fits a
Student-t degrees-of-freedom parameter (`nu`) alongside the variance
parameters by maximum likelihood; a low fitted `nu` (roughly < 10) is
itself a diagnostic that tails matter for that name.
```

```
"""
```

```
from __future__ import annotations
```

```
import math
```

```
from dataclasses import dataclass, field
```

```
import numpy as np
```

```
import pandas as pd
```

```
from scipy.optimize import minimize
```

```
from scipy.special import gammaln
```

```
TRADING_DAYS_PER_YEAR = 252
```

```
def close_to_close_vol(close: pd.Series, annualization_factor: int = TRADING_DAYS_PER_YEAR) -> float:
```

```

"""Annualized std of log returns – the simplest, noisiest, and most
common realized-vol estimator. Ignores everything except closing
prices, so it throws away the intraday range information the other two
estimators use."""
log_returns = np.log(close / close.shift(1)).dropna()
if len(log_returns) < 2:
    raise ValueError("Need at least 3 closing prices to estimate volatility.")
return float(log_returns.std(ddof=1) * math.sqrt(annualization_factor))

```

```

def parkinson_vol(high: pd.Series, low: pd.Series, annualization_factor: int = TRADING_DAYS_PER_YEAR) ->
float:

```

```

    """Parkinson (1980) range-based estimator:

```

```

        sigma^2 = (1 / (4*n*ln2)) * sum(ln(H_i/L_i)^2)

```

```

More efficient than close-to-close because the day's high/low range
carries information about volatility that happened between the close
prices. Assumes no overnight gaps/drift and continuous trading, so it
tends to underestimate vol on gappy names.
"""

```

```

if len(high) != len(low):
    raise ValueError("high and low must be the same length")
n = len(high)
if n < 1:
    raise ValueError("Need at least 1 day of high/low data.")
log_hl = np.log(high.to_numpy() / low.to_numpy())
variance = float(np.sum(log_hl**2) / (4 * n * math.log(2)))
return math.sqrt(variance * annualization_factor)

```

```

def garman_klass_vol(
    open_: pd.Series,
    high: pd.Series,
    low: pd.Series,
    close: pd.Series,
    annualization_factor: int = TRADING_DAYS_PER_YEAR,
) -> float:
    """Garman-Klass (1980) estimator – adds the open/close relationship on
top of Parkinson's high/low range:

```

```

        sigma^2 = (1/n) * sum[ 0.5*ln(H/L)^2 - (2*ln2 - 1)*ln(C/O)^2 ]

```

```

The most statistically efficient of the three here under the assumption
of continuous GBM with zero drift – but that assumption (no jumps, no
overnight drift) is exactly what real markets violate around earnings,
gaps, and trend days, so treat it as one estimate among several, not
ground truth.
"""

```

```

n = len(close)
if not (len(open_) == len(high) == len(low) == n):
    raise ValueError("open, high, low, close must all be the same length")
if n < 1:
    raise ValueError("Need at least 1 day of OHLC data.")

log_hl = np.log(high.to_numpy() / low.to_numpy())
log_co = np.log(close.to_numpy() / open_.to_numpy())
term = 0.5 * log_hl**2 - (2 * math.log(2) - 1) * log_co**2
variance = float(np.sum(term) / n)
if variance < 0:

```

```

        raise ValueError(
            "Garman-Klass variance came out negative – this can happen with "
            "very few, unusual-looking bars (e.g. 0=H=L=C). Use more data."
        )
    return math.sqrt(variance * annualization_factor)

@dataclass
class GarchFit:
    """A fitted GARCH(1,1):  $\sigma_t^2 = \omega + \alpha r_{t-1}^2 + \beta \sigma_{t-1}^2$ ."""

    omega: float
    alpha: float
    beta: float
    mean_return: float
    demeaned_returns: np.ndarray = field(repr=False)
    conditional_variance: np.ndarray = field(repr=False) # fitted  $\sigma_t^2$ , same length as demeaned_returns

    @property
    def long_run_variance(self) -> float:
        """ $\omega / (1 - \alpha - \beta)$  – the variance GARCH reverts to as the
        forecast horizon grows, given  $\alpha + \beta < 1$  (stationarity)."""
        return self.omega / (1 - self.alpha - self.beta)

    @property
    def persistence(self) -> float:
        return self.alpha + self.beta

def fit_garch_11(returns: pd.Series) -> GarchFit:
    """Fit GARCH(1,1) by maximum likelihood (Gaussian conditional density).

    Returns are demeaned first – GARCH models the variance of the
    *unexpected* return, not the level. The negative log-likelihood is:

    
$$NLL = 0.5 * \sum ( \ln(2\pi\sigma_t^2) + r_t^2 / \sigma_t^2 )$$


    minimized subject to  $\omega > 0$ ,  $\alpha \geq 0$ ,  $\beta \geq 0$ ,  $\alpha + \beta < 1$ 
    (the stationarity condition – without it the variance process doesn't
    revert to a long-run level and 'annualized vol' stops being meaningful).
    """
    r = returns.dropna().to_numpy(dtype=float)
    if len(r) < 30:
        raise ValueError("GARCH(1,1) needs a reasonably long return history (30+ points) to fit sensibly.")
    mean_r = float(r.mean())
    r = r - mean_r
    sample_var = float(r.var(ddof=1))

    def neg_log_likelihood(params: np.ndarray) -> float:
        omega, alpha, beta = params
        if omega <= 0 or alpha < 0 or beta < 0 or alpha + beta >= 1:
            return 1e10 # infeasible region – push the optimizer away
        n = len(r)
        sigma2 = np.empty(n)
        sigma2[0] = sample_var
        for t in range(1, n):
            sigma2[t] = omega + alpha * r[t - 1]**2 + beta * sigma2[t - 1]
        sigma2 = np.maximum(sigma2, 1e-12) # guard log(0) for numerically degenerate params
        return 0.5 * float(np.sum(np.log(2 * np.pi * sigma2) + r**2 / sigma2))

```

```

initial_guess = [sample_var * 0.05, 0.05, 0.90]
bounds = [(1e-12, None), (0.0, 0.999), (0.0, 0.999)]
result = minimize(neg_log_likelihood, initial_guess, bounds=bounds, method="L-BFGS-B")
omega, alpha, beta = result.x

# Recompute the fitted variance series once more at the optimum so we
# can return it (avoids re-deriving it from scratch at forecast time).
n = len(r)
sigma2 = np.empty(n)
sigma2[0] = sample_var
for t in range(1, n):
    sigma2[t] = omega + alpha * r[t - 1] ** 2 + beta * sigma2[t - 1]

return GarchFit(omega=omega, alpha=alpha, beta=beta, mean_return=mean_r, demeaned_returns=r,
conditional_variance=sigma2)

def garch_forecast_vol(fit: GarchFit, horizon_days: int = 1, annualization_factor: int =
TRADING_DAYS_PER_YEAR) -> float:
    """h-day-ahead annualized vol forecast from a fitted GARCH(1,1).

    One-step:  $\sigma^2_{t+1} = \omega + \alpha r_t^2 + \beta \sigma_t^2$ 
    Multi-step: the forecast mean-reverts geometrically toward the long-run
    variance at rate  $(\alpha + \beta)$  per day:

        
$$\sigma^2_{t+h} = LRV + (\alpha + \beta)^{h-1} * (\sigma^2_{t+1} - LRV)$$


    As horizon_days -> infinity this converges to the long-run variance -
    i.e. GARCH's forecast of "how volatile will it be eventually" is just
    the unconditional variance, regardless of today's conditions.
    """
    if horizon_days < 1:
        raise ValueError("horizon_days must be >= 1")

    last_sigma2 = fit.conditional_variance[-1]
    last_return = fit.demeaned_returns[-1]
    one_step_var = fit.omega + fit.alpha * last_return**2 + fit.beta * last_sigma2

    lrv = fit.long_run_variance
    if horizon_days == 1:
        horizon_var = one_step_var
    else:
        horizon_var = lrv + (fit.persistence ** (horizon_days - 1)) * (one_step_var - lrv)

    return math.sqrt(max(horizon_var, 0.0) * annualization_factor)

```

risk_tool/hedge.py

Beta-hedge sizing for any position (shares or options) against any correlated instrument (an ETF, or futures via a contract multiplier). Built around one idea that's easy to get backwards under pressure: a position's exposure to the hedge instrument is its DELTA-adjusted exposure, not its notional — an option moves less than 1:1 with the underlying, so hedge sizing has to account for that. `estimate_beta()` runs an OLS regression of the underlying's returns on the hedge instrument's returns and reports R^2 alongside beta, specifically so the hedge's limitation is visible: it only offsets the portion of the move that R^2 actually explains, not the idiosyncratic remainder — which, for a directional options bet, is usually the entire point of the trade.

```
"""Beta-hedge sizing: offset a directional position (shares or delta-bearing
options) with a correlated hedge instrument — an ETF, or a futures contract
via its per-unit contract multiplier.
```

```
The core idea: a position's exposure to the hedge instrument isn't its
notional, it's its DELTA-adjusted exposure (shares held as-is; delta *
contracts * shares_per_contract for options, since an option moves less
than 1:1 with the underlying). Convert that to a dollar amount, scale by
beta (the regression slope of the underlying's returns on the hedge
instrument's returns), and that's the dollar amount of hedge instrument to
take the *opposite* side of the position's implied hedge-instrument
exposure.
```

```
What this does NOT do:
```

- Offset theta or vega on an options position — only directional (delta) risk correlated with the hedge instrument.
- Guarantee a profitable trade. It reduces the correlated portion of the P&L swing; the position can still lose money on its idiosyncratic (non-hedge-explained) move, which R^2 tells you the size of.
- Stay hedged over time without rebalancing. Option delta changes with spot/time (gamma/theta), and beta itself is a rolling-window estimate that drifts — both mean this hedge decays and needs periodic recomputation, not a "set and forget" position.

```
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
import numpy as np
import pandas as pd
```

```
@dataclass
```

```
class BetaEstimate:
```

```
    beta: float # OLS slope: underlying_return ~= beta * hedge_return
    r_squared: float # fraction of the underlying's return variance the hedge instrument explains
    n_obs: int
```

```
def simple_returns(prices: pd.Series) -> pd.Series:
```

```
    """Period-over-period % change, first (NaN) row dropped."""
    return prices.pct_change().dropna()
```

```

def estimate_beta(underlying_returns: pd.Series, hedge_returns: pd.Series) -> BetaEstimate:
    """OLS slope of underlying_returns on hedge_returns, plus R².

    Aligns on index (so callers can pass returns computed independently,
    e.g. from two get_equity_historical() pulls on different calendars)
    and only uses overlapping observations.

    This is a rolling-window estimate over whatever history you feed it –
    re-run it periodically rather than sizing a hedge once and forgetting
    it; beta over the last month and beta over the last year routinely
    disagree.
    """
    idx = underlying_returns.dropna().index.intersection(hedge_returns.dropna().index)
    u = underlying_returns.loc[idx].to_numpy(dtype=float)
    h = hedge_returns.loc[idx].to_numpy(dtype=float)
    n = len(u)
    if n < 3:
        raise ValueError(f"Need at least 3 overlapping return observations to estimate beta, got {n}.")

    h_mean, u_mean = h.mean(), u.mean()
    cov = float(np.sum((h - h_mean) * (u - u_mean)))
    var_h = float(np.sum((h - h_mean) ** 2))
    var_u = float(np.sum((u - u_mean) ** 2))
    if var_h == 0:
        raise ValueError("Hedge instrument returns have zero variance – can't estimate beta.")

    beta = cov / var_h
    r_squared = (cov**2) / (var_u * var_h) if var_u > 0 else 0.0
    return BetaEstimate(beta=beta, r_squared=r_squared, n_obs=n)


def option_position_exposure_shares(delta: float, contracts: float, shares_per_contract: float = 100.0) -> float:
    """Delta-equivalent signed share exposure of an option position.

    `delta` carries the sign your broker quotes it with – negative for a
    long put or short call, positive for a long call or short put. Pass a
    negative `contracts` for a short options position if your delta is
    already signed for a long position (or just sign delta yourself,
    whichever is less error-prone for the caller)."""
    return delta * contracts * shares_per_contract


def share_position_exposure_shares(quantity: float, side: str) -> float:
    """side: 'long' or 'short'. quantity should be positive (unsigned)."""
    side = side.lower()
    if side not in ("long", "short"):
        raise ValueError(f"side must be 'long' or 'short', got {side!r}")
    if quantity < 0:
        raise ValueError("quantity must be non-negative – use side to set direction.")
    return quantity if side == "long" else -quantity


@dataclass
class HedgeSizeResult:
    exposure_shares: float # signed, delta-equivalent shares of the underlying
    exposure_dollars: float # signed
    hedge_dollars: float # signed; positive = go long the hedge instrument, negative = go short

```

```

    hedge_units: float # signed; ETF/stock shares (multiplier=1) or futures contracts
    (multiplier=barrels/contract)
    direction: str # "long" or "short" – which side of the hedge instrument to take

```

```

def size_hedge(
    exposure_shares: float,
    underlying_price: float,
    beta: float,
    hedge_price: float,
    hedge_contract_multiplier: float = 1.0,
) -> HedgeSizeResult:
    """The one function to call from outside this module for sizing.

    hedge_dollars = -(exposure_shares * underlying_price) * beta
    hedge_units   = hedge_dollars / (hedge_price * hedge_contract_multiplier)

```

A net-short position (negative exposure_shares) with positive beta produces positive hedge_dollars – i.e. buy/long the hedge instrument, since the short position already carries implicit short exposure to anything positively correlated with the underlying. Flip the position to net-long, or use a negative beta, and the sign flips accordingly – read `direction` / the sign of `hedge\_units` rather than reasoning about it by hand each time.

hedge_contract_multiplier: 1.0 for an ETF/stock hedge (hedge_units is then a plain share count). For a futures hedge, pass the contract's units-per-contract (e.g. 1000 for CME WTI/CL, 100 for Micro WTI/MCL) with hedge_price quoted per unit (per barrel) – hedge_units then comes out in contracts.

```

"""
if underlying_price <= 0:
    raise ValueError("underlying_price must be positive")
if hedge_price <= 0:
    raise ValueError("hedge_price must be positive")
if hedge_contract_multiplier <= 0:
    raise ValueError("hedge_contract_multiplier must be positive")

exposure_dollars = exposure_shares * underlying_price
hedge_dollars = -exposure_dollars * beta
hedge_units = hedge_dollars / (hedge_price * hedge_contract_multiplier)

return HedgeSizeResult(
    exposure_shares=exposure_shares,
    exposure_dollars=exposure_dollars,
    hedge_dollars=hedge_dollars,
    hedge_units=hedge_units,
    direction="long" if hedge_units >= 0 else "short",
)

```

```

def option_intrinsic_pl(
    spot_at_scenario: float,
    strike: float,
    premium: float,
    option_type: str,
    contracts: float,
    shares_per_contract: float = 100.0,
) -> float:
    """P&L at expiration (intrinsic value only – no remaining time value) for

```

```

an option position. Use a negative `contracts` for a short position;
`premium` should be signed the same way you paid/received it (positive
for what you paid on a long position)."""
option_type = option_type.lower()
if option_type not in ("call", "put"):
    raise ValueError(f"option_type must be 'call' or 'put', got {option_type!r}")
intrinsic = max(spot_at_scenario - strike, 0.0) if option_type == "call" else max(strike -
spot_at_scenario, 0.0)
return (intrinsic - premium) * contracts * shares_per_contract

def share_position_pl(price_at_scenario: float, entry_price: float, exposure_shares: float) -> float:
    """P&L for a share position. exposure_shares is signed (+ long, - short),
    as returned by share_position_exposure_shares."""
    return exposure_shares * (price_at_scenario - entry_price)

def hedge_instrument_pl(hedge_price_at_scenario: float, hedge_price_now: float, hedge_units: float) -> float:
    """P&L for the hedge leg. hedge_units is signed (+ long, - short), as
    returned by size_hedge - pass it straight through."""
    return hedge_units * (hedge_price_at_scenario - hedge_price_now)

```

risk_tool/ibkr_client.py

An optional, entirely separate live-data source (Interactive Brokers via `ib_insync`) that nothing else in `risk_tool` depends on or imports — every model in the package runs on manual inputs. `MockIBKRClient` implements the identical interface with deterministic fake data, so any code that consumes an IBKR client can be exercised in tests without TWS or IB Gateway running at all. `ib_insync` itself is imported lazily inside each method rather than at module load, so importing this file never fails just because IBKR isn't installed or configured — it only matters once you call `connect()`.

```
"""Thin wrapper around ib_insync for live IBKR quotes and option position
monitoring.

OPTIONAL — nothing in pricing.py, greeks.py, sizing.py, risk_manager.py, or
strike_selection.py imports this module or requires a live connection.
Every core model runs entirely on manual inputs. This module exists purely
to feed those models live numbers instead of typing them in by hand, and to
let risk_manager watch your real IBKR positions if you want that.

MockIBKRClient implements the same interface with deterministic fake data,
so the CLI/dashboard code path can be exercised and tested without TWS or
IB Gateway running at all.
"""
from __future__ import annotations

from dataclasses import dataclass

@dataclass
class LiveQuote:
    symbol: str
    bid: float
    ask: float
    last: float
    mark: float # (bid + ask) / 2, same convention used elsewhere in this project

@dataclass
class LiveOptionPosition:
    symbol: str
    option_type: str # "call" or "put"
    strike: float
    expiration: str # "YYYY-MM-DD"
    quantity: int
    avg_cost: float # per-contract entry cost
    current_price: float # per-contract mark; 0.0 if not available (see IBKRClient docstring)
    delta: float | None
    implied_vol: float | None

class MockIBKRClient:
    """Deterministic fake data for development and tests. Configure it with
    whatever quotes/positions your test needs; it never touches a network."""
```

```

def __init__(
    self,
    quotes: dict[str, LiveQuote] | None = None,
    positions: list[LiveOptionPosition] | None = None,
):
    self._quotes = quotes or {}
    self._positions = positions or []
    self.connected = False

def connect(self) -> None:
    self.connected = True

def disconnect(self) -> None:
    self.connected = False

def get_quote(self, symbol: str) -> LiveQuote:
    if symbol not in self._quotes:
        raise KeyError(f"No mock quote configured for {symbol!r}")
    return self._quotes[symbol]

def get_option_positions(self) -> list[LiveOptionPosition]:
    return list(self._positions)

class IBKRClient:
    """Real ib_insync-backed client. Requires TWS or IB Gateway running
    locally with the API enabled – default port 7497 is IB's paper-trading
    port; use 7496 for a live account, at your own risk.

    ib_insync is imported lazily inside each method rather than at module
    load time, so importing risk_tool.ibkr_client never fails just because
    you haven't set up IBKR – it only matters once you actually call
    connect().
    """

    def __init__(self, host: str = "127.0.0.1", port: int = 7497, client_id: int = 17):
        self.host = host
        self.port = port
        self.client_id = client_id
        self._ib = None

    def connect(self) -> None:
        from ib_insync import IB

        self._ib = IB()
        self._ib.connect(self.host, self.port, clientId=self.client_id)

    def disconnect(self) -> None:
        if self._ib is not None:
            self._ib.disconnect()

    def _require_connection(self) -> None:
        if self._ib is None or not self._ib.isConnected():
            raise RuntimeError("Not connected to IBKR – call connect() first (requires TWS/IB Gateway running).")

    def get_quote(self, symbol: str) -> LiveQuote:
        from ib_insync import Stock

        self._require_connection()

```

```

contract = Stock(symbol, "SMART", "USD")
self._ib.qualifyContracts(contract)
ticker = self._ib.reqMktData(contract, "", False, False)
self._ib.sleep(1) # ib_insync needs a beat for the snapshot fields to populate
bid, ask, last = ticker.bid, ticker.ask, ticker.last
mark = (bid + ask) / 2 if bid and ask else last
return LiveQuote(symbol=symbol, bid=bid, ask=ask, last=last, mark=mark)

def get_option_positions(self) -> list[LiveOptionPosition]:
    """NOTE: current_price/delta/IMPLIED_VOL are left unset (None/0.0)
    here - filling them in requires a live market-data subscription per
    option contract (reqMktData with the options generic tick list),
    which is a meaningfully bigger integration than position listing
    and isn't wired up yet. Feed those fields from the dashboard's
    existing Robinhood option-chain data instead in the meantime, or
    extend this method if you have the IBKR options market data
    subscription needed to fill them in properly.
    """

    self._require_connection()
    positions: list[LiveOptionPosition] = []
    for pos in self._ib.positions():
        contract = pos.contract
        if contract.secType != "OPT":
            continue
        positions.append(
            LiveOptionPosition(
                symbol=contract.symbol,
                option_type="call" if contract.right == "C" else "put",
                strike=contract.strike,
                expiration=contract.lastTradeDateOrContractMonth,
                quantity=int(pos.position),
                avg_cost=pos.avgCost / 100, # ib_insync reports option avgCost scaled by the 100x
                multiplier=
                    current_price=0.0,
                    delta=None,
                    implied_vol=None,
            )
        )
    return positions

```

risk_tool/cli.py

A complete, standalone command-line entry point (python -m risk_tool.cli) that exercises the entire pricing → strike ranking → sizing → exit-levels pipeline from manual flags, with zero dependency on Streamlit or a live broker connection. This is what proves risk_tool truly has no hidden coupling to the dashboard — the same functions the UI calls are directly callable from a terminal.

```
"""CLI entry point: manual-input strike selection, sizing, and entry/exit
levels. No live connection required – every input is a flag.
```

Example:

```
python -m risk_tool.cli --ticker AAPL --spot 190 --iv 0.32 --dte 30 \\  
--direction call --account 50000 --strike-increment 5
```

With your own vol view for the realized-vs-implied edge check:

```
python -m risk_tool.cli --ticker AAPL --spot 190 --iv 0.32 --dte 30 \\  
--direction call --account 50000 --strike-increment 5 --realized-vol 0.45
```

```
"""
```

```
from __future__ import annotations
```

```
import argparse  
import sys
```

```
from risk_tool.config import DEFAULT_CONFIG, RiskConfig  
from risk_tool.sizing import size_position  
from risk_tool.strike_selection import compare_implied_vs_realized_ev, evaluate_strike,  
generate_candidate_strikes
```

```
def build_arg_parser() -> argparse.ArgumentParser:  
    parser = argparse.ArgumentParser(description="Options risk-management and strike-selection tool.")  
    parser.add_argument("--ticker", required=True)  
    parser.add_argument("--spot", type=float, required=True, help="Current underlying price")  
    parser.add_argument("--iv", type=float, required=True, help="Market implied vol, e.g. 0.32 for 32%")  
    parser.add_argument("--dte", type=int, required=True, help="Days to expiry")  
    parser.add_argument("--direction", choices=["call", "put"], required=True)  
    parser.add_argument("--account", type=float, required=True, help="Account size in dollars")  
    parser.add_argument("--strike-increment", type=float, required=True, help="Strike spacing for this  
underlying")  
    parser.add_argument("--r", type=float, default=DEFAULT_CONFIG.risk_free_rate, help="Risk-free rate")  
    parser.add_argument("--q", type=float, default=DEFAULT_CONFIG.dividend_yield, help="Dividend yield")  
    parser.add_argument("--num-strikes", type=int, default=DEFAULT_CONFIG.num_strikes_each_side,  
help="Candidate strikes per side of ATM")  
    parser.add_argument("--profit-target-pct", type=float, default=DEFAULT_CONFIG.default_profit_target_pct,  
help="e.g. 1.0 for +100%")  
    parser.add_argument("--max-position-pct", type=float, default=DEFAULT_CONFIG.max_position_pct_of_account,  
help="Hard cap, e.g. 0.05 for 5% of account")  
    parser.add_argument("--full-kelly", action="store_true", help="Use full Kelly instead of the half-Kelly  
default")  
    parser.add_argument(  
        "--realized-vol",  
        type=float,  
        default=None,  
        help="Your own realized/forecast vol – enables the realized-vs-implied edge comparison and ranks by
```

```

it",
)
return parser

def _print_strike_row(rank: int, strike_ev, extra: dict | None = None) -> None:
    flag = " [POOR SETUP]" if strike_ev.is_poor_setup else ""
    print(
        f"    #{rank} K=${strike_ev.strike:<8.2f} premium=${strike_ev.premium:<7.2f} "
        f"P_win(risk-neutral)={strike_ev.p_win_risk_neutral:6.1%} EV=${strike_ev.ev:+7.2f} "
        f"R:R={strike_ev.risk_reward:5.2f}:1{flag}"
    )
    print(
        f"        delta={strike_ev.delta:+.3f} theta/day={strike_ev.theta:+.3f} vega={strike_ev.vega:.3f}"
    )
    if extra:
        for label, value in extra.items():
            print(f"        {label}: {value}")

def run(argv: list[str] | None = None) -> int:
    args = build_arg_parser().parse_args(argv)

    config = RiskConfig(
        risk_free_rate=args.r,
        dividend_yield=args.q,
        num_strikes_each_side=args.num_strikes,
        default_profit_target_pct=args.profit_target_pct,
        max_position_pct_of_account=args.max_position_pct,
        use_half_kelly=not args.full_kelly,
    )
    T = args.dte / 365.0
    strikes = generate_candidate_strikes(args.spot, args.strike_increment, config.num_strikes_each_side)

    print(f"\n{args.ticker} – spot ${args.spot:.2f}, {args.direction} thesis, {args.dte} DTE, IV {args.iv:.1%}")
    print(f"Config: profit target +{config.profit_target_pct:.0%}, stop {config.stop_loss_pct:.0%}, "
          f"'{half' if config.use_half_kelly else 'full'}-Kelly, hard cap "
          f"{config.max_position_pct_of_account:.0%} of account\n")

    if args.realized_vol is not None:
        print(f"Realized-vs-implied edge check: market IV {args.iv:.1%} vs. your vol {args.realized_vol:.1%}
"
            f"(spread {args.realized_vol - args.iv:+.1%})\n")
        comparisons = [
            compare_implied_vs_realized_ev(args.spot, K, T, config.risk_free_rate, config.dividend_yield,
            args.iv, args.realized_vol, args.direction, config)
            for K in strikes
        ]
        comparisons.sort(key=lambda c: c.edge_ev.ev, reverse=True)
        top = comparisons[:3]
        print("Top strikes by EDGE EV (your vol view, market's actual premium):")
        for i, c in enumerate(top, start=1):
            _print_strike_row(i, c.edge_ev, extra={
                "risk-neutral EV (market IV throughout)": f"${c.risk_neutral_ev.ev:+.2f}",
            })
        best_strike_ev = top[0].edge_ev
        best_p_win = best_strike_ev.p_win_risk_neutral
    else:
        ranked = sorted(

```

```

        (evaluate_strike(args.spot, K, T, config.risk_free_rate, config.dividend_yield, args.iv,
args.direction, config) for K in strikes),
        key=lambda s: s.ev,
        reverse=True,
    )
    top = ranked[:3]
    print("Top strikes by risk-neutral EV (market IV throughout – see strike_selection.py docstring on
what this does and doesn't mean):")
    for i, s in enumerate(top, start=1):
        _print_strike_row(i, s)
    best_strike_ev = top[0]
    best_p_win = best_strike_ev.p_win_risk_neutral

    print(f"\nRecommended strike: ${best_strike_ev.strike:.2f} {args.direction} @
${best_strike_ev.premium:.2f}\n")

    sizing = size_position(
        account_size=args.account,
        premium_per_contract=best_strike_ev.premium,
        p_win=best_p_win,
        profit_if_win_per_contract=best_strike_ev.profit_if_win,
        config=config,
    )
    print("Position sizing:")
    print(f" Kelly fraction (raw):           ${sizing.kelly_fraction_raw:+.3f}")
    print(f" Kelly fraction (used):           ${sizing.kelly_fraction_used:.3f}")
    print(f" Kelly-suggested size:             ${sizing.kelly_dollar_size:,.2f}")
    print(f" Hard cap ({config.max_position_pct_of_account:.0%} of account):
${sizing.hard_cap_dollar_size:,.2f}")
    print(f" >>> Recommended size:             ${sizing.recommended_dollar_size:,.2f}
({sizing.recommended_contracts} contracts)"
        f"{' [capped by hard limit]' if sizing.capped_by_hard_limit else ''}")

    entry = best_strike_ev.premium
    profit_target_price = entry * (1 + config.profit_target_pct)
    stop_price = entry * (1 + config.stop_loss_pct)
    print("\nPre-committed entry/exit levels:")
    print(f" Entry premium:                 ${entry:.2f}")
    print(f" Profit target (+{config.profit_target_pct:.0%}):      ${profit_target_price:.2f}")
    print(f" Stop loss ({config.stop_loss_pct:.0%}):              ${stop_price:.2f}")
    print(f" Delta exit if |delta| falls below {config.delta_exit_threshold:.2f}")
    print(f" Theta exit if DTE <= {config.theta_exit_dte_threshold} and still OTM")
    print(f" IV-crush exit if IV drops >= {config.iv_crush_threshold_points:.0%} points from entry
({args.iv:.1%})")
    print()
    return 0

def main() -> None:
    sys.exit(run())

if __name__ == "__main__":
    main()

```

`risk_tool/__init__.py`

Empty — just marks risk_tool as a package.

app/ — I/O and presentation, isolated from the models

app/auth.py

Robinhood login handling via robin_stocks, including TOTP-based unattended MFA. The one piece of logic worth understanding: `is_demo_mode()` checks both a plain environment variable (local `.env`) and Streamlit's `st.secrets` (Streamlit Community Cloud's secrets mechanism, which doesn't reliably mirror into `os.environ`) — and when demo mode is on, `ensure_logged_in()` short-circuits to `True` immediately, so a public deployment never shows a login form or touches Robinhood at all.

```
"""Robinhood login handling.

robin_stocks caches its own session token under ~/.tokens/robinhood.pickle,
so once you've logged in once with `store_session=True` (the default), most
days you won't be prompted for MFA again. We still support a TOTP secret in
.env for fully unattended re-auth (e.g. after the cached session expires).
"""

import os

import pyotp
import robin_stocks.robinhood as rh
import streamlit as st
from dotenv import load_dotenv

load_dotenv()

def is_demo_mode() -> bool:
    """True if DEMO_MODE is set, via either a local .env (os.environ, loaded
    above) or Streamlit Community Cloud's Secrets UI (st.secrets – Cloud does
    NOT reliably mirror secrets into os.environ, so os.getenv alone isn't
    enough there)."""
    if os.getenv("DEMO_MODE", "").strip().lower() == "true":
        return True
    try:
        return str(st.secrets.get("DEMO_MODE", "")).strip().lower() == "true"
    except Exception:
        return False

class LoginError(RuntimeError):
    pass

def _mfa_code() -> str | None:
    secret = os.getenv("ROBINHOOD_TOTP_SECRET", "").strip()
    if not secret:
        return None
    return pyotp.TOTP(secret).now()

def login(mfa_from_user: str | None = None) -> dict:
    """Log in to Robinhood. Returns robin_stocks' login payload.

    Raises LoginError with a user-facing message on failure so the Streamlit
    UI can show it instead of crashing.
```

```

"""
username = os.getenv("ROBINHOOD_USERNAME", "").strip()
password = os.getenv("ROBINHOOD_PASSWORD", "").strip()
if not username or not password:
    raise LoginError(
        "Set ROBINHOOD_USERNAME and ROBINHOOD_PASSWORD in your .env file "
        "(copy .env.example to .env first).")
    )

mfa_code = mfa_from_user or _mfa_code()

try:
    login_payload = rh.login(
        username=username,
        password=password,
        mfa_code=mfa_code,
        store_session=True,
    )
except Exception as exc: # robin_stocks raises bare Exceptions on auth failure
    raise LoginError(f"Robinhood login failed: {exc}") from exc

if not login_payload or "access_token" not in login_payload:
    raise LoginError(
        "Robinhood login did not return an access token – check your "
        "credentials, or enter the MFA code sent to you and try again."
    )
return login_payload

def logout() -> None:
    try:
        rh.logout()
    finally:
        for key in ("rh_logged_in",):
            st.session_state.pop(key, None)

def ensure_logged_in() -> bool:
    """Streamlit-friendly login gate. Returns True once authenticated.

    In DEMO_MODE there's no Robinhood account behind this at all – skip the
    gate entirely so a public deployment never prompts for (or needs) real
    credentials."""
    if is_demo_mode():
        return True
    if st.session_state.get("rh_logged_in"):
        return True

    st.subheader("Log in to Robinhood")
    st.caption(
        "Credentials are read from your local .env file and sent directly to "
        "Robinhood – nothing is stored anywhere else."
    )

    needs_manual_mfa = st.session_state.get("rh_needs_mfa", False)
    mfa_input = None
    if needs_manual_mfa:
        mfa_input = st.text_input("Enter the MFA/2FA code Robinhood sent you", key="mfa_code_input")

    if st.button("Log in", type="primary"):

```

```
try:
    login(mfa_from_user=mfa_input)
    st.session_state["rh_logged_in"] = True
    st.session_state["rh_needs_mfa"] = False
    st.rerun()
except LoginError as exc:
    msg = str(exc)
    if "verification" in msg.lower() or "mfa" in msg.lower():
        st.session_state["rh_needs_mfa"] = True
        st.rerun()
    else:
        st.error(msg)
return False
```

app/data_fetch.py

Every single robin_stocks call in the entire project lives here, and every function returns a plain pandas DataFrame or dict — never a Streamlit object, never raw robin_stocks response shapes. That isolation is what makes the rest of the app testable in principle and is exactly the seam demo_data.py plugs into as a drop-in replacement: same function names, same return shapes, zero Robinhood calls.

```
"""All Robinhood data pulls, isolated from Streamlit/UI concerns.

Every function returns plain pandas DataFrames (or dicts for single-value
summaries) so app.py stays presentation-only and this module is unit
testable without a live session.
"""
from __future__ import annotations

import os
from concurrent.futures import ThreadPoolExecutor
from datetime import datetime, timezone

import pandas as pd
import robin_stocks.robinhood as rh

def get_vol_tickers() -> list[str]:
    raw = os.getenv("VOL_TICKERS", "VXX,UVXY,SVXY,VIXY,UVIX,SVIX")
    return [t.strip().upper() for t in raw.split(",") if t.strip()]

def get_portfolio_overview() -> dict:
    profile = rh.profiles.load_portfolio_profile() or {}
    account = rh.profiles.load_account_profile() or {}

    equity = float(profile.get("equity") or 0)
    prev_equity = float(profile.get("equity_previous_close") or equity)
    day_pl = equity - prev_equity
    day_pl_pct = (day_pl / prev_equity * 100) if prev_equity else 0.0

    return {
        "equity": equity,
        "cash": float(account.get("cash") or profile.get("withdrawable_amount") or 0),
        "buying_power": float(account.get("buying_power") or 0),
        "day_pl": day_pl,
        "day_pl_pct": day_pl_pct,
        "market_value": float(profile.get("market_value") or 0),
    }

def get_portfolio_history(span: str = "month", interval: str = "day") -> pd.DataFrame:
    hist = rh.account.get_historical_portfolio(interval=interval, span=span) or {}
    equities = hist.get("equity_historicals", [])
    if not equities:
        return pd.DataFrame(columns=["date", "equity"])
    df = pd.DataFrame(equities)
    df["date"] = pd.to_datetime(df["begins_at"])
```

```

df["equity"] = df["adjusted_close_equity"].astype(float)
return df[["date", "equity"]]

def get_equity_positions() -> pd.DataFrame:
    positions = rh.account.get_open_stock_positions() or []
    rows = []
    for pos in positions:
        qty = float(pos.get("quantity") or 0)
        if qty == 0:
            continue
        instrument_url = pos.get("instrument")
        symbol = rh.stocks.get_symbol_by_url(instrument_url) if instrument_url else pos.get("symbol")
        quote = rh.stocks.get_latest_price(symbol)[0] if symbol else None
        last_price = float(quote) if quote else 0.0
        avg_cost = float(pos.get("average_buy_price") or 0)
        market_value = qty * last_price
        cost_basis = qty * avg_cost
        rows.append(
            {
                "symbol": symbol,
                "quantity": qty,
                "avg_cost": avg_cost,
                "last_price": last_price,
                "market_value": market_value,
                "unrealized_pl": market_value - cost_basis,
                "unrealized_pl_pct": ((last_price - avg_cost) / avg_cost * 100) if avg_cost else 0.0,
                "is_vol_ticker": symbol in get_vol_tickers(),
            }
        )
    return pd.DataFrame(rows)

def _safe_float(value, default=0.0):
    try:
        return float(value)
    except (TypeError, ValueError):
        return default

def get_option_positions() -> pd.DataFrame:
    positions = rh.options.get_open_option_positions() or []
    rows = []
    for pos in positions:
        qty = _safe_float(pos.get("quantity"))
        if qty == 0:
            continue

        instrument_data = rh.options.get_option_instrument_data_by_id(pos["option_id"]) or {}
        symbol = pos.get("chain_symbol")
        strike = _safe_float(instrument_data.get("strike_price"))
        exp_date = instrument_data.get("expiration_date")
        opt_type = instrument_data.get("type") # call / put
        side = pos.get("type") # long / short

        market_data_list = rh.options.get_option_market_data_by_id(pos["option_id"]) or []
        market_data = market_data_list[0] if market_data_list else {}

        mark_price = _safe_float(market_data.get("mark_price"))
        avg_price = _safe_float(pos.get("average_price")) / 100 # RH stores per-contract *100

```

```

multiplier = 100
signed_qty = qty if side == "long" else -qty

dte = None
if exp_date:
    try:
        dte = (datetime.strptime(exp_date, "%Y-%m-%d").date() -
datetime.now(timezone.utc).date()).days
    except ValueError:
        dte = None

rows.append(
    {
        "symbol": symbol,
        "type": opt_type,
        "side": side,
        "strike": strike,
        "expiration": exp_date,
        "dte": dte,
        "quantity": qty,
        "avg_price": avg_price,
        "mark_price": mark_price,
        "market_value": mark_price * qty * multiplier,
        "unrealized_pl": (mark_price - avg_price) * signed_qty * multiplier,
        "delta": _safe_float(market_data.get("delta")) * signed_qty * multiplier,
        "theta": _safe_float(market_data.get("theta")) * signed_qty * multiplier,
        "vega": _safe_float(market_data.get("vega")) * signed_qty * multiplier,
        "gamma": _safe_float(market_data.get("gamma")) * signed_qty * multiplier,
        "implied_volatility": _safe_float(market_data.get("implied_volatility")),
    }
)
return pd.DataFrame(rows)

def get_open_orders() -> pd.DataFrame:
    equity_orders = rh.orders.get_all_open_stock_orders() or []
    option_orders = rh.orders.get_all_open_option_orders() or []

    rows = []
    for o in equity_orders:
        symbol = rh.stocks.get_symbol_by_url(o.get("instrument")) if o.get("instrument") else None
        rows.append(
            {
                "instrument_type": "equity",
                "symbol": symbol,
                "side": o.get("side"),
                "quantity": _safe_float(o.get("quantity")),
                "price": _safe_float(o.get("price")),
                "state": o.get("state"),
                "created_at": o.get("created_at"),
            }
        )
    for o in option_orders:
        rows.append(
            {
                "instrument_type": "option",
                "symbol": o.get("chain_symbol"),
                "side": o.get("direction"),
                "quantity": _safe_float(o.get("quantity")),
                "price": _safe_float(o.get("price")),
            }
        )

```

```

        "state": o.get("state"),
        "created_at": o.get("created_at"),
    }
)
return pd.DataFrame(rows)

def get_order_history(days_back: int = 90) -> pd.DataFrame:
    """Normalized fill history across equities + options, for the journal."""
    equity_orders = rh.orders.get_all_stock_orders() or []
    option_orders = rh.orders.get_all_option_orders() or []

    cutoff = pd.Timestamp.now(tz="UTC") - pd.Timedelta(days=days_back)
    rows = []

    for o in equity_orders:
        if o.get("state") != "filled":
            continue
        created = pd.to_datetime(o.get("created_at"), utc=True, errors="coerce")
        if pd.isna(created) or created < cutoff:
            continue
        symbol = rh.stocks.get_symbol_by_url(o.get("instrument")) if o.get("instrument") else None
        qty = _safe_float(o.get("cumulative_quantity") or o.get("quantity"))
        avg_price = _safe_float(o.get("average_price"))
        rows.append(
            {
                "date": created,
                "instrument_type": "equity",
                "symbol": symbol,
                "side": o.get("side"),
                "quantity": qty,
                "price": avg_price,
                "amount": qty * avg_price * (1 if o.get("side") == "sell" else -1),
                "fees": _safe_float(o.get("fees") or 0),
                "order_id": o.get("id"),
            }
        )

    for o in option_orders:
        if o.get("state") != "filled":
            continue
        created = pd.to_datetime(o.get("created_at"), utc=True, errors="coerce")
        if pd.isna(created) or created < cutoff:
            continue
        qty = _safe_float(o.get("processed_quantity") or o.get("quantity"))
        avg_price = _safe_float(o.get("processed_premium") or o.get("price"))
        legs = o.get("legs") or [{}]
        opening = (o.get("opening_strategy") or "").strip() != ""
        rows.append(
            {
                "date": created,
                "instrument_type": "option",
                "symbol": o.get("chain_symbol"),
                "side": legs[0].get("side") if legs else o.get("direction"),
                "quantity": qty,
                "price": avg_price,
                "amount": qty * avg_price * 100 * (1 if (legs[0].get("side") if legs else None) == "sell"
else -1),
                "fees": 0.0,
                "order_id": o.get("id"),
            }
        )

```

```

        "strategy": "opening" if opening else "closing",
    }
)

df = pd.DataFrame(rows)
if df.empty:
    return df
return df.sort_values("date", ascending=False).reset_index(drop=True)

def get_underlying_price(symbol: str) -> float:
    price = rh.stocks.get_latest_price(symbol)
    return _safe_float(price[0]) if price else 0.0

def get_stock_quote(symbol: str) -> dict:
    """Best bid/ask for the underlying plus a mark price computed as their
    midpoint – Robinhood's stock quote endpoint has no mark field of its own
    (unlike options), so we derive it the same way brokers do."""
    quotes = rh.stocks.get_quotes(symbol) or []
    q = quotes[0] if quotes else {}

    bid = _safe_float(q.get("bid_price"))
    ask = _safe_float(q.get("ask_price"))
    last = _safe_float(q.get("last_trade_price"))
    mark = (bid + ask) / 2 if bid and ask else last

    return {"symbol": symbol, "bid": bid, "ask": ask, "mark": mark, "last_trade_price": last}

def get_option_chain_expirations(symbol: str) -> list[str]:
    chain = rh.options.get_chains(symbol) or {}
    return sorted(chain.get("expiration_dates") or [])

def get_option_chain_skew(symbol: str, expiration_date: str) -> pd.DataFrame:
    """Full call/put chain for one expiration, with IV/greeks per strike.

    Zero-IV rows (no live quote) are dropped – they're dead strikes, not
    real skew data, and would just show up as noise pinned to the x-axis.

    A single expiration can have 300+ contracts, and Robinhood's API needs
    one HTTP round-trip per contract for market data – done sequentially
    that's 30-40+ seconds. These calls are independent and I/O-bound (each
    thread is just waiting on the network), so a thread pool cuts this to a
    few seconds without changing any of the per-contract logic below.
    """
    contracts = rh.options.find_tradable_options(symbol, expirationDate=expiration_date, optionType=None) or []
    contract_ids = [c["id"] for c in contracts if c.get("id")]

    def fetch_market_data(contract_id: str) -> dict:
        market_data_list = rh.options.get_option_market_data_by_id(contract_id) or []
        return market_data_list[0] if market_data_list else {}

    with ThreadPoolExecutor(max_workers=20) as executor:
        market_data_by_id = dict(zip(contract_ids, executor.map(fetch_market_data, contract_ids)))

    rows = []
    for contract in contracts:

```

```

        contract_id = contract.get("id")
        if not contract_id:
            continue
        md = market_data_by_id.get(contract_id, {})
        iv = _safe_float(md.get("implied_volatility"))
        if iv <= 0:
            continue
        rows.append(
            {
                "strike": _safe_float(contract.get("strike_price")),
                "type": contract.get("type"),
                "iv": iv,
                "delta": _safe_float(md.get("delta")),
                "bid": _safe_float(md.get("bid_price")),
                "ask": _safe_float(md.get("ask_price")),
                "volume": _safe_float(md.get("volume")),
                "open_interest": _safe_float(md.get("open_interest")),
            }
        )
    df = pd.DataFrame(rows)
    if df.empty:
        return df

    df["mid"] = (df["bid"] + df["ask"]) / 2
    df["spread"] = df["ask"] - df["bid"]
    df["spread_pct"] = (df["spread"] / df["mid"]).replace(0, pd.NA) * 100).fillna(0.0)

    return df.sort_values(["type", "strike"]).reset_index(drop=True)

```

```

def get_equity_historical(symbol: str, interval: str = "day", span: str = "year") -> pd.DataFrame:
    """Daily OHLC history, used to feed risk_tool.realized_vol (close-to-close,
    Parkinson, Garman-Klass, GARCH all need real price history, not just a quote)."""
    bars = rh.stocks.get_stock_historical(symbol, interval=interval, span=span, bounds="regular") or []
    rows = [
        {
            "date": bar.get("begins_at"),
            "open": _safe_float(bar.get("open_price")),
            "high": _safe_float(bar.get("high_price")),
            "low": _safe_float(bar.get("low_price")),
            "close": _safe_float(bar.get("close_price")),
        }
        for bar in bars
    ]
    df = pd.DataFrame(rows)
    if df.empty:
        return df
    df["date"] = pd.to_datetime(df["date"])
    return df.sort_values("date").reset_index(drop=True)

```

```

def get_vol_ticker_quotes() -> pd.DataFrame:
    tickers = get_vol_tickers()
    if not tickers:
        return pd.DataFrame()
    quotes = rh.stocks.get_quotes(tickers) or []
    rows = []
    for t, q in zip(tickers, quotes):
        if not q:
            continue

```

```
last = _safe_float(q.get("last_trade_price"))
prev_close = _safe_float(q.get("previous_close"))
rows.append(
    {
        "symbol": t,
        "last_price": last,
        "prev_close": prev_close,
        "change_pct": ((last - prev_close) / prev_close * 100) if prev_close else 0.0,
    }
)
return pd.DataFrame(rows)
```

app/demo_data.py

A deterministic, synthetic stand-in for `data_fetch.py`, built specifically so this dashboard could be deployed publicly without ever touching a real brokerage account. Every price series is generated from a one-factor return model — a shared 'market factor' plus a per-symbol beta and idiosyncratic noise, both seeded from a hash of the ticker so results are stable across reruns but still show plausible cross-symbol correlation. The one deliberate touch: option prices and Greeks in this fake data are computed by calling the REAL `risk_tool.pricing/greeks` Black-Scholes engine — so the numbers aren't hand-picked, they're internally consistent outputs of the actual pricing model, just fed a synthetic spot and vol instead of a live quote.

```
"""Synthetic data for DEMO_MODE — a drop-in replacement for data_fetch.py with
the exact same function names and return shapes, so dashboard.py doesn't know
or care which one it's talking to.

No network calls, no Robinhood account, nothing real. Every number here is
generated deterministically from a seed derived from the symbol name (so the
same ticker always produces the same synthetic chart/price across reruns,
which matters for a demo someone might click around in more than once) plus
one shared "market factor" all symbols partially load onto, so cross-symbol
correlation in the Correlation Explorer looks like something real rather than
either 0 or 1.

Option prices/Greeks are computed with risk_tool's actual Black-Scholes engine
(not hand-picked), so the numbers you see are internally consistent — this
part of the demo is genuinely running the same math as the real thing, just
fed a synthetic spot/vol instead of a live quote.
"""
from __future__ import annotations

import hashlib
import math
import os
from datetime import datetime, timedelta, timezone

import numpy as np
import pandas as pd

from risk_tool.greeks import all_greeks
from risk_tool.pricing import black_scholes_price

RISK_FREE_RATE = 0.05
DIVIDEND_YIELD = 0.0
MARKET_SEED = 20260101 # fixed — the shared factor every symbol partially loads onto

def _seed(symbol: str, salt: str = "") -> int:
    return int(hashlib.sha256(f"{symbol}:{salt}".encode()).hexdigest(), 16) % (2**32)

def get_vol_tickers() -> list[str]:
    raw = os.getenv("VOL_TICKERS", "VXX,UVXY,SVXY,VIXY,UVIX,SVIX")
    return [t.strip().upper() for t in raw.split(",") if t.strip()]
```

```

def _market_factor(n_days: int) -> np.ndarray:
    rng = np.random.default_rng(MARKET_SEED)
    return rng.normal(0.0003, 0.010, n_days)

def _price_series(symbol: str, n_days: int = 260) -> pd.DataFrame:
    """Deterministic synthetic daily OHLC via a one-factor return model:
    daily_return = drift + beta * market_factor + idiosyncratic_noise."""
    rng = np.random.default_rng(_seed(symbol, "series"))
    base_price = float(rng.uniform(25, 420))
    beta = float(rng.uniform(0.2, 1.5))
    idio_vol = float(rng.uniform(0.008, 0.022))
    drift = float(rng.uniform(-0.0004, 0.0007))

    market = _market_factor(n_days)
    idio = rng.normal(0.0, idio_vol, n_days)
    daily_returns = drift + beta * market + idio

    closes = base_price * np.cumprod(1 + daily_returns)
    intraday_range = np.abs(rng.normal(0.0, idio_vol * 0.6, n_days))
    highs = closes * (1 + intraday_range)
    lows = closes * (1 - intraday_range)
    opens = np.concatenate([[base_price], closes[:-1]])

    end = datetime.now(timezone.utc).date()
    dates = pd.bdate_range(end=end, periods=n_days)

    return pd.DataFrame(
        {"date": pd.to_datetime(dates), "open": opens, "high": highs, "low": lows, "close": closes}
    )

def get_underlying_price(symbol: str) -> float:
    return float(_price_series(symbol, n_days=5)["close"].iloc[-1])

def get_stock_quote(symbol: str) -> dict:
    rng = np.random.default_rng(_seed(symbol, "quote"))
    last = get_underlying_price(symbol)
    spread = last * float(rng.uniform(0.0005, 0.002))
    bid, ask = last - spread / 2, last + spread / 2
    return {"symbol": symbol, "bid": bid, "ask": ask, "mark": (bid + ask) / 2, "last_trade_price": last}

def get_equity_historical(symbol: str, interval: str = "day", span: str = "year") -> pd.DataFrame:
    n_days = {"month": 22, "3month": 65, "year": 260, "5year": 1260}.get(span, 260)
    return _price_series(symbol, n_days=n_days)

def get_portfolio_overview() -> dict:
    equity = 128_450.32
    prev_equity = 127_209.77
    day_pl = equity - prev_equity
    return {
        "equity": equity,
        "cash": 18_220.10,
        "buying_power": 36_440.20,
        "day_pl": day_pl,
        "day_pl_pct": day_pl / prev_equity * 100,
    }

```

```

        "market_value": 110_230.22,
    }

def get_portfolio_history(span: str = "month", interval: str = "day") -> pd.DataFrame:
    rng = np.random.default_rng(MARKET_SEED + 1)
    n_days = 30
    end_equity = 128_450.32
    daily_returns = rng.normal(0.0012, 0.006, n_days)
    factors = np.cumprod(1 + daily_returns[:-1])[:-1]
    equity_path = end_equity / factors
    dates = pd.bdate_range(end=datetime.now(timezone.utc).date(), periods=n_days)
    return pd.DataFrame({"date": pd.to_datetime(dates), "equity": equity_path})

_EQUITY_BOOK = [
    {"symbol": "XOM", "quantity": 300.0, "cost_mult": 0.94, "is_vol_ticker": False},
    {"symbol": "AAPL", "quantity": 150.0, "cost_mult": 1.06, "is_vol_ticker": False},
    {"symbol": "VXX", "quantity": 500.0, "cost_mult": 1.18, "is_vol_ticker": True},
]

def get_equity_positions() -> pd.DataFrame:
    rows = []
    for pos in _EQUITY_BOOK:
        last_price = get_underlying_price(pos["symbol"])
        avg_cost = last_price * pos["cost_mult"]
        market_value = pos["quantity"] * last_price
        cost_basis = pos["quantity"] * avg_cost
        rows.append(
            {
                "symbol": pos["symbol"],
                "quantity": pos["quantity"],
                "avg_cost": avg_cost,
                "last_price": last_price,
                "market_value": market_value,
                "unrealized_pl": market_value - cost_basis,
                "unrealized_pl_pct": (last_price - avg_cost) / avg_cost * 100,
                "is_vol_ticker": pos["is_vol_ticker"],
            }
        )
    return pd.DataFrame(rows)

_OPTION_BOOK = [
    {"symbol": "XOM", "type": "put", "side": "long", "strike_mult": 0.98, "dte": 30, "quantity": 10.0,
    "sigma": 0.27},
    {"symbol": "SPY", "type": "call", "side": "long", "strike_mult": 1.02, "dte": 45, "quantity": 5.0,
    "sigma": 0.16},
    {"symbol": "VXX", "type": "call", "side": "short", "strike_mult": 1.15, "dte": 20, "quantity": 8.0,
    "sigma": 0.85},
]

def get_option_positions() -> pd.DataFrame:
    rows = []
    today = datetime.now(timezone.utc).date()
    for pos in _OPTION_BOOK:
        spot = get_underlying_price(pos["symbol"])
        strike = round(spot * pos["strike_mult"] / 0.5) * 0.5

```

```

    expiration = today + timedelta(days=pos["dte"])
    T = pos["dte"] / 365.0
    sigma = pos["sigma"]

    mark_price = black_scholes_price(spot, strike, T, RISK_FREE_RATE, DIVIDEND_YIELD, sigma, pos["type"])
    greeks = all_greeks(spot, strike, T, RISK_FREE_RATE, DIVIDEND_YIELD, sigma, pos["type"])
    entry_drift = 0.85 if pos["side"] == "long" else 1.15
    avg_price = mark_price * entry_drift

    qty = pos["quantity"]
    multiplier = 100
    signed_qty = qty if pos["side"] == "long" else -qty

    rows.append(
        {
            "symbol": pos["symbol"],
            "type": pos["type"],
            "side": pos["side"],
            "strike": strike,
            "expiration": expiration.isoformat(),
            "dte": pos["dte"],
            "quantity": qty,
            "avg_price": avg_price,
            "mark_price": mark_price,
            "market_value": mark_price * qty * multiplier,
            "unrealized_pl": (mark_price - avg_price) * signed_qty * multiplier,
            "delta": greeks["delta"] * signed_qty * multiplier,
            "theta": greeks["theta"] * signed_qty * multiplier,
            "vega": greeks["vega"] * signed_qty * multiplier,
            "gamma": greeks["gamma"] * signed_qty * multiplier,
            "implied_volatility": sigma,
        }
    )
)
return pd.DataFrame(rows)

def get_open_orders() -> pd.DataFrame:
    return pd.DataFrame(
        [
            {"instrument_type": "equity", "symbol": "AAPL", "side": "buy", "quantity": 25.0, "price":
get_underlying_price("AAPL") * 0.97, "state": "open", "created_at": (datetime.now(timezone.utc) -
timedelta(hours=3)).isoformat()},
            {"instrument_type": "option", "symbol": "SPY", "side": "sell", "quantity": 2.0, "price": 4.35,
"state": "open", "created_at": (datetime.now(timezone.utc) - timedelta(hours=1)).isoformat()},
        ]
    )

def get_order_history(days_back: int = 90) -> pd.DataFrame:
    rng = np.random.default_rng(_seed("order_history", str(days_back)))
    symbols = ["XOM", "AAPL", "TSLA", "MSFT", "SPY"]
    rows = []
    now = datetime.now(timezone.utc)
    order_id = 1000

    for i in range(18):
        symbol = symbols[i % len(symbols)]
        is_option = bool(rng.integers(0, 2))
        days_ago_open = int(rng.uniform(5, days_back))
        entry_price = get_underlying_price(symbol) * float(rng.uniform(0.85, 1.15))

```

```

    qty = float(rng.integers(1, 10)) if is_option else float(rng.integers(10, 200))
    move = float(rng.normal(0.01, 0.06))
    exit_price = entry_price * (1 + move)
    multiplier = 100 if is_option else 1

    open_date = now - timedelta(days=days_ago_open)
    close_date = open_date + timedelta(days=int(rng.uniform(1, min(days_ago_open, 20) + 1)))

    rows.append({
        "date": open_date, "instrument_type": "option" if is_option else "equity", "symbol": symbol,
        "side": "buy", "quantity": qty, "price": entry_price,
        "amount": -qty * entry_price * multiplier, "fees": 0.0, "order_id": str(order_id),
        **({"strategy": "opening"} if is_option else {}),
    })
    rows.append({
        "date": close_date, "instrument_type": "option" if is_option else "equity", "symbol": symbol,
        "side": "sell", "quantity": qty, "price": exit_price,
        "amount": qty * exit_price * multiplier, "fees": 0.0, "order_id": str(order_id + 1),
        **({"strategy": "closing"} if is_option else {}),
    })
    order_id += 2

df = pd.DataFrame(rows)
df["date"] = pd.to_datetime(df["date"], utc=True)
return df.sort_values("date", ascending=False).reset_index(drop=True)

def get_stock_quote_batch(symbols: list[str]) -> pd.DataFrame:
    return pd.DataFrame([get_stock_quote(s) for s in symbols])

def get_vol_ticker_quotes() -> pd.DataFrame:
    rows = []
    for t in get_vol_tickers():
        last = get_underlying_price(t)
        rng = np.random.default_rng(_seed(t, "prevclose"))
        prev_close = last * float(rng.uniform(0.97, 1.03))
        rows.append({"symbol": t, "last_price": last, "prev_close": prev_close, "change_pct": (last -
prev_close) / prev_close * 100})
    return pd.DataFrame(rows)

def get_option_chain_expirations(symbol: str) -> list[str]:
    today = datetime.now(timezone.utc).date()
    days_to_friday = (4 - today.weekday()) % 7 or 7
    first_friday = today + timedelta(days=days_to_friday)
    return [(first_friday + timedelta(weeks=w)).isoformat() for w in range(6)]

def get_option_chain_skew(symbol: str, expiration_date: str) -> pd.DataFrame:
    spot = get_underlying_price(symbol)
    today = datetime.now(timezone.utc).date()
    exp = datetime.fromisoformat(expiration_date).date()
    dte = max((exp - today).days, 1)
    T = dte / 365.0

    rng = np.random.default_rng(_seed(symbol, f"chain:{expiration_date}"))
    atm_iv = float(rng.uniform(0.18, 0.45))
    increment = 1.0 if spot < 50 else (2.5 if spot < 150 else 5.0)
    strikes = [round((spot + k * increment) / increment) * increment for k in range(-7, 8)]

```

```

rows = []
for strike in strikes:
    moneyness = math.log(strike / spot)
    iv = max(atm_iv + 0.35 * moneyness**2 - 0.05 * moneyness, 0.05) # smile + mild skew
    for option_type in ("call", "put"):
        price = black_scholes_price(spot, strike, T, RISK_FREE_RATE, DIVIDEND_YIELD, iv, option_type)
        delta = all_greeks(spot, strike, T, RISK_FREE_RATE, DIVIDEND_YIELD, iv, option_type)["delta"]
        spread_pct = 2.0 + 6.0 * abs(moneyness)
        spread = price * spread_pct / 100
        bid, ask = max(price - spread / 2, 0.01), price + spread / 2
        rows.append({
            "strike": strike, "type": option_type, "iv": iv, "delta": delta,
            "bid": round(bid, 2), "ask": round(ask, 2),
            "volume": int(rng.uniform(5, 800)), "open_interest": int(rng.uniform(50, 5000)),
        })

df = pd.DataFrame(rows)
df["mid"] = (df["bid"] + df["ask"]) / 2
df["spread"] = df["ask"] - df["bid"]
df["spread_pct"] = (df["spread"] / df["mid"].replace(0, pd.NA) * 100).fillna(0.0)
return df.sort_values(["type", "strike"]).reset_index(drop=True)

```

app/vol_analysis.py

Book-level analysis that operates on the DataFrames `data_fetch/demo_data` return: aggregating Greeks across a whole option book, breaking vega/theta down by underlying, computing IV-skew metrics (ATM IV, 25-delta risk reversal) from an option chain, and — the two functions added for the Correlation Explorer tab — `cumulative_return()` (indexes a price series to 0% at its first observation) and `correlation_stats()` (price-level AND daily-return correlation between two symbols, with an explicit note on why price-level correlation is usually inflated by shared trend and return correlation is the more honest signal).

```
"""Book-level volatility exposure analysis on top of raw position DataFrames."""
from __future__ import annotations

import pandas as pd

def aggregate_greeks(option_positions: pd.DataFrame) -> dict:
    if option_positions.empty:
        return {"net_delta": 0.0, "net_theta": 0.0, "net_vega": 0.0, "net_gamma": 0.0}
    return {
        "net_delta": float(option_positions["delta"].sum()),
        "net_theta": float(option_positions["theta"].sum()),
        "net_vega": float(option_positions["vega"].sum()),
        "net_gamma": float(option_positions["gamma"].sum()),
    }

def vol_etp_exposure(equity_positions: pd.DataFrame) -> pd.DataFrame:
    """Directional (non-options) exposure to VIX-linked ETPs specifically."""
    if equity_positions.empty:
        return equity_positions
    return equity_positions[equity_positions["is_vol_ticker"]].copy()

def premium_by_underlying(option_positions: pd.DataFrame) -> pd.DataFrame:
    """Net premium (market value) and vega grouped by underlying — which
    names are actually driving the book's vol exposure."""
    if option_positions.empty:
        return option_positions
    grouped = (
        option_positions.groupby("symbol")
        .agg(
            contracts=("quantity", "sum"),
            market_value=("market_value", "sum"),
            unrealized_pl=("unrealized_pl", "sum"),
            net_delta=("delta", "sum"),
            net_theta=("theta", "sum"),
            net_vega=("vega", "sum"),
            avg_iv=("implied_volatility", "mean"),
        )
        .reset_index()
        .sort_values("net_vega", key=abs, ascending=False)
    )
    return grouped
```

```

def near_expiry(option_positions: pd.DataFrame, within_days: int = 7) -> pd.DataFrame:
    if option_positions.empty:
        return option_positions
    return option_positions[option_positions["dte"].fillna(999) <= within_days].sort_values("dte")

def skew_metrics(chain: pd.DataFrame, spot: float) -> dict:
    """ATM IV and 25-delta risk reversal (call IV minus put IV) – the two
    standard single-number summaries of a skew curve's level and slant."""
    if chain.empty or not spot:
        return {"atm_iv": None, "risk_reversal_25d": None}

    calls = chain[chain["type"] == "call"]
    puts = chain[chain["type"] == "put"]

    atm_ivs = []
    if not calls.empty:
        atm_ivs.append(calls.loc[(calls["strike"] - spot).abs().idxmin(), "iv"])
    if not puts.empty:
        atm_ivs.append(puts.loc[(puts["strike"] - spot).abs().idxmin(), "iv"])
    atm_iv = sum(atm_ivs) / len(atm_ivs) if atm_ivs else None

    risk_reversal = None
    if not calls.empty and not puts.empty:
        call_25d = calls.loc[(calls["delta"] - 0.25).abs().idxmin()]
        put_25d = puts.loc[(puts["delta"] + 0.25).abs().idxmin()]
        risk_reversal = float(call_25d["iv"] - put_25d["iv"])

    return {"atm_iv": atm_iv, "risk_reversal_25d": risk_reversal}

def cumulative_return(close: pd.Series) -> pd.Series:
    """Indexed % return from the first observation – lets two differently-priced
    series be overlaid on one chart on a common scale."""
    if close.empty:
        return close
    return (close / close.iloc[0] - 1.0) * 100.0

def correlation_stats(hist_a: pd.DataFrame, hist_b: pd.DataFrame) -> dict:
    """Price-level and daily-return correlation between two get_equity_historicals()
    frames (each needs 'date' and 'close'), aligned on date.

    Price-level correlation is usually inflated by any shared long-run trend –
    two names that both drifted up over the window will look correlated even if
    their day-to-day moves are unrelated. Return correlation strips the trend
    out and is the more honest co-movement signal."""
    merged = pd.merge(hist_a[["date", "close"]], hist_b[["date", "close"]], on="date", suffixes=("_a", "_b"))
    if len(merged) < 3:
        return {"price_corr": None, "return_corr": None, "n_obs": len(merged)}

    price_corr = merged["close_a"].corr(merged["close_b"])
    ret_a = merged["close_a"].pct_change().dropna()
    ret_b = merged["close_b"].pct_change().dropna()
    return_corr = ret_a.corr(ret_b)

    return {
        "price_corr": float(price_corr) if pd.notna(price_corr) else None,

```

```

        "return_corr": float(return_corr) if pd.notna(return_corr) else None,
        "n_obs": len(merged),
    }

def held_contracts_in_chain(
    option_positions: pd.DataFrame, chain: pd.DataFrame, symbol: str, expiration: str
) -> pd.DataFrame:
    """Your open option positions that fall within the given chain lookup,
    so they can be marked on the skew/spread charts for that underlying+expiration."""
    if option_positions.empty or chain.empty:
        return option_positions.iloc[0:0]

    held = option_positions[(option_positions["symbol"] == symbol) & (option_positions["expiration"] ==
expiration)]
    if held.empty:
        return held

    return held.merge(chain[["strike", "type", "iv", "spread", "spread_pct", "mid"]], on=["strike", "type"],
how="left")

```

app/journal.py

Normalizes the raw order-history DataFrame into a clean trade journal and handles CSV export — a thin, deliberately simple module.

```
"""Trade journal: normalize fill history into an exportable CSV log."""
from __future__ import annotations

from pathlib import Path

import pandas as pd

EXPORT_DIR = Path(__file__).resolve().parent.parent / "exports"


def build_journal(order_history: pd.DataFrame) -> pd.DataFrame:
    if order_history.empty:
        return order_history
    journal = order_history.copy()
    journal["date"] = pd.to_datetime(journal["date"]).dt.tz_convert(None)
    journal = journal.sort_values("date", ascending=False)
    return journal[
        [c for c in ["date", "instrument_type", "symbol", "side", "quantity", "price", "amount", "fees",
"strategy", "order_id"] if c in journal.columns]
    ]


def export_csv(journal: pd.DataFrame, filename: str | None = None) -> Path:
    EXPORT_DIR.mkdir(parents=True, exist_ok=True)
    if filename is None:
        filename = f"trade_journal_{pd.Timestamp.now():%Y%m%d_%H%M%S}.csv"
    path = EXPORT_DIR / filename
    journal.to_csv(path, index=False)
    return path
```

app/performance.py

FIFO round-trip trade matching: walks each symbol's chronological fills with a deque-based lot-matching algorithm (a buy first closes any open short lots oldest-first, then opens a new long lot with whatever's left — mirrored for sells), which is the same mechanic a broker's own realized-P&L accounting uses.

Documents its own real limitation openly: Robinhood's fill history only carries the underlying symbol for options, not per-contract strike/expiration identity, so options with multiple concurrent contracts on the same underlying can get cross-matched. Equity matching has no such ambiguity and is exact.

```
"""Realized round-trip trade matching + win-rate stats, built from
data_fetch.get_order_history's fill log.

FIFO-matches each symbol's chronological buy/sell fills into closed
round-trip trades (long: buy-then-sell; short: sell-then-buy) -- the same
mechanics a broker's own realized-P&L accounting uses, so partial fills
and re-entries net out correctly. Options carry the standard 100-share
contract multiplier; equities don't.

Known limitation, not fixable from data currently pulled: Robinhood's
fill history exposes only the underlying symbol for options, not a
per-contract strike/expiration/type identity (only *open positions* carry
that). So options are matched per-underlying-symbol -- if you hold
multiple different contracts on the same underlying concurrently, fills
across them can get cross-matched. Equity matching has no such ambiguity
and is exact.
"""
from __future__ import annotations

from collections import deque
from dataclasses import dataclass

import pandas as pd

_CONTRACT_MULTIPLIER = {"equity": 1, "option": 100}

@dataclass
class _Lot:
    qty: float
    price: float
    date: pd.Timestamp

@dataclass
class RoundTrip:
    symbol: str
    instrument_type: str
    direction: str # "long" (buy opened it) or "short" (sell opened it)
    quantity: float
    entry_price: float
    exit_price: float
    entry_date: pd.Timestamp
    exit_date: pd.Timestamp
```

```

realized_pl: float
is_win: bool

def match_round_trips(order_history: pd.DataFrame) -> list[RoundTrip]:
    """Chronological FIFO match per (symbol, instrument_type) group. A buy
    first closes any open short lots (oldest first), then opens/adds to a
    long lot with whatever quantity is left over; a sell is the mirror."""
    if order_history.empty:
        return []

    trips: list[RoundTrip] = []
    for (symbol, instrument_type), group in order_history.groupby(["symbol", "instrument_type"]):
        multiplier = _CONTRACT_MULTIPLIER.get(instrument_type, 1)
        long_lots: deque[_Lot] = deque()
        short_lots: deque[_Lot] = deque()

        for _, fill in group.sort_values("date").iterrows():
            side = str(fill.get("side") or "").lower()
            qty = float(fill["quantity"])
            price = float(fill["price"])
            date = fill["date"]
            if qty <= 0 or price <= 0 or side not in ("buy", "sell"):
                continue

            opposing = short_lots if side == "buy" else long_lots
            remaining = qty
            while remaining > 1e-9 and opposing:
                lot = opposing[0]
                matched = min(remaining, lot.qty)
                direction = "short" if side == "buy" else "long"
                pl = ((lot.price - price) if side == "buy" else (price - lot.price)) * matched * multiplier
                trips.append(RoundTrip(symbol, instrument_type, direction, matched, lot.price, price,
lot.date, date, pl, pl > 0))
                lot.qty -= matched
                remaining -= matched
                if lot.qty <= 1e-9:
                    opposing.popleft()

            if remaining > 1e-9:
                (long_lots if side == "buy" else short_lots).append(_Lot(remaining, price, date))

        trips.sort(key=lambda t: t.exit_date)
        return trips

_TRIP_COLUMNS = [
    "symbol", "instrument_type", "direction", "quantity", "entry_price", "exit_price",
    "entry_date", "exit_date", "realized_pl", "is_win", "holding_days",
]

def round_trips_to_frame(trips: list[RoundTrip]) -> pd.DataFrame:
    if not trips:
        return pd.DataFrame(columns=_TRIP_COLUMNS)
    df = pd.DataFrame([t.__dict__ for t in trips])
    df["holding_days"] = (df["exit_date"] - df["entry_date"]).dt.total_seconds() / 86400
    return df[_TRIP_COLUMNS]

```

```

def win_rate_stats(trips_df: pd.DataFrame) -> dict:
    """None fields mean undefined (no trades / no wins / no losses), not zero."""
    if trips_df.empty:
        return {
            "total_trades": 0, "wins": 0, "losses": 0, "win_rate": None,
            "avg_win": None, "avg_loss": None, "expectancy": None,
            "total_realized_pl": 0.0, "profit_factor": None,
        }
    wins = trips_df[trips_df["is_win"]]
    losses = trips_df[~trips_df["is_win"]]
    total = len(trips_df)
    win_rate = len(wins) / total
    avg_win = float(wins["realized_pl"].mean()) if not wins.empty else None
    avg_loss = float(losses["realized_pl"].mean()) if not losses.empty else None
    gross_loss = -float(losses["realized_pl"].sum()) if not losses.empty else 0.0
    gross_win = float(wins["realized_pl"].sum()) if not wins.empty else 0.0
    return {
        "total_trades": total,
        "wins": len(wins),
        "losses": len(losses),
        "win_rate": win_rate,
        "avg_win": avg_win,
        "avg_loss": avg_loss,
        "expectancy": win_rate * (avg_win or 0.0) + (1 - win_rate) * (avg_loss or 0.0),
        "total_realized_pl": float(trips_df["realized_pl"].sum()),
        "profit_factor": (gross_win / gross_loss) if gross_loss > 0 else None,
    }

def win_rate_by_instrument_type(trips_df: pd.DataFrame) -> pd.DataFrame:
    if trips_df.empty:
        return pd.DataFrame(columns=["instrument_type", "total_trades", "wins", "win_rate",
"total_realized_pl"])
    rows = []
    for instrument_type, group in trips_df.groupby("instrument_type"):
        stats = win_rate_stats(group)
        rows.append({
            "instrument_type": instrument_type,
            "total_trades": stats["total_trades"],
            "wins": stats["wins"],
            "win_rate": stats["win_rate"],
            "total_realized_pl": stats["total_realized_pl"],
        })
    return pd.DataFrame(rows)

```

app/colors.py

Shared chart color tokens (categorical palette, diverging pair, status colors) used consistently across every Plotly chart in dashboard.py, so the whole app reads as one visual system instead of each chart picking its own colors.

```
"""Shared color tokens for charts (see dataviz skill reference palette)."""

CATEGORICAL = [
    "#2a78d6", # blue
    "#1baf7a", # aqua
    "#eda100", # yellow
    "#008300", # green
    "#4a3aa7", # violet
    "#e34948", # red
    "#e87ba4", # magenta
    "#eb6834", # orange
]

DIVERGING_POS = "#2a78d6" # blue
DIVERGING_NEG = "#e34948" # red
DIVERGING_MID = "#f0efec"

STATUS_GOOD = "#0ca30c"
STATUS_WARNING = "#fab219"
STATUS_SERIOUS = "#ec835a"
STATUS_CRITICAL = "#d03b3b"

INK_PRIMARY = "#0b0b0b"
INK_SECONDARY = "#52514e"
INK_MUTED = "#898781"
GRIDLINE = "#e1e0d9"
SURFACE = "#fcfcfb"
```

app/__init__.py

Empty — just marks app as a package.

dashboard.py — the Streamlit UI

dashboard.py

The entire UI layer, deliberately thin: every `render_*` function is presentation only, calling straight into `data_fetch` (or `demo_data`, chosen once at import time based on `is_demo_mode()`) for data and into `risk_tool` for every number that involves actual math. Ten tabs, each backed by one render function: Overview, Positions & Greeks, Vol Exposure, Vol Skew (live IV-by-strike and bid/ask-spread skew, with held contracts marked directly on the chart), Risk Tool (the strike-selection/sizing/exit-levels pipeline, wired to a live position monitor that runs the five exit rules against real open positions every refresh), Correlation Explorer, Hedge Calculator (with a movable P&L scenario chart — a Streamlit slider driving a live-recomputed Plotly chart across a full bearish-to-bullish range), Orders & History, Win Rate, and Journal/Export. The `load_data()` function is the single place all Robinhood/demo calls happen per refresh, cached for 60 seconds so navigating between tabs doesn't re-hit the API on every click.

```
"""Personal vol-trading dashboard – Robinhood options + VIX-ETP book.

Run with: streamlit run dashboard.py
"""
import pandas as pd
import plotly.graph_objects as go
import streamlit as st

from app import colors, journal, performance, vol_analysis
from app.auth import ensure_logged_in, is_demo_mode, logout
from risk_tool import hedge
from risk_tool import realized_vol as rv
from risk_tool import risk_manager
from risk_tool import sizing as risk_sizing
from risk_tool import strike_selection
from risk_tool.config import DEFAULT_CONFIG, RiskConfig

if is_demo_mode():
    from app import demo_data as data_fetch # same function names/shapes as data_fetch – synthetic, no live account
else:
    from app import data_fetch

st.set_page_config(page_title="Vol Trading Dashboard", page_icon="📈", layout="wide")

@st.cache_data(ttl=60, show_spinner=False)
def load_data():
    return {
        "overview": data_fetch.get_portfolio_overview(),
        "history": data_fetch.get_portfolio_history(),
        "equity_positions": data_fetch.get_equity_positions(),
        "option_positions": data_fetch.get_option_positions(),
        "open_orders": data_fetch.get_open_orders(),
        "order_history": data_fetch.get_order_history(),
        "vol_quotes": data_fetch.get_vol_ticker_quotes(),
    }
```

```

def money(x: float) -> str:
    sign = "-" if x < 0 else ""
    return f"{sign}${abs(x):,.2f}"

def render_overview(d: dict):
    ov = d["overview"]
    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Portfolio equity", money(ov["equity"]))
    c2.metric("Cash", money(ov["cash"]))
    c3.metric("Buying power", money(ov["buying_power"]))
    c4.metric("Day P&L", money(ov["day_pl"]), f"{ov['day_pl_pct']:.2f}%")

    hist = d["history"]
    if not hist.empty:
        fig = go.Figure()
        fig.add_trace(
            go.Scatter(
                x=hist["date"],
                y=hist["equity"],
                mode="lines",
                line=dict(color=colors.CATEGORICAL[0], width=2),
                hovertemplate="%{x|%b %d}<br>${y:,.0f}<extra></extra>",
                name="Equity",
            )
        )
        fig.update_layout(
            height=280,
            margin=dict(l=10, r=10, t=10, b=10),
            plot_bgcolor=colors.SURFACE,
            paper_bgcolor=colors.SURFACE,
            xaxis=dict(showgrid=False, color=colors.INK_MUTED),
            yaxis=dict(showgrid=True, gridcolor=colors.GRIDLINE, color=colors.INK_MUTED, tickprefix="$"),
            showlegend=False,
        )
        st.plotly_chart(fig, use_container_width=True)
    else:
        st.caption("No portfolio history returned yet.")

def render_positions(d: dict):
    st.subheader("Equity positions")
    eq = d["equity_positions"]
    if eq.empty:
        st.caption("No open equity positions.")
    else:
        st.dataframe(
            eq,
            use_container_width=True,
            hide_index=True,
            column_config={
                "quantity": st.column_config.NumberColumn(format="%.0f"),
                "avg_cost": st.column_config.NumberColumn(format="%.2f"),
                "last_price": st.column_config.NumberColumn(format="%.2f"),
                "market_value": st.column_config.NumberColumn(format="%.2f"),
                "unrealized_pl": st.column_config.NumberColumn(format="%.2f"),
                "unrealized_pl_pct": st.column_config.NumberColumn(format="%.2f%%"),
            },
        )

```

```

st.subheader("Option positions")
opt = d["option_positions"]
if opt.empty:
    st.caption("No open option positions.")
else:
    display_cols = [
        "symbol", "type", "side", "strike", "expiration", "dte", "quantity",
        "avg_price", "mark_price", "market_value", "unrealized_pl",
        "implied_volatility", "delta", "theta", "vega", "gamma",
    ]
    st.dataframe(
        opt[display_cols],
        use_container_width=True,
        hide_index=True,
        column_config={
            "strike": st.column_config.NumberColumn(format="%.2f"),
            "avg_price": st.column_config.NumberColumn(format="%.2f"),
            "mark_price": st.column_config.NumberColumn(format="%.2f"),
            "market_value": st.column_config.NumberColumn(format="%.2f"),
            "unrealized_pl": st.column_config.NumberColumn(format="%.2f"),
            "implied_volatility": st.column_config.NumberColumn(format="percent"),
            "delta": st.column_config.NumberColumn(format="%.2f"),
            "theta": st.column_config.NumberColumn(format="%.2f"),
            "vega": st.column_config.NumberColumn(format="%.2f"),
            "gamma": st.column_config.NumberColumn(format="%.3f"),
        },
    )

    near = vol_analysis.near_expiry(opt)
    if not near.empty:
        st.warning(f"{len(near)} option position(s) expiring within 7 days – check assignment/roll risk.")

def render_vol_exposure(d: dict):
    opt = d["option_positions"]
    eq = d["equity_positions"]
    greeks = vol_analysis.aggregate_greeks(opt)

    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Net delta (shares-equiv.)", f"{greeks['net_delta']:, .0f}")
    c2.metric("Net theta ($/day)", money(greeks["net_theta"]))
    c3.metric("Net vega ($/vol pt)", money(greeks["net_vega"]))
    c4.metric("Net gamma", f"{greeks['net_gamma']:, .2f}")

    st.subheader("Vega & theta by underlying")
    by_underlying = vol_analysis.premium_by_underlying(opt)
    if by_underlying.empty:
        st.caption("No option positions to break down.")
    else:
        fig = go.Figure()
        bar_colors = [colors.DIVERGING_POS if v >= 0 else colors.DIVERGING_NEG for v in
            by_underlying["net_vega"]]
        fig.add_trace(
            go.Bar(
                x=by_underlying["symbol"],
                y=by_underlying["net_vega"],
                marker_color=bar_colors,
                hovertemplate="%{x}<br>Net vega: %{y:.2f}<extra></extra>",
            )

```

```

    )
    fig.update_layout(
        height=280,
        margin=dict(l=10, r=10, t=10, b=10),
        plot_bgcolor=colors.SURFACE,
        paper_bgcolor=colors.SURFACE,
        xaxis=dict(showgrid=False, color=colors.INK_MUTED),
        yaxis=dict(showgrid=True, gridcolor=colors.GRIDLINE, zerolinecolor=colors.INK_MUTED,
color=colors.INK_MUTED),
        showlegend=False,
    )
    st.plotly_chart(fig, use_container_width=True)
    st.dataframe(by_underlying, use_container_width=True, hide_index=True)

    st.subheader("VIX-linked ETP holdings")
    vol_holdings = vol_analysis.vol_etp_exposure(eq)
    quotes = d["vol_quotes"]
    if not quotes.empty:
        cols = st.columns(len(quotes))
        for col, (_, row) in zip(cols, quotes.iterrows()):
            col.metric(row["symbol"], f"${row['last_price']:.2f}", f"{row['change_pct']:.2f}%")
    if vol_holdings.empty:
        st.caption("No direct positions in your configured vol-ETP list (see VOL_TICKERS in .env).")
    else:
        st.dataframe(vol_holdings, use_container_width=True, hide_index=True)

@st.cache_data(ttl=300, show_spinner=False)
def fetch_expirations(symbol: str):
    return data_fetch.get_option_chain_expirations(symbol)

@st.cache_data(ttl=120, show_spinner="Pulling option chain...")
def fetch_chain(symbol: str, expiration: str):
    return data_fetch.get_option_chain_skew(symbol, expiration)

@st.fragment(run_every=5)
def render_live_quote(symbol: str):
    quote = data_fetch.get_stock_quote(symbol)
    c1, c2, c3 = st.columns(3)
    c1.metric("Best bid", f"${quote['bid']:.2f}")
    c2.metric("Best ask", f"${quote['ask']:.2f}")
    c3.metric("Mark (mid)", f"${quote['mark']:.2f}")
    st.caption(f"Last trade ${quote['last_trade_price']:.2f} · refreshes every 5s")

def render_vol_skew(d: dict):
    st.subheader("Volatility & liquidity skew explorer")
    st.caption(
        "Look up the live option chain for any underlying – your held contracts for the "
        "selected expiration are marked on both charts."
    )

    held_positions = d["option_positions"]
    if "skew_symbol_input" not in st.session_state:
        default_symbol = held_positions.iloc[0]["symbol"] if not held_positions.empty else ""
        st.session_state["skew_symbol_input"] = default_symbol

    symbol = st.text_input("Underlying ticker", key="skew_symbol_input", placeholder="e.g.

```

```

SPY").strip().upper()
if not symbol:
    st.caption("Enter a ticker to load its expirations.")
    return

try:
    expirations = fetch_expirations(symbol)
except Exception as exc:
    st.error(f"Couldn't load expirations for {symbol}: {exc}")
    return
if not expirations:
    st.warning(f"No option chain found for {symbol}. Check the ticker and that it has listed options.")
    return

col_a, col_b = st.columns([4, 1])
with col_a:
    expiration = st.selectbox("Expiration", expirations, key="skew_expiration")
with col_b:
    st.write("")
    if st.button("Refresh chain"):
        fetch_chain.clear()

try:
    chain = fetch_chain(symbol, expiration)
except Exception as exc:
    st.error(f"Couldn't load the option chain: {exc}")
    return
if chain.empty:
    st.warning("No quoted contracts returned for this expiration.")
    return

st.markdown("##### Live underlying quote")
render_live_quote(symbol)

spot = data_fetch.get_stock_quote(symbol)["mark"]
metrics = vol_analysis.skew_metrics(chain, spot)

c1, c2 = st.columns(2)
c1.metric("ATM IV", f"{metrics['atm_iv']:.1%}" if metrics["atm_iv"] is not None else "--")
c2.metric(
    "25Δ risk reversal (C - P)",
    f"{metrics['risk_reversal_25d']:.1%}" if metrics["risk_reversal_25d"] is not None else "--",
)

calls = chain[chain["type"] == "call"]
puts = chain[chain["type"] == "put"]
held = vol_analysis.held_contracts_in_chain(held_positions, chain, symbol, expiration)

def held_marker_trace(y_col: str, hover_suffix: str, tickformat: str):
    if held.empty:
        return None
    return go.Scatter(
        x=held["strike"],
        y=held[y_col],
        mode="markers",
        name="Your position",
        marker=dict(symbol="diamond-open", size=15, color=colors.STATUS_WARNING, line=dict(width=2)),
        customdata=held[["side", "quantity", "type"]],
        hovertemplate=(
            "Your position - %{customdata[1]}x %{customdata[0]} %{customdata[2]}"

```

```

        f"<br>Strike ${x:.2f}<br>{hover_suffix} ${y:{tickformat}}<extra></extra>"
    ),
)

st.markdown("#### Volatility skew (IV by strike)")
fig_iv = go.Figure()
fig_iv.add_trace(
    go.Scatter(
        x=calls["strike"], y=calls["iv"], mode="lines+markers", name="Calls",
        line=dict(color=colors.CATEGORICAL[0], width=2),
        marker=dict(size=6),
        hovertemplate="Strike ${x:.2f}<br>IV ${y:.1%}<extra>Call</extra>",
    )
)
fig_iv.add_trace(
    go.Scatter(
        x=puts["strike"], y=puts["iv"], mode="lines+markers", name="Puts",
        line=dict(color=colors.CATEGORICAL[5], width=2),
        marker=dict(size=6),
        hovertemplate="Strike ${x:.2f}<br>IV ${y:.1%}<extra>Put</extra>",
    )
)
iv_marker = held_marker_trace("iv", "IV", ".1%")
if iv_marker:
    fig_iv.add_trace(iv_marker)
if spot:
    fig_iv.add_vline(x=spot, line=dict(color=colors.INK_MUTED, dash="dash", width=1),
annotation_text="Mark", annotation_position="top")
fig_iv.update_layout(
    height=360,
    margin=dict(l=10, r=10, t=10, b=10),
    plot_bgcolor=colors.SURFACE,
    paper_bgcolor=colors.SURFACE,
    xaxis=dict(title="Strike", showgrid=False, color=colors.INK_MUTED),
    yaxis=dict(title="Implied volatility", showgrid=True, gridcolor=colors.GRIDLINE,
color=colors.INK_MUTED, tickformat=".0%"),
    legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1),
)
st.plotly_chart(fig_iv, use_container_width=True)

st.markdown("#### Bid/ask spread skew (liquidity by strike)")
st.caption("Spread shown as % of mid price – the standard way to compare liquidity across strikes with
very different premiums.")
fig_spread = go.Figure()
fig_spread.add_trace(
    go.Scatter(
        x=calls["strike"], y=calls["spread_pct"], mode="lines+markers", name="Calls",
        line=dict(color=colors.CATEGORICAL[0], width=2),
        marker=dict(size=6),
        hovertemplate="Strike ${x:.2f}<br>Spread ${y:.1f}%<extra>Call</extra>",
    )
)
fig_spread.add_trace(
    go.Scatter(
        x=puts["strike"], y=puts["spread_pct"], mode="lines+markers", name="Puts",
        line=dict(color=colors.CATEGORICAL[5], width=2),
        marker=dict(size=6),
        hovertemplate="Strike ${x:.2f}<br>Spread ${y:.1f}%<extra>Put</extra>",
    )
)

```

```

spread_marker = held_marker_trace("spread_pct", "Spread", ".1f")
if spread_marker:
    fig_spread.add_trace(spread_marker)
if spot:
    fig_spread.add_vline(x=spot, line=dict(color=colors.INK_MUTED, dash="dash", width=1),
    annotation_text="Mark", annotation_position="top")
fig_spread.update_layout(
    height=360,
    margin=dict(l=10, r=10, t=10, b=10),
    plot_bgcolor=colors.SURFACE,
    paper_bgcolor=colors.SURFACE,
    xaxis=dict(title="Strike", showgrid=False, color=colors.INK_MUTED),
    yaxis=dict(title="Spread (% of mid)", showgrid=True, gridcolor=colors.GRIDLINE,
    color=colors.INK_MUTED, ticksuffix="%"),
    legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1),
)
st.plotly_chart(fig_spread, use_container_width=True)

st.dataframe(
    chain,
    use_container_width=True,
    hide_index=True,
    column_config={
        "strike": st.column_config.NumberColumn(format="%.2f"),
        "iv": st.column_config.NumberColumn(format="percent"),
        "delta": st.column_config.NumberColumn(format="%.3f"),
        "bid": st.column_config.NumberColumn(format="%.2f"),
        "ask": st.column_config.NumberColumn(format="%.2f"),
        "mid": st.column_config.NumberColumn(format="%.2f"),
        "spread": st.column_config.NumberColumn(format="%.2f"),
        "spread_pct": st.column_config.NumberColumn(format="%.1f%"),
        "volume": st.column_config.NumberColumn(format="%.0f"),
        "open_interest": st.column_config.NumberColumn(format="%.0f"),
    },
)

@st.cache_data(ttl=3600, show_spinner="Pulling price history...")
def fetch_price_history(symbol: str):
    return data_fetch.get_equity_historical(symbol)

def _strike_ev_table(strike_evs) -> pd.DataFrame:
    return pd.DataFrame(
        [
            {
                "strike": s.strike,
                "premium": s.premium,
                "p_win (risk-neutral)": s.p_win_risk_neutral,
                "EV": s.ev,
                "risk:reward": s.risk_reward,
                "setup": "Poor (< min R:R)" if s.is_poor_setup else "OK",
                "delta": s.delta,
                "theta/day": s.theta,
                "vega": s.vega,
            }
            for s in strike_evs
        ]
    )

```

```

STRIKE_EV_COLUMN_CONFIG = {
    "strike": st.column_config.NumberColumn(format="$.2f"),
    "premium": st.column_config.NumberColumn(format="$.2f"),
    "p_win (risk-neutral)": st.column_config.NumberColumn(format="percent"),
    "EV": st.column_config.NumberColumn(format="$.2f"),
    "risk-neutral EV": st.column_config.NumberColumn(format="$.2f"),
    "risk:reward": st.column_config.NumberColumn(format="%.2f:1"),
    "delta": st.column_config.NumberColumn(format="%.3f"),
    "theta/day": st.column_config.NumberColumn(format="%.3f"),
    "vega": st.column_config.NumberColumn(format="%.3f"),
}

def render_risk_tool(d: dict):
    st.subheader("Options risk management & strike selection")
    st.caption(
        "This tool does not predict direction. You supply the thesis (ticker, direction, timeframe) - "
        "it prices candidate strikes on expected value, sizes the position, and gives you pre-committed "
        "exit levels. All win probabilities are risk-neutral (derived from option prices), not forecasts."
    )

    account_equity = float(d["overview"].get("equity") or 0.0)

    with st.form("risk_tool_inputs"):
        c1, c2, c3 = st.columns(3)
        ticker = c1.text_input("Ticker", value=st.session_state.get("skew_symbol_input", "").strip().upper())
        direction = c2.selectbox("Direction", ["call", "put"])
        dte = c3.number_input("Days to expiry", min_value=1, value=30, step=1)

        c4, c5, c6 = st.columns(3)
        spot_input = c4.number_input("Spot price ($, 0 = auto-fetch)", min_value=0.0, value=0.0, step=0.5)
        iv_input = c5.number_input("Market IV (% , 0 = use chain ATM IV)", min_value=0.0, value=0.0, step=1.0)
        strike_increment = c6.number_input("Strike increment ($)", min_value=0.5, value=5.0, step=0.5)

        c7, c8 = st.columns(2)
        account_override = c7.number_input("Account size ($)", min_value=0.0, value=account_equity,
step=1000.0)
        use_realized = c8.checkbox("Compute realized vol from 1yr price history (adds the realized-vs-implied
edge check)")

        submitted = st.form_submit_button("Analyze", type="primary")

    if not submitted:
        st.info("Fill in a thesis above and click Analyze.")
        return
    if not ticker:
        st.error("Enter a ticker.")
        return

    spot = spot_input if spot_input > 0 else data_fetch.get_stock_quote(ticker)["mark"]
    if not spot:
        st.error(f"Couldn't get a live price for {ticker}.")
        return

    market_iv = iv_input / 100 if iv_input > 0 else None
    if market_iv is None:
        try:
            expirations = data_fetch.get_option_chain_expirations(ticker)
            today = pd.Timestamp.now().normalize()

```

```

        target_date = today + pd.Timedelta(days=int(dte))
        nearest_exp = min(expirations, key=lambda e: abs((pd.Timestamp(e) - target_date).days))
        chain = data_fetch.get_option_chain_skew(ticker, nearest_exp)
        market_iv = vol_analysis.skew_metrics(chain, spot)["atm_iv"]
    except Exception as exc:
        st.error(f"Couldn't determine market IV from the option chain ({exc}) – enter it manually
above.")

    return

if not market_iv:
    st.error("Couldn't determine market IV – enter it manually above.")
    return

realized_or_forecast_vol = None
if use_realized:
    try:
        hist = fetch_price_history(ticker)
        realized_or_forecast_vol = rv.close_to_close_vol(hist["close"])
    except Exception as exc:
        st.warning(f"Couldn't compute realized vol ({exc}) – continuing with market IV only.")

config = DEFAULT_CONFIG
T = dte / 365.0
strikes = strike_selection.generate_candidate_strikes(spot, strike_increment,
config.num_strikes_each_side)

if realized_or_forecast_vol:
    st.markdown("##### Realized-vs-implied edge check")
    spread = realized_or_forecast_vol - market_iv
    c1, c2, c3 = st.columns(3)
    c1.metric("Market IV", f"{market_iv:.1%}")
    c2.metric("Realized vol (close-to-close, 1yr)", f"{realized_or_forecast_vol:.1%}")
    c3.metric("Spread (realized - implied)", f"{spread:+.1%}")
    st.caption(
        "Positive spread = your realized vol exceeds market IV (options may be cheap relative to actual "
        "movement); negative = the opposite. This is the only place an actual edge can come from in this
"

        "model – see strike_selection.py's module docstring."
    )

    comparisons = [
        strike_selection.compare_implied_vs_realized_ev(
            spot, K, T, config.risk_free_rate, config.dividend_yield, market_iv,
            realized_or_forecast_vol, direction, config
        )
        for K in strikes
    ]
    comparisons.sort(key=lambda c: c.edge_ev.ev, reverse=True)

    st.markdown("##### Strikes ranked by edge EV (your vol view, market's actual premium)")
    edge_table = _strike_ev_table([c.edge_ev for c in comparisons])
    edge_table.insert(edge_table.columns.get_loc("EV") + 1, "risk-neutral EV", [c.risk_neutral_ev.ev for
c in comparisons])
    st.dataframe(edge_table, use_container_width=True, hide_index=True,
column_config=STRIKE_EV_COLUMN_CONFIG)
    best = comparisons[0].edge_ev
    best_p_win = best.p_win_risk_neutral
else:
    ranked = strike_selection.rank_strikes_by_ev(spot, T, config.risk_free_rate, config.dividend_yield,
market_iv, direction, strikes, config)
    st.markdown("##### Strikes ranked by risk-neutral EV (market IV throughout)")

```

```

        st.dataframe(_strike_ev_table(ranked), use_container_width=True, hide_index=True,
column_config=STRIKE_EV_COLUMN_CONFIG)
        best = ranked[0]
        best_p_win = best.p_win_risk_neutral

    st.markdown("##### Recommended strike")
    c1, c2, c3 = st.columns(3)
    c1.metric("Strike", f"${best.strike:.2f}")
    c2.metric("Premium", f"${best.premium:.2f}")
    c3.metric("EV", f"${best.ev:+.2f}")
    if best.is_poor_setup:
        st.warning(f"Risk/reward {best.risk_reward:.2f}:1 is below your {config.min_risk_reward_ratio:.1f}:1
minimum – poor setup by this rule, even though it's the best of the candidates.")

    sizing_result = risk_sizing.size_position(
        account_size=account_override or 1.0,
        premium_per_contract=best.premium,
        p_win=best_p_win,
        profit_if_win_per_contract=best.profit_if_win,
        config=config,
    )
    st.markdown("##### Position sizing")
    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Kelly fraction used", f"${sizing_result.kelly_fraction_used:.1%}")
    c2.metric("Kelly-suggested size", f"${sizing_result.kelly_dollar_size:,.0f}")
    c3.metric("Hard cap (5% default)", f"${sizing_result.hard_cap_dollar_size:,.0f}")
    c4.metric("Recommended", f"${sizing_result.recommended_dollar_size:,.0f}", f"
{sizing_result.recommended_contracts} contracts")
    if sizing_result.capped_by_hard_limit:
        st.caption("Capped by the hard %-of-account limit, not by Kelly – the cap always wins.")

    st.markdown("##### Pre-committed entry/exit levels")
    entry = best.premium
    c1, c2 = st.columns(2)
    c1.metric("Profit target", f"${entry * (1 + config.profit_target_pct):.2f}", f"+
{config.profit_target_pct:.0%}")
    c2.metric("Stop loss", f"${entry * (1 + config.stop_loss_pct):.2f}", f"{config.stop_loss_pct:.0%}")
    st.caption(
        f"Also exits if |delta| < {config.delta_exit_threshold}, DTE <= {config.theta_exit_dte_threshold}
while still OTM, "
        f"or IV drops >= {config.iv_crush_threshold_points:.0%} points from entry ({market_iv:.1%})."
    )

def render_position_monitor(d: dict):
    st.subheader("Live position monitor – exit rules on your actual open positions")
    positions = d["option_positions"]
    if positions.empty:
        st.caption("No open option positions to monitor.")
        return

    config = DEFAULT_CONFIG
    st.session_state.setdefault("entry_iv_cache", {})

    for _, pos in positions.iterrows():
        cache_key = f"{pos['symbol']}_{pos['strike']}_{pos['expiration']}_{pos['type']}_{pos['side']}"
        if cache_key not in st.session_state["entry_iv_cache"]:
            st.session_state["entry_iv_cache"][cache_key] = pos["implied_volatility"]
        entry_iv = st.session_state["entry_iv_cache"][cache_key]

```

```

try:
    spot = data_fetch.get_underlying_price(pos["symbol"])
except Exception:
    spot = pos["strike"]

dte = int(pos["dte"]) if pd.notna(pos["dte"]) else 999
per_contract_delta = pos["delta"] / (pos["quantity"] * 100) if pos["quantity"] else 0.0

position = risk_manager.Position(
    symbol=pos["symbol"],
    option_type=pos["type"],
    strike=pos["strike"],
    spot=spot,
    dte=dte,
    quantity=int(pos["quantity"]),
    entry_premium=pos["avg_price"],
    current_premium=pos["mark_price"],
    entry_iv=entry_iv,
    current_iv=pos["implied_volatility"],
    delta=per_contract_delta,
)

signals = risk_manager.evaluate_all_rules(position, config)
triggered = [s for s in signals if s.triggered]

label = f"{pos['symbol']} ${pos['strike']:.2f} {pos['type']} exp {pos['expiration']} ({pos['side']}
x{pos['quantity']:.0f})"
if triggered:
    st.error(f"**{label}** - {len(triggered)} exit rule(s) triggered")
    for s in triggered:
        st.write(f"- **{s.rule}**: {s.reason}")
else:
    st.success(f"**{label}** - no exit rules triggered")
with st.expander("Full rule breakdown"):
    for s in signals:
        marker = "●" if s.triggered else "○"
        st.write(f"{marker} **{s.rule}**: {s.reason}")

st.caption(
    "Entry IV is approximated as the IV first observed by this dashboard for each position – not the true
"
    "trade-entry IV, which isn't persisted anywhere. The IV-crush exit is only meaningful once the
dashboard "
    "has been watching a position continuously since you opened it."
)

st.markdown("##### Portfolio governors")
net_delta_dollars = float(positions["delta"].sum())
net_vega_dollars = float(positions["vega"].sum())
daily_pnl = float(d["overview"].get("day_p1") or 0.0)

c1, c2 = st.columns(2)
max_delta_input = c1.number_input("Max net delta ($, 0 = no cap)", min_value=0.0, value=0.0, step=1000.0)
max_vega_input = c2.number_input("Max net vega ($, 0 = no cap)", min_value=0.0, value=0.0, step=100.0)

governor_config = RiskConfig(
    daily_max_loss_pct=config.daily_max_loss_pct,
    max_net_delta_dollars=max_delta_input if max_delta_input > 0 else None,
    max_net_vega_dollars=max_vega_input if max_vega_input > 0 else None,
)

state = risk_manager.PortfolioState(

```

```

        account_size=float(d["overview"].get("equity") or 1.0),
        daily_pnl=daily_pnl,
        net_delta_dollars=net_delta_dollars,
        net_vega_dollars=net_vega_dollars,
    )
governor_result = risk_manager.check_portfolio_governors(state, governor_config)

c1, c2, c3 = st.columns(3)
c1.metric("Day P&L", f"${daily_pnl:,.2f}")
c2.metric("Net delta ($)", f"${net_delta_dollars:,.0f}")
c3.metric("Net vega ($)", f"${net_vega_dollars:,.2f}")
if governor_result.halted:
    st.error("**New entries halted:**")
    for reason in governor_result.reasons:
        st.write(f"- {reason}")
else:
    st.success("No portfolio governors triggered – new entries not halted.")

def render_correlation_explorer(d: dict):
    st.subheader("Correlation explorer")
    st.caption(
        "Cumulative return and correlation for any two symbols – not just what's in your book. "
        "Price-level correlation is usually inflated by shared trend; daily-return correlation is "
        "the cleaner day-to-day co-movement signal."
    )

    with st.form("correlation_explorer_inputs"):
        c1, c2 = st.columns(2)
        symbol_a = c1.text_input("Symbol A", value="XOM").strip().upper()
        symbol_b = c2.text_input("Symbol B", value="USO").strip().upper()
        submitted = st.form_submit_button("Compare", type="primary")

    if not submitted:
        st.info("Enter two symbols and click Compare.")
        return

    if not symbol_a or not symbol_b:
        st.error("Enter both symbols.")
        return

    try:
        hist_a = fetch_price_history(symbol_a)
        hist_b = fetch_price_history(symbol_b)
    except Exception as exc:
        st.error(f"Couldn't load price history ({exc}).")
        return

    if hist_a.empty or hist_b.empty:
        st.warning(f"No price history returned for {symbol_a if hist_a.empty else symbol_b}.")
        return

    stats = vol_analysis.correlation_stats(hist_a, hist_b)
    if stats["price_corr"] is None:
        st.warning("Not enough overlapping trading days to compute correlation.")
        return

    c1, c2, c3 = st.columns(3)
    c1.metric("Price-level correlation", f"{stats['price_corr']:.2f}")
    c2.metric("Daily-return correlation", f"{stats['return_corr']:.2f}")
    c3.metric("Overlapping trading days", f"{stats['n_obs']}")

```

```

merged = pd.merge(hist_a[["date", "close"]], hist_b[["date", "close"]], on="date", suffixes=("_a", "_b"))
cum_a = vol_analysis.cumulative_return(merged["close_a"])
cum_b = vol_analysis.cumulative_return(merged["close_b"])

fig = go.Figure()
fig.add_trace(
    go.Scatter(
        x=merged["date"], y=cum_a, mode="lines", name=symbol_a,
        line=dict(color=colors.CATEGORICAL[0], width=2),
        hovertemplate="%{x|%b %d}<br>" + symbol_a + " %{y:+.1f}%<extra></extra>",
    )
)
fig.add_trace(
    go.Scatter(
        x=merged["date"], y=cum_b, mode="lines", name=symbol_b,
        line=dict(color=colors.CATEGORICAL[7], width=2),
        hovertemplate="%{x|%b %d}<br>" + symbol_b + " %{y:+.1f}%<extra></extra>",
    )
)
fig.add_hline(y=0, line=dict(color=colors.INK_MUTED, dash="dash", width=1))
fig.update_layout(
    height=340,
    margin=dict(l=10, r=10, t=10, b=10),
    plot_bgcolor=colors.SURFACE,
    paper_bgcolor=colors.SURFACE,
    xaxis=dict(showgrid=False, color=colors.INK_MUTED),
    yaxis=dict(showgrid=True, gridcolor=colors.GRIDLINE, zerolinecolor=colors.INK_MUTED,
color=colors.INK_MUTED, ticksuffix="%"),
    legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1),
)
st.plotly_chart(fig, use_container_width=True)
st.caption(f"Cumulative return indexed to 0% at {merged['date'].iloc[0]:%Y-%m-%d} · daily bars, 1yr
lookback.")

def render_hedge_calculator(d: dict):
    st.subheader("Beta-hedge sizing")
    st.caption(
        "Sizes an offsetting position in a correlated hedge instrument (an ETF, or futures via its "
        "contract multiplier), using delta-adjusted exposure – not premium or notional. This offsets "
        "only the portion of the position's move that correlates with the hedge instrument ( $R^2$ , below) "
        "– not theta, vega, or the idiosyncratic move you're actually betting on. It doesn't make a "
        "trade 'must profit'; it isolates the bet from the hedge instrument's direction. Works for any "
        "two symbols – not just what's in your book."
    )

    held_options = d["option_positions"]
    held_equity = d["equity_positions"]
    position_labels = ["Manual entry"]
    position_lookup = {}
    for _, pos in held_options.iterrows():
        label = f"{pos['symbol']} ${pos['strike']:.2f} {pos['type']} exp {pos['expiration']} ({pos['side']}
x{pos['quantity']:.0f})"
        position_labels.append(label)
        position_lookup[label] = ("option", pos)
    for _, pos in held_equity.iterrows():
        label = f"{pos['symbol']} – {pos['quantity']:.0f} shares"
        position_labels.append(label)
        position_lookup[label] = ("equity", pos)

```

```

chosen_label = st.selectbox("Prefill from an open position (optional)", position_labels)
prefill = position_lookup.get(chosen_label)

with st.form("hedge_calc_inputs"):
    c1, c2 = st.columns(2)
    with c1:
        st.markdown("##### Position")
        if prefill and prefill[0] == "option":
            pos = prefill[1]
            pos_type = st.selectbox("Position type", ["Option", "Shares"], index=0)
        elif prefill and prefill[0] == "equity":
            pos = prefill[1]
            pos_type = st.selectbox("Position type", ["Shares", "Option"], index=0)
        else:
            pos = None
            pos_type = st.selectbox("Position type", ["Shares", "Option"], index=0)

        if pos_type == "Shares":
            default_symbol = pos["symbol"] if prefill and prefill[0] == "equity" else ""
            default_qty = abs(float(pos["quantity"])) if prefill and prefill[0] == "equity" else 100.0
            default_dir = "Long" if (prefill and prefill[0] == "equity" and pos["quantity"] > 0) else
"Short"

            underlying_symbol = st.text_input("Underlying symbol", value=default_symbol).strip().upper()
            share_dir = st.selectbox("Direction", ["Short", "Long"], index=0 if default_dir == "Short"
else 1)

            share_qty = st.number_input("Shares", min_value=0.0, value=default_qty, step=1.0)
            share_entry_price = st.number_input("Entry price ($, 0 = use current price)", min_value=0.0,
value=0.0, step=0.5)
            opt_delta = opt_contracts = opt_mult = opt_side = opt_strike = opt_premium = None
        else:
            default_symbol = pos["symbol"] if prefill and prefill[0] == "option" else ""
            # pos["delta"] from data_fetch is already position-level (delta * signed_qty * 100) – back
out per-contract.
            if prefill and prefill[0] == "option" and pos["quantity"]:
                default_delta = float(pos["delta"]) / (float(pos["quantity"]) * 100.0)
                default_delta = abs(default_delta) if pos["side"] == "long" else -abs(default_delta)
            else:
                default_delta = -0.46
            default_contracts = abs(float(pos["quantity"])) if prefill and prefill[0] == "option" else
5.0

            default_side = pos["type"].capitalize() if prefill and prefill[0] == "option" else "Put"
            default_strike = float(pos["strike"]) if prefill and prefill[0] == "option" else 0.0
            default_premium = float(pos["avg_price"]) if prefill and prefill[0] == "option" else 0.0

            underlying_symbol = st.text_input("Underlying symbol", value=default_symbol).strip().upper()
            opt_side = st.selectbox("Call or put", ["Put", "Call"], index=0 if default_side == "Put" else
1)

            opt_delta = st.number_input("Delta (signed, from your broker)", value=float(default_delta),
step=0.01, format="%.4f")
            opt_contracts = st.number_input("Contracts", min_value=0.0, value=default_contracts,
step=1.0)

            opt_mult = st.number_input("Shares per contract", min_value=1.0, value=100.0, step=1.0)
            opt_strike = st.number_input("Strike ($, for the scenario chart)", min_value=0.0,
value=default_strike, step=0.5)
            opt_premium = st.number_input("Entry premium ($/contract, for the scenario chart)",
min_value=0.0, value=default_premium, step=0.01)
            share_dir = share_qty = share_entry_price = None

            underlying_price_input = st.number_input("Underlying price ($, 0 = auto-fetch)", min_value=0.0,
value=0.0, step=0.5)

```

```

with c2:
    st.markdown("##### Hedge instrument")
    hedge_symbol = st.text_input("Hedge symbol (ETF, for beta estimation)",
value="USO").strip().upper()
    hedge_price_input = st.number_input("Hedge price per unit ($, 0 = auto-fetch)", min_value=0.0,
value=0.0, step=0.5)
    hedge_mult = st.number_input(
        "Units per contract (1 = ETF/stock shares; barrels/contract for futures – 1000 CL, 100 MCL)",
        min_value=1.0, value=1.0, step=1.0,
    )
    beta_mode = st.radio("Beta", ["Estimate from price history", "Enter manually"], horizontal=True)
    if beta_mode == "Enter manually":
        beta_manual = st.number_input("Beta", value=0.35, step=0.01, format="%.4f")
        lookback_days = None
    else:
        beta_manual = None
        lookback_days = st.number_input("Lookback (calendar days)", min_value=10, value=90, step=10)

    submitted = st.form_submit_button("Calculate", type="primary")

if submitted:
    if not underlying_symbol or not hedge_symbol:
        st.error("Enter both an underlying symbol and a hedge symbol.")
        return

    if pos_type == "Shares":
        exposure_shares = hedge.share_position_exposure_shares(share_qty, share_dir.lower())
    else:
        exposure_shares = hedge.option_position_exposure_shares(opt_delta, opt_contracts, opt_mult)

    try:
        underlying_price = underlying_price_input if underlying_price_input > 0 else
data_fetch.get_stock_quote(underlying_symbol)["mark"]
        hedge_price = hedge_price_input if hedge_price_input > 0 else
data_fetch.get_stock_quote(hedge_symbol)["mark"]
    except Exception as exc:
        st.error(f"Couldn't fetch a live price ({exc}) – enter prices manually above.")
        return

    if not underlying_price or not hedge_price:
        st.error("Couldn't get live prices for one of the symbols – enter them manually above.")
        return

    beta_est = None
    if beta_manual is not None:
        beta = beta_manual
    else:
        try:
            hist_u = fetch_price_history(underlying_symbol)
            hist_h = fetch_price_history(hedge_symbol)
            cutoff = pd.Timestamp.now(tz=hist_u["date"].dt.tz) - pd.Timedelta(days=lookback_days)
            hist_u = hist_u[hist_u["date"] >= cutoff].set_index("date")["close"]
            hist_h = hist_h[hist_h["date"] >= cutoff].set_index("date")["close"]
            beta_est = hedge.estimate_beta(hedge.simple_returns(hist_u), hedge.simple_returns(hist_h))
            beta = beta_est.beta
        except Exception as exc:
            st.error(f"Couldn't estimate beta from price history ({exc}) – switch to manual entry
above.")

        return

```

```

result = hedge.size_hedge(
    exposure_shares=exposure_shares,
    underlying_price=underlying_price,
    beta=beta,
    hedge_price=hedge_price,
    hedge_contract_multiplier=hedge_mult,
)

st.session_state["hedge_calc"] = {
    "pos_type": pos_type,
    "underlying_symbol": underlying_symbol,
    "hedge_symbol": hedge_symbol,
    "underlying_price": underlying_price,
    "hedge_price": hedge_price,
    "hedge_mult": hedge_mult,
    "beta": beta,
    "beta_est": beta_est,
    "lookback_days": lookback_days,
    "result": result,
    "exposure_shares": exposure_shares,
    "opt_side": opt_side,
    "opt_strike": opt_strike,
    "opt_premium": opt_premium,
    "opt_contracts": opt_contracts,
    "opt_mult": opt_mult,
    "share_entry_price": share_entry_price if share_entry_price else underlying_price if pos_type ==
"Shares" else None,
}

calc = st.session_state.get("hedge_calc")
if not calc:
    st.info("Fill in the position and hedge instrument above, then click Calculate.")
    return

result = calc["result"]
hedge_symbol = calc["hedge_symbol"]
hedge_mult = calc["hedge_mult"]

if calc["beta_est"] is not None:
    st.markdown("#### Beta estimate")
    c1, c2, c3 = st.columns(3)
    c1.metric("Beta", f"{calc['beta_est'].beta:.4f}")
    c2.metric("R² (variance explained)", f"{calc['beta_est'].r_squared:.1%}")
    c3.metric("Observations", f"{calc['beta_est'].n_obs}")
    st.caption(f"{calc['lookback_days']}-day daily-return regression of {calc['underlying_symbol']} on
{hedge_symbol}. Re-run periodically – this drifts.")

st.markdown("#### Hedge")
if result.direction == "long":
    st.success(f"**BUY / LONG** {hedge_symbol}")
else:
    st.error(f"**SELL / SHORT** {hedge_symbol}")

c1, c2, c3 = st.columns(3)
unit_label = "contracts" if hedge_mult > 1 else "shares"
c1.metric(f"{hedge_symbol} {unit_label} needed", f"{abs(result.hedge_units):.2f}")
c2.metric("Hedge notional", money(abs(result.hedge_dollars)))
c3.metric("Position exposure", f"{result.exposure_shares:+.1f} sh eq.
({money(result.exposure_dollars)})")

```

```

st.caption(
    "hedge_dollars = -(exposure_shares × underlying_price) × beta, then divided by "
    "(hedge_price × units_per_contract). Delta and beta both drift – treat this as a "
    "starting size to re-check, not a fire-and-forget position."
)

render_hedge_scenario(calc)

def render_hedge_scenario(calc: dict):
    """Movable P&L scenario chart for whatever position/hedge was just calculated
    above – drag the slider to any underlying move, bearish or bullish."""
    st.markdown("#### Scenario P&L (movable)")

    if calc["pos_type"] == "Option" and not calc["opt_strike"]:
        st.caption("Enter a strike above (in the Position column) to see the scenario chart for an option
        position.")
        return

    move_pct = st.slider(
        f"{calc['underlying_symbol']} price move (%)", min_value=-30.0, max_value=30.0, value=-5.0,
        step=0.25, key="hedge_scenario_move"
    )

    xs = [round(x * 0.5, 2) for x in range(-60, 61)] # -30% .. +30% in 0.5% steps
    underlying_price = calc["underlying_price"]
    hedge_price = calc["hedge_price"]
    beta = calc["beta"]
    result = calc["result"]

    def position_pl(pct: float) -> float:
        s_t = underlying_price * (1 + pct / 100)
        if calc["pos_type"] == "Shares":
            return hedge.share_position_pl(s_t, calc["share_entry_price"], calc["exposure_shares"])
        return hedge.option_intrinsic_pl(
            s_t, calc["opt_strike"], calc["opt_premium"], calc["opt_side"].lower(), calc["opt_contracts"],
            calc["opt_mult"]
        )

    def hedge_pl(pct: float) -> float:
        hedge_move_pct = pct / beta if beta else 0.0
        hedge_t = hedge_price * (1 + hedge_move_pct / 100)
        return hedge.hedge_instrument_pl(hedge_t, hedge_price, result.hedge_units)

    pos_series = [position_pl(x) for x in xs]
    hedge_series = [hedge_pl(x) for x in xs]
    combined_series = [p + h for p, h in zip(pos_series, hedge_series)]

    fig = go.Figure()
    fig.add_trace(go.Scatter(x=xs, y=pos_series, mode="lines", name="Position",
    line=dict(color=colors.CATEGORICAL[0], width=2)))
    fig.add_trace(go.Scatter(x=xs, y=hedge_series, mode="lines", name="Hedge",
    line=dict(color=colors.CATEGORICAL[7], width=2)))
    fig.add_trace(go.Scatter(x=xs, y=combined_series, mode="lines", name="Combined",
    line=dict(color=colors.CATEGORICAL[1], width=3)))
    fig.add_hline(y=0, line=dict(color=colors.INK_MUTED, dash="dash", width=1))
    fig.add_vline(x=move_pct, line=dict(color=colors.INK_PRIMARY, width=1, dash="dot"))
    fig.update_layout(
        height=340,
        margin=dict(l=10, r=10, t=10, b=10),

```

```

        plot_bgcolor=colors.SURFACE,
        paper_bgcolor=colors.SURFACE,
        xaxis=dict(title=f"{calc['underlying_symbol']} price move (%)", showgrid=False,
color=colors.INK_MUTED, ticksuffix="%"),
        yaxis=dict(title="P&L ($)", showgrid=True, gridcolor=colors.GRIDLINE, zerolinecolor=colors.INK_MUTED,
color=colors.INK_MUTED, tickprefix="$"),
        legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1),
    )
    st.plotly_chart(fig, use_container_width=True)

    pos_at_scenario = position_pl(move_pct)
    hedge_at_scenario = hedge_pl(move_pct)
    c1, c2, c3 = st.columns(3)
    c1.metric("Position P&L", money(pos_at_scenario))
    c2.metric("Hedge P&L", money(hedge_at_scenario))
    c3.metric("Combined P&L", money(pos_at_scenario + hedge_at_scenario))

    label = "expiration (intrinsic value only)" if calc["pos_type"] == "Option" else "the scenario price"
    st.caption(
        f"P&L at {label}. The hedge leg assumes {calc['hedge_symbol']} moves exactly {beta:.4f}x less than "
        f"{calc['underlying_symbol']}'s move (the beta relationship) – real moves deviate from this, which is
"
        "exactly the risk a beta hedge doesn't cover."
    )

def render_orders(d: dict):
    st.subheader("Open orders")
    open_orders = d["open_orders"]
    if open_orders.empty:
        st.caption("No open orders.")
    else:
        st.dataframe(open_orders, use_container_width=True, hide_index=True)

    st.subheader("Recent fills (last 90 days)")
    hist = d["order_history"]
    if hist.empty:
        st.caption("No recent fills.")
    else:
        st.dataframe(
            hist,
            use_container_width=True,
            hide_index=True,
            column_config={
                "price": st.column_config.NumberColumn(format="%.2f"),
                "amount": st.column_config.NumberColumn(format="%.2f"),
                "fees": st.column_config.NumberColumn(format="%.2f"),
            },
        )

def render_journal(d: dict):
    hist = d["order_history"]
    log = journal.build_journal(hist)
    st.subheader("Trade journal")
    if log.empty:
        st.caption("No filled trades in range to journal yet.")
        return

    st.dataframe(log, use_container_width=True, hide_index=True)

```

```

csv_bytes = log.to_csv(index=False).encode("utf-8")
st.download_button(
    "Download CSV",
    data=csv_bytes,
    file_name=f"trade_journal_{pd.Timestamp.now():%Y%m%d}.csv",
    mime="text/csv",
    type="primary",
)
if st.button("Save a copy to exports/"):
    path = journal.export_csv(log)
    st.success(f"Saved to {path}")

def render_win_rate(d: dict):
    st.subheader("Win rate")
    st.caption(
        "Realized round-trip trades, FIFO-matched from your filled order history (same 90-day window as "
        "Orders & History). Equity matching is exact; options are matched per underlying symbol only, since "
        "fill history doesn't carry per-contract strike/expiration identity – if you hold multiple different "
        "contracts on the same underlying at once, fills across them can get cross-matched. Only **closed** "
        "trades count here; open positions with no matching exit aren't included."
    )

    trips_df = performance.round_trips_to_frame(performance.match_round_trips(d["order_history"]))
    if trips_df.empty:
        st.caption("No closed round-trip trades in the last 90 days yet.")
        return

    stats = performance.win_rate_stats(trips_df)
    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Win rate", f"{stats['win_rate']:.1%}", f"{stats['wins']}W / {stats['losses']}L")
    c2.metric("Total realized P&L", money(stats["total_realized_pl"]))
    c3.metric("Avg win / avg loss", f"{money(stats['avg_win'] or 0)} / {money(stats['avg_loss'] or 0)}")
    c4.metric("Profit factor", f"{stats['profit_factor']:.2f}" if stats["profit_factor"] is not None else
    "-")

    st.markdown("##### By instrument type")
    by_type = performance.win_rate_by_instrument_type(trips_df)
    st.dataframe(
        by_type,
        use_container_width=True,
        hide_index=True,
        column_config={
            "win_rate": st.column_config.NumberColumn(format="percent"),
            "total_realized_pl": st.column_config.NumberColumn(format="$.2f"),
        },
    )

    st.markdown("##### Cumulative realized P&L")
    chart_df = trips_df.sort_values("exit_date")
    cum_pl = chart_df["realized_pl"].cumsum()
    fig = go.Figure()
    fig.add_trace(
        go.Scatter(
            x=chart_df["exit_date"],
            y=cum_pl,
            mode="lines",
            line=dict(color=colors.CATEGORICAL[0], width=2),
            hovertemplate="%{x|%b %d}<br>${y:,.2f}<extra></extra>",

```

```

    )
)
fig.update_layout(
    height=280,
    margin=dict(l=10, r=10, t=10, b=10),
    plot_bgcolor=colors.SURFACE,
    paper_bgcolor=colors.SURFACE,
    xaxis=dict(showgrid=False, color=colors.INK_MUTED),
    yaxis=dict(showgrid=True, gridcolor=colors.GRIDLINE, zerolinecolor=colors.INK_MUTED,
color=colors.INK_MUTED, tickprefix="$"),
    showlegend=False,
)
st.plotly_chart(fig, use_container_width=True)

st.markdown("#### Closed trades")
st.dataframe(
    trips_df.sort_values("exit_date", ascending=False),
    use_container_width=True,
    hide_index=True,
    column_config={
        "entry_price": st.column_config.NumberColumn(format="$%.2f"),
        "exit_price": st.column_config.NumberColumn(format="$%.2f"),
        "realized_pl": st.column_config.NumberColumn(format="$%.2f"),
        "holding_days": st.column_config.NumberColumn(format="%.1f"),
    },
)

def main():
    st.title("📊 Vol Trading Dashboard")

    if is_demo_mode():
        st.info(
            "🤖 **Demo mode** – every number on this page is synthetic (generated by `app/demo_data.py`), "
            "regenerated deterministically per session. This is not a real brokerage account, and no live "
            "trading connection exists in this deployment. The pricing/Greeks math itself (`risk_tool/`) is "
            "the real engine – only the input data is fake.",
            icon="🤖",
        )

    if not ensure_logged_in():
        return

    top_l, top_r = st.columns([6, 1])
    with top_l:
        st.caption("Data refreshes every 60s while the app is open. Use Refresh for an immediate pull.")
    with top_r:
        if st.button("Refresh"):
            load_data.clear()
        if not is_demo_mode() and st.button("Log out"):
            logout()
            st.rerun()

    d = load_data()

    tabs = st.tabs(
        [
            "Overview", "Positions & Greeks", "Vol Exposure", "Vol Skew", "Risk Tool",
            "Correlation Explorer", "Hedge Calculator", "Orders & History", "Win Rate", "Journal / Export",
        ]
    )

```

```
)
with tabs[0]:
    render_overview(d)
with tabs[1]:
    render_positions(d)
with tabs[2]:
    render_vol_exposure(d)
with tabs[3]:
    render_vol_skew(d)
with tabs[4]:
    render_risk_tool(d)
    st.divider()
    render_position_monitor(d)
with tabs[5]:
    render_correlation_explorer(d)
with tabs[6]:
    render_hedge_calculator(d)
with tabs[7]:
    render_orders(d)
with tabs[8]:
    render_win_rate(d)
with tabs[9]:
    render_journal(d)

if __name__ == "__main__":
    main()
```

tests/ — 122 passing cases, all independent of a live connection

tests/test_pricing.py

Tests the pricing engine against facts independent of the code itself, not against its own output: a published Hull textbook value ($S=42$, $K=40$, $r=10\%$, $\sigma=20\%$, $T=0.5y \rightarrow \text{call} \approx \4.76 , $\text{put} \approx \$0.81$), put-call parity as an exact algebraic identity (checked to $1e-9$, not approximated), the CRR binomial tree converging to Black-Scholes as steps increase, and the specific fact that an American call on a non-dividend stock must price identically to European (never optimal to early-exercise) while an American put can be worth strictly more — the classic textbook divergence case.

```
"""Correctness tests for risk_tool.pricing.

These check the pricing engine against independently-known facts, not
against itself: a published textbook example, an algebraic identity
(put-call parity) that must hold exactly regardless of inputs, and a
known qualitative fact about American vs. European exercise.
"""

import math

import pytest

from risk_tool.pricing import (
    binomial_crr_price,
    black_scholes_price,
    compute_d1_d2,
    implied_volatility,
    itm_probability,
)

# Hull, "Options, Futures, and Other Derivatives" – the standard textbook
# example:  $S=42$ ,  $K=40$ ,  $r=10\%$ ,  $\sigma=20\%$ ,  $T=0.5y$ , no dividend.
# Published answers: call  $\approx 4.76$ , put  $\approx 0.81$ .
HULL_PARAMS = dict(S=42.0, K=40.0, T=0.5, r=0.10, q=0.0, sigma=0.20)

def test_black_scholes_matches_known_textbook_value():
    call = black_scholes_price(**HULL_PARAMS, option_type="call")
    put = black_scholes_price(**HULL_PARAMS, option_type="put")
    assert call == pytest.approx(4.76, abs=0.02)
    assert put == pytest.approx(0.81, abs=0.02)

@pytest.mark.parametrize(
    "S,K,T,r,q,sigma",
    [
        (100, 100, 1.0, 0.05, 0.0, 0.2),
        (55, 60, 0.25, 0.03, 0.02, 0.35),
        (200, 150, 2.0, 0.01, 0.0, 0.5),
        (42, 40, 0.5, 0.10, 0.0, 0.2),
    ],
)

def test_put_call_parity_holds_exactly(S, K, T, r, q, sigma):
    """ $C - P = S e^{-qT} - K e^{-rT}$  must hold algebraically for any inputs -
```

```

this isn't an approximation, so the tolerance is tight."""
call = black_scholes_price(S, K, T, r, q, sigma, "call")
put = black_scholes_price(S, K, T, r, q, sigma, "put")
lhs = call - put
rhs = S * math.exp(-q * T) - K * math.exp(-r * T)
assert lhs == pytest.approx(rhs, abs=1e-9)

def test_itm_probability_is_bounded_and_complementary():
    """N(d2) for a call and N(-d2) for a put must sum to 1 - they're
    complementary events under the same risk-neutral measure."""
    params = dict(S=100, K=105, T=0.5, r=0.04, q=0.0, sigma=0.25)
    p_call_itm = itm_probability(**params, option_type="call")
    p_put_itm = itm_probability(**params, option_type="put")
    assert 0.0 <= p_call_itm <= 1.0
    assert 0.0 <= p_put_itm <= 1.0
    assert p_call_itm + p_put_itm == pytest.approx(1.0, abs=1e-12)

def test_compute_d1_d2_rejects_invalid_inputs():
    with pytest.raises(ValueError):
        compute_d1_d2(S=100, K=100, T=0, r=0.05, q=0, sigma=0.2) # T=0
    with pytest.raises(ValueError):
        compute_d1_d2(S=100, K=100, T=1, r=0.05, q=0, sigma=0) # sigma=0

class TestBinomialTree:
    params = dict(S=100.0, K=100.0, T=1.0, r=0.05, sigma=0.25)

    def test_european_binomial_converges_to_black_scholes_no_dividend(self):
        for option_type in ("call", "put"):
            bs = black_scholes_price(**self.params, q=0.0, option_type=option_type)
            crr = binomial_crr_price(**self.params, q=0.0, option_type=option_type, american=False,
n_steps=500)
            assert crr == pytest.approx(bs, abs=0.05)

    def test_european_binomial_converges_to_black_scholes_with_dividend(self):
        for option_type in ("call", "put"):
            bs = black_scholes_price(**self.params, q=0.03, option_type=option_type)
            crr = binomial_crr_price(**self.params, q=0.03, option_type=option_type, american=False,
n_steps=500)
            assert crr == pytest.approx(bs, abs=0.05)

    def test_american_call_equals_european_when_no_dividend(self):
        """It is never optimal to early-exercise a call on a non-dividend
        stock (you'd throw away remaining time value for nothing) - so the
        American and European CRR prices must match exactly here. If they
        don't, the early-exercise check in binomial_crr_price is broken."""
        european = binomial_crr_price(**self.params, q=0.0, option_type="call", american=False, n_steps=300)
        american = binomial_crr_price(**self.params, q=0.0, option_type="call", american=True, n_steps=300)
        assert american == pytest.approx(european, abs=1e-9)

    def test_american_put_has_positive_early_exercise_premium(self):
        """Puts (unlike calls) can have real early-exercise value - a deep
        ITM put on a non-dividend stock is the classic case, since you'd
        rather collect the strike now and earn interest on it. American
        should therefore be strictly worth more than European here."""
        deep_itm_params = dict(S=50.0, K=100.0, T=1.0, r=0.08, sigma=0.20)
        european = binomial_crr_price(**deep_itm_params, q=0.0, option_type="put", american=False,
n_steps=300)

```

```

    american = binomial_crr_price(**deep_itm_params, q=0.0, option_type="put", american=True,
n_steps=300)
    assert american > european

def test_rejects_non_positive_steps(self):
    with pytest.raises(ValueError):
        binomial_crr_price(**self.params, q=0.0, option_type="call", n_steps=0)

class TestImpliedVolatility:
    @pytest.mark.parametrize("option_type", ["call", "put"])
    @pytest.mark.parametrize("true_sigma", [0.15, 0.30, 0.60, 1.10])
    def test_round_trip_recovers_known_sigma(self, option_type, true_sigma):
        """Price at a known sigma, then invert – should recover it, whether
        Newton-Raphson converges cleanly or falls back to Brent's method."""
        params = dict(S=100.0, K=100.0, T=0.5, r=0.03, q=0.0)
        price = black_scholes_price(**params, sigma=true_sigma, option_type=option_type)
        recovered = implied_volatility(price, **params, option_type=option_type)
        assert recovered == pytest.approx(true_sigma, abs=1e-4)

    def test_low_vega_regime_still_recovers_correctly(self):
        """Short-dated, meaningfully OTM options sit in the low-vega regime
        where Newton-Raphson is unreliable and the Brent's-method fallback
        matters – but the price here is still large enough to be a real,
        quotable price (not underflowed numerical noise), so recovery must
        still be accurate. (Going even further OTM makes the true price
        underflow toward machine noise, at which point IV is genuinely
        unrecoverable from the price alone – a real market maker couldn't
        back it out either, so that's not a case worth asserting on.)"""
        params = dict(S=100.0, K=115.0, T=0.05, r=0.03, q=0.0)
        true_sigma = 0.15
        price = black_scholes_price(**params, sigma=true_sigma, option_type="call")
        recovered = implied_volatility(price, **params, option_type="call")
        assert recovered == pytest.approx(true_sigma, abs=1e-3)

```

tests/test_greeks.py

Mostly algebraic identities that must hold for ANY inputs, which is a stronger test than checking against one example: $\text{delta_call} - \text{delta_put} = e^{-qT}$ exactly; gamma and vega are identical for calls and puts (they don't depend on option_type in Black-Scholes); theta is cross-checked against a finite-difference bump of the actual price function it's supposed to be the derivative of, not just its own closed-form formula.

```
"""Correctness tests for risk_tool.greeks – mostly identities that must
hold algebraically, plus sign/bounds sanity checks."""
import math

import pytest

from risk_tool.greeks import all_greeks, delta, gamma, n_d1_d2, rho, theta, vega
from risk_tool.pricing import black_scholes_price

PARAMS = dict(S=100.0, K=100.0, T=0.5, r=0.05, q=0.02, sigma=0.25)

def test_delta_call_minus_put_equals_dividend_discount():
    """delta_call - delta_put = e^{-qT} * (N(d1) - (N(d1)-1)) = e^{-qT} exactly,
    for any inputs – an algebraic identity, not an approximation."""
    d_call = delta(**PARAMS, option_type="call")
    d_put = delta(**PARAMS, option_type="put")
    expected = math.exp(-PARAMS["q"] * PARAMS["T"])
    assert (d_call - d_put) == pytest.approx(expected, abs=1e-12)

def test_delta_bounds():
    assert 0.0 <= delta(**PARAMS, option_type="call") <= 1.0
    assert -1.0 <= delta(**PARAMS, option_type="put") <= 0.0

def test_gamma_and_vega_positive_and_identical_for_call_and_put():
    """Gamma and vega don't depend on option_type in Black-Scholes – both
    calls and puts on the same contract specs have identical values."""
    g = gamma(**PARAMS)
    v = vega(**PARAMS)
    assert g > 0
    assert v > 0

def test_rho_call_positive_put_negative():
    """Higher rates help calls (present value of paying K later drops) and
    hurt puts – a standard sign-sanity check."""
    assert rho(**PARAMS, option_type="call") > 0
    assert rho(**PARAMS, option_type="put") < 0

def test_theta_matches_finite_difference_of_price():
    """Theta should match a small finite-difference bump of the actual BS
    price in time – this cross-checks the closed-form theta formula
    against the pricing function it's supposed to be the derivative of."""
```

```

for option_type in ("call", "put"):
    base_price = black_scholes_price(**PARAMS, option_type=option_type)
    dt = 1e-5
    bumped_params = {**PARAMS, "T": PARAMS["T"] - dt}
    bumped_price = black_scholes_price(**bumped_params, option_type=option_type)
    finite_diff_theta_per_year = (bumped_price - base_price) / dt
    finite_diff_theta_per_day = finite_diff_theta_per_year / 365.0

    analytic_theta = theta(**PARAMS, option_type=option_type, per_day=True)
    assert analytic_theta == pytest.approx(finite_diff_theta_per_day, abs=1e-3)

def test_n_d1_d2_matches_call_delta_when_no_dividend():
    """With q=0, call delta reduces exactly to N(d1) – the case where the
    'N(d1) is not quite delta' caveat disappears."""
    params_no_div = {**PARAMS, "q": 0.0}
    nd = n_d1_d2(**params_no_div)
    d_call = delta(**params_no_div, option_type="call")
    assert nd["N_d1"] == pytest.approx(d_call, abs=1e-12)

def test_all_greeks_returns_expected_keys():
    result = all_greeks(**PARAMS, option_type="call")
    expected_keys = {"delta", "gamma", "vega", "theta", "rho", "d1", "d2", "N_d1", "N_d2"}
    assert set(result.keys()) == expected_keys

```

tests/test_sizing.py

The load-bearing test in the whole sizing module: constructs a deliberately huge apparent edge ($p_{\text{win}}=0.95$, $b=5$) so full Kelly would suggest betting a large fraction of the account, then asserts the hard cap still wins and caps it at exactly 5%. Also verifies half-Kelly is exactly half of full Kelly, a negative-edge bet floors to zero size (not a negative bet), and recommended contracts is a floor division of the dollar size.

```
"""Correctness tests for risk_tool.sizing – the hard-cap-always-wins
guarantee is the most important property to verify here."""
import pytest

from risk_tool.config import RiskConfig
from risk_tool.sizing import half_kelly_fraction, kelly_fraction, size_position

def test_kelly_fraction_known_formula_value():
    #  $p=0.6$ ,  $b=1 \rightarrow f^* = (0.6 \times 1 - 0.4) / 1 = 0.2$ 
    assert kelly_fraction(p_win=0.6, b=1.0) == pytest.approx(0.2)

def test_kelly_fraction_negative_when_no_edge():
    #  $p=0.4$ ,  $b=1 \rightarrow f^* = (0.4 - 0.6) / 1 = -0.2$  – "don't take this bet"
    assert kelly_fraction(p_win=0.4, b=1.0) == pytest.approx(-0.2)

def test_kelly_fraction_scales_with_better_odds():
    assert kelly_fraction(p_win=0.5, b=3.0) > kelly_fraction(p_win=0.5, b=1.0)

def test_kelly_fraction_rejects_invalid_inputs():
    with pytest.raises(ValueError):
        kelly_fraction(p_win=1.5, b=1.0)
    with pytest.raises(ValueError):
        kelly_fraction(p_win=0.5, b=0.0)

def test_half_kelly_is_exactly_half_of_full_kelly():
    assert half_kelly_fraction(0.6, 1.0) == pytest.approx(kelly_fraction(0.6, 1.0) / 2)

class TestSizePosition:
    def test_hard_cap_always_wins_when_kelly_suggests_more(self):
        """Construct a huge apparent edge (high  $p_{\text{win}}$ , big payout) so full
        Kelly would suggest betting a large fraction of the account – the
        hard cap must still win."""
        config = RiskConfig(use_half_kelly=False, max_position_pct_of_account=0.05)
        result = size_position(
            account_size=100_000,
            premium_per_contract=1.0,
            p_win=0.95,
            profit_if_win_per_contract=5.0, #  $b = 5$ , huge edge at  $p=0.95$ 
            config=config,
        )
        assert result.capped_by_hard_limit is True
```

```

    assert result.recommended_dollar_size == pytest.approx(5_000) # 5% of 100k
    assert result.kelly_dollar_size > result.recommended_dollar_size

def test_kelly_wins_when_smaller_than_cap(self):
    """A modest, realistic edge should size well under the 5% cap –
    Kelly's (halved) suggestion should be what's actually used.

    p=0.52, b=1 -> full Kelly f*=(0.52-0.48)/1=0.04, half-Kelly=0.02 (2%),
    clear of the 5% cap (chosen deliberately, not coincidentally equal to it)."""
    config = RiskConfig(use_half_kelly=True, max_position_pct_of_account=0.05)
    result = size_position(
        account_size=100_000,
        premium_per_contract=2.0,
        p_win=0.52,
        profit_if_win_per_contract=2.0, # b = 1
        config=config,
    )
    assert result.capped_by_hard_limit is False
    assert result.recommended_dollar_size == pytest.approx(result.kelly_dollar_size)
    assert result.recommended_dollar_size < 5_000

def test_negative_edge_floors_to_zero_size(self):
    config = RiskConfig(use_half_kelly=True)
    result = size_position(
        account_size=100_000,
        premium_per_contract=2.0,
        p_win=0.3,
        profit_if_win_per_contract=2.0, # b = 1, p=0.3 -> negative Kelly
        config=config,
    )
    assert result.kelly_fraction_raw < 0
    assert result.kelly_fraction_used == 0.0
    assert result.recommended_dollar_size == 0.0
    assert result.recommended_contracts == 0

def test_half_kelly_config_flag_actually_halves_the_size(self):
    common_kwargs = dict(account_size=100_000, premium_per_contract=2.0, p_win=0.6,
profit_if_win_per_contract=2.0)
    full = size_position(**common_kwargs, config=RiskConfig(use_half_kelly=False,
max_position_pct_of_account=1.0))
    half = size_position(**common_kwargs, config=RiskConfig(use_half_kelly=True,
max_position_pct_of_account=1.0))
    assert half.kelly_dollar_size == pytest.approx(full.kelly_dollar_size / 2)

def test_recommended_contracts_is_floor_of_dollar_size(self):
    config = RiskConfig(use_half_kelly=False, max_position_pct_of_account=0.05)
    result = size_position(account_size=10_000, premium_per_contract=1.37, p_win=0.6,
profit_if_win_per_contract=1.37, config=config)
    contract_cost = 1.37 * 100
    assert result.recommended_contracts == int(result.recommended_dollar_size // contract_cost)

def test_rejects_non_positive_account_or_premium(self):
    with pytest.raises(ValueError):
        size_position(account_size=0, premium_per_contract=1.0, p_win=0.5,
profit_if_win_per_contract=1.0)
    with pytest.raises(ValueError):
        size_position(account_size=10_000, premium_per_contract=0, p_win=0.5,
profit_if_win_per_contract=1.0)

```

tests/test_risk_manager.py

Every one of the five exit rules tested at its exact boundary (profit target triggers at precisely +100%, stop loss at precisely -50%), plus the option-type-specific direction logic for the theta/OTM exit (OTM means something different for a call than a put) and the delta exit correctly using absolute value so it works the same for puts (negative delta) as calls. Portfolio governors are tested both for triggering when a cap is breached and for correctly NOT triggering when a cap is left unset (None) — an unset cap must never accidentally behave like a zero cap.

```
"""Correctness tests for risk_tool.risk_manager – each rule is tested for
both its triggered and not-triggered branch, using numbers you can check
by hand."""
import pytest

from risk_tool.config import RiskConfig
from risk_tool.risk_manager import (
    Position,
    check_delta_exit,
    check_iv_crush_exit,
    check_portfolio_governors,
    check_profit_target,
    check_stop_loss,
    check_theta_exit,
    PortfolioState,
    pct_pnl,
)

CONFIG = RiskConfig(
    profit_target_pct=1.00,
    stop_loss_pct=-0.50,
    delta_exit_threshold=0.15,
    theta_exit_dte_threshold=3,
    iv_crush_threshold_points=0.20,
)

def make_position(**overrides) -> Position:
    defaults = dict(
        symbol="TEST",
        option_type="call",
        strike=100.0,
        spot=105.0,
        dte=20,
        quantity=1,
        entry_premium=2.00,
        current_premium=2.00,
        entry_iv=0.30,
        current_iv=0.30,
        delta=0.50,
    )
    defaults.update(overrides)
    return Position(**defaults)
```

```

def test_pct_pnl_basic():
    pos = make_position(entry_premium=2.00, current_premium=3.00)
    assert pct_pnl(pos) == pytest.approx(0.5)

class TestProfitTarget:
    def test_triggers_at_exactly_100pct_gain(self):
        pos = make_position(entry_premium=2.00, current_premium=4.00) # +100%
        signal = check_profit_target(pos, CONFIG)
        assert signal.triggered is True
        assert "100%" in signal.reason or "1" in signal.reason

    def test_does_not_trigger_below_target(self):
        pos = make_position(entry_premium=2.00, current_premium=3.00) # +50%
        assert check_profit_target(pos, CONFIG).triggered is False

class TestStopLoss:
    def test_triggers_at_exactly_neg_50pct(self):
        pos = make_position(entry_premium=2.00, current_premium=1.00) # -50%
        assert check_stop_loss(pos, CONFIG).triggered is True

    def test_does_not_trigger_above_stop(self):
        pos = make_position(entry_premium=2.00, current_premium=1.50) # -25%
        assert check_stop_loss(pos, CONFIG).triggered is False

class TestDeltaExit:
    def test_triggers_when_delta_decays_below_threshold(self):
        pos = make_position(delta=0.10)
        assert check_delta_exit(pos, CONFIG).triggered is True

    def test_does_not_trigger_with_healthy_delta(self):
        pos = make_position(delta=0.45)
        assert check_delta_exit(pos, CONFIG).triggered is False

    def test_uses_absolute_value_for_puts(self):
        pos = make_position(option_type="put", delta=-0.10)
        assert check_delta_exit(pos, CONFIG).triggered is True

class TestThetaExit:
    def test_triggers_when_near_expiry_and_otm_call(self):
        pos = make_position(option_type="call", spot=95.0, strike=100.0, dte=2) # OTM call, 2 DTE
        assert check_theta_exit(pos, CONFIG).triggered is True

    def test_does_not_trigger_when_near_expiry_but_itm(self):
        pos = make_position(option_type="call", spot=110.0, strike=100.0, dte=2) # ITM call, 2 DTE
        assert check_theta_exit(pos, CONFIG).triggered is False

    def test_does_not_trigger_when_otm_but_plenty_of_time(self):
        pos = make_position(option_type="call", spot=95.0, strike=100.0, dte=30)
        assert check_theta_exit(pos, CONFIG).triggered is False

    def test_put_otm_direction_is_correct(self):
        # A put is OTM when spot > strike.
        pos = make_position(option_type="put", spot=110.0, strike=100.0, dte=1)
        assert check_theta_exit(pos, CONFIG).triggered is True
        pos_itm = make_position(option_type="put", spot=90.0, strike=100.0, dte=1)

```

```

    assert check_theta_exit(pos_itm, CONFIG).triggered is False

class TestIvCrushExit:
    def test_triggers_when_iv_drops_past_threshold(self):
        pos = make_position(entry_iv=0.50, current_iv=0.25) # 25-point drop
        assert check_iv_crush_exit(pos, CONFIG).triggered is True

    def test_does_not_trigger_on_small_iv_move(self):
        pos = make_position(entry_iv=0.50, current_iv=0.45) # 5-point drop
        assert check_iv_crush_exit(pos, CONFIG).triggered is False

    def test_does_not_trigger_when_iv_rises(self):
        pos = make_position(entry_iv=0.30, current_iv=0.45)
        assert check_iv_crush_exit(pos, CONFIG).triggered is False

class TestPortfolioGovernors:
    def test_halts_when_daily_loss_limit_breached(self):
        config = RiskConfig(daily_max_loss_pct=0.03)
        state = PortfolioState(account_size=100_000, daily_pnl=-3_500, net_delta_dollars=0,
net_vega_dollars=0)
        result = check_portfolio_governors(state, config)
        assert result.halted is True
        assert any("Daily P&L" in r for r in result.reasons)

    def test_does_not_halt_within_daily_loss_limit(self):
        config = RiskConfig(daily_max_loss_pct=0.03)
        state = PortfolioState(account_size=100_000, daily_pnl=-1_000, net_delta_dollars=0,
net_vega_dollars=0)
        result = check_portfolio_governors(state, config)
        assert result.halted is False

    def test_delta_cap_enforced_when_set(self):
        config = RiskConfig(daily_max_loss_pct=1.0, max_net_delta_dollars=10_000)
        state = PortfolioState(account_size=100_000, daily_pnl=0, net_delta_dollars=15_000,
net_vega_dollars=0)
        result = check_portfolio_governors(state, config)
        assert result.halted is True
        assert any("delta" in r.lower() for r in result.reasons)

    def test_delta_cap_not_enforced_when_unset(self):
        config = RiskConfig(daily_max_loss_pct=1.0, max_net_delta_dollars=None)
        state = PortfolioState(account_size=100_000, daily_pnl=0, net_delta_dollars=999_999,
net_vega_dollars=0)
        result = check_portfolio_governors(state, config)
        assert result.halted is False

    def test_vega_cap_enforced_when_set(self):
        config = RiskConfig(daily_max_loss_pct=1.0, max_net_vega_dollars=500)
        state = PortfolioState(account_size=100_000, daily_pnl=0, net_delta_dollars=0, net_vega_dollars=800)
        result = check_portfolio_governors(state, config)
        assert result.halted is True
        assert any("vega" in r.lower() for r in result.reasons)

```

tests/test_realized_vol.py

Close-to-close, Parkinson, and Garman-Klass estimators checked against independently hand-computed values on small fixed datasets, plus a sanity check that Parkinson and Garman-Klass (which use more of each day's price action) roughly agree in magnitude on realistic data. GARCH is tested for a well-posed, stationary fit ($\alpha + \beta < 1$) and for its forecast correctly converging to the long-run unconditional variance as the horizon grows — the mathematical property that makes 'long-run GARCH forecast' a well-defined concept at all.

```
"""Correctness tests for risk_tool.realized_vol.

For the range estimators (Parkinson, Garman-Klass) we cross-check the
module's output against the same formula recomputed independently in the
test, on a small hand-built OHLC sample — this catches implementation bugs
(off-by-one in n, wrong annualization, wrong log base) without relying on
recovering a "true" simulated vol, which is inherently noisy and would make
the test flaky.
"""

import math

import numpy as np
import pandas as pd
import pytest

from risk_tool.realized_vol import (
    close_to_close_vol,
    fit_garch_11,
    garch_forecast_vol,
    garman_klass_vol,
    parkinson_vol,
)

def test_close_to_close_vol_matches_independent_calculation():
    closes = pd.Series([100.0, 102.0, 101.0, 105.0, 103.0])
    expected_log_returns = np.diff(np.log(closes.to_numpy()))
    expected = expected_log_returns.std(ddof=1) * math.sqrt(252)
    assert close_to_close_vol(closes, annualization_factor=252) == pytest.approx(expected, rel=1e-9)

def test_close_to_close_vol_requires_enough_data():
    with pytest.raises(ValueError):
        close_to_close_vol(pd.Series([100.0]))

def test_parkinson_matches_independent_calculation():
    high = pd.Series([102.0, 105.0, 103.0])
    low = pd.Series([99.0, 101.0, 100.0])
    log_hl = np.log(high.to_numpy() / low.to_numpy())
    expected_var = np.sum(log_hl**2) / (4 * len(high) * math.log(2))
    expected = math.sqrt(expected_var * 252)
    assert parkinson_vol(high, low, annualization_factor=252) == pytest.approx(expected, rel=1e-9)
```

```

def test_parkinson_rejects_mismatched_lengths():
    with pytest.raises(ValueError):
        parkinson_vol(pd.Series([100.0, 101.0]), pd.Series([99.0]))

def test_garman_klass_matches_independent_calculation():
    open_ = pd.Series([100.0, 102.0, 101.0])
    high = pd.Series([103.0, 104.0, 102.5])
    low = pd.Series([99.0, 101.0, 100.0])
    close = pd.Series([102.0, 101.5, 102.0])

    log_hl = np.log(high.to_numpy() / low.to_numpy())
    log_co = np.log(close.to_numpy() / open_.to_numpy())
    term = 0.5 * log_hl**2 - (2 * math.log(2) - 1) * log_co**2
    expected = math.sqrt((np.sum(term) / len(close)) * 252)

    assert garman_klass_vol(open_, high, low, close, annualization_factor=252) == pytest.approx(expected,
rel=1e-9)

def test_garman_klass_and_parkinson_agree_in_magnitude_on_realistic_data():
    """Sanity check, not a precision check: both range estimators should
    land in the same ballpark on the same realistic-looking data, since
    they're measuring the same underlying phenomenon."""
    rng = np.random.default_rng(42)
    n = 100
    close = 100 * np.exp(np.cumsum(rng.normal(0, 0.015, n)))
    open_ = close * (1 + rng.normal(0, 0.002, n))
    high = np.maximum(open_, close) * (1 + np.abs(rng.normal(0, 0.005, n)))
    low = np.minimum(open_, close) * (1 - np.abs(rng.normal(0, 0.005, n)))

    p = parkinson_vol(pd.Series(high), pd.Series(low))
    gk = garman_klass_vol(pd.Series(open_), pd.Series(high), pd.Series(low), pd.Series(close))
    assert 0.05 < p < 1.0
    assert 0.05 < gk < 1.0
    assert p == pytest.approx(gk, rel=0.5) # same ballpark, not a precision match

class TestGarch:
    @pytest.fixture
    def returns(self):
        """Synthetic returns with genuine volatility clustering (a GARCH-like
        process), so the fit has real structure to recover rather than pure
        noise."""
        rng = np.random.default_rng(7)
        n = 800
        omega, alpha, beta = 1e-6, 0.08, 0.88
        sigma2 = np.empty(n)
        r = np.empty(n)
        sigma2[0] = omega / (1 - alpha - beta)
        r[0] = rng.normal(0, math.sqrt(sigma2[0]))
        for t in range(1, n):
            sigma2[t] = omega + alpha * r[t - 1]**2 + beta * sigma2[t - 1]
            r[t] = rng.normal(0, math.sqrt(sigma2[t]))
        return pd.Series(r)

    def test_fit_is_stationary_and_well_posed(self, returns):
        fit = fit_garch_11(returns)
        assert fit.omega > 0

```

```

    assert fit.alpha >= 0
    assert fit.beta >= 0
    assert fit.persistence < 1.0 # stationarity – required for long_run_variance to make sense
    assert fit.long_run_variance > 0

def test_requires_minimum_history(self):
    with pytest.raises(ValueError):
        fit_garch_11(pd.Series(np.random.default_rng(0).normal(0, 0.01, 10)))

def test_forecast_converges_to_long_run_vol_at_long_horizon(self, returns):
    fit = fit_garch_11(returns)
    long_horizon_vol = garch_forecast_vol(fit, horizon_days=5000)
    expected_long_run_vol = math.sqrt(fit.long_run_variance * 252)
    assert long_horizon_vol == pytest.approx(expected_long_run_vol, rel=1e-3)

def test_one_step_forecast_is_positive_and_finite(self, returns):
    fit = fit_garch_11(returns)
    vol = garch_forecast_vol(fit, horizon_days=1)
    assert vol > 0
    assert math.isfinite(vol)

def test_rejects_invalid_horizon(self, returns):
    fit = fit_garch_11(returns)
    with pytest.raises(ValueError):
        garch_forecast_vol(fit, horizon_days=0)

```

tests/test_strike_selection.py

Covers candidate-strike generation (correctly spans both sides of ATM, rounded to the given increment), evaluate_strike() checked against a fully manual EV calculation, and — the most conceptually important test in this file — that the realized-vs-implied edge comparison uses market IV for the premium (what you actually pay) and your own vol for the win probability in exactly the right places, with the resulting vol_spread sign matching realized-minus-implied as documented.

```
"""Correctness tests for risk_tool.strike_selection.

Two things matter most here: the EV arithmetic itself (checked against a
manual calculation), and that compare_implied_vs_realized_ev actually uses
market IV vs. your own vol estimate in exactly the documented places (not
accidentally mixing them up) — that wiring correctness is the whole point
of the realized-vs-implied feature.
"""
import pytest

from risk_tool.config import RiskConfig
from risk_tool.pricing import black_scholes_price, itm_probability
from risk_tool.strike_selection import (
    compare_implied_vs_realized_ev,
    evaluate_strike,
    generate_candidate_strikes,
    rank_strikes_by_ev,
)

S, T, r, q, sigma = 100.0, 0.25, 0.03, 0.0, 0.30

def test_generate_candidate_strikes_rounds_to_atm_and_spans_both_sides():
    strikes = generate_candidate_strikes(spot=101.0, strike_increment=5.0, num_each_side=2)
    assert strikes == [90.0, 95.0, 100.0, 105.0, 110.0]

def test_generate_candidate_strikes_rejects_invalid_inputs():
    with pytest.raises(ValueError):
        generate_candidate_strikes(spot=100, strike_increment=0, num_each_side=2)
    with pytest.raises(ValueError):
        generate_candidate_strikes(spot=100, strike_increment=5, num_each_side=-1)

def test_evaluate_strike_ev_matches_manual_calculation():
    config = RiskConfig(default_profit_target_pct=1.0, min_risk_reward_ratio=2.0)
    K = 105.0
    result = evaluate_strike(S, K, T, r, q, sigma, "call", config)

    expected_premium = black_scholes_price(S, K, T, r, q, sigma, "call")
    expected_p_win = itm_probability(S, K, T, r, q, sigma, "call")
    expected_profit_if_win = expected_premium * 1.0
    expected_ev = expected_p_win * expected_profit_if_win - (1 - expected_p_win) * expected_premium

    assert result.premium == pytest.approx(expected_premium)
    assert result.p_win_risk_neutral == pytest.approx(expected_p_win)
```

```

    assert result.ev == pytest.approx(expected_ev)
    assert result.risk_reward == pytest.approx(1.0) # profit_target_pct=1.0 -> ratio is always 1:1 by
construction
    assert result.is_poor_setup is True # 1:1 < the 2:1 minimum

def test_higher_profit_target_improves_risk_reward_and_can_clear_poor_setup_flag():
    config = RiskConfig(min_risk_reward_ratio=2.0)
    poor = evaluate_strike(S, 105.0, T, r, q, sigma, "call", config, profit_target_pct=1.0)
    good = evaluate_strike(S, 105.0, T, r, q, sigma, "call", config, profit_target_pct=2.5)
    assert poor.is_poor_setup is True
    assert good.is_poor_setup is False
    assert good.risk_reward > poor.risk_reward

def test_call_itm_probability_decreases_monotonically_with_strike():
    """A well-known, safe monotonicity fact: for a call, higher strikes are
    strictly less likely to finish ITM, holding everything else fixed."""
    strikes = [80.0, 90.0, 100.0, 110.0, 120.0]
    probs = [evaluate_strike(S, K, T, r, q, sigma, "call").p_win_risk_neutral for K in strikes]
    assert probs == sorted(probs, reverse=True)

def test_rank_strikes_by_ev_sorts_descending():
    strikes = generate_candidate_strikes(spot=S, strike_increment=5, num_each_side=3)
    ranked = rank_strikes_by_ev(S, T, r, q, sigma, "call", strikes)
    evs = [s.ev for s in ranked]
    assert evs == sorted(evs, reverse=True)
    assert len(ranked) == len(strikes)

def test_rank_strikes_drops_strikes_over_the_premium_cap():
    strikes = generate_candidate_strikes(spot=S, strike_increment=5, num_each_side=3)
    cheapest_premium = min(evaluate_strike(S, K, T, r, q, sigma, "call").premium for K in strikes)
    tight_cap = cheapest_premium * 100 * 1.01 # only the single cheapest strike should survive
    ranked = rank_strikes_by_ev(S, T, r, q, sigma, "call", strikes, max_premium_dollars=tight_cap)
    assert len(ranked) >= 1
    assert all(s.premium * 100 <= tight_cap for s in ranked)

class TestRealizedVsImpliedEV:
    def test_uses_market_iv_and_realized_vol_in_exactly_the_right_places(self):
        K = 105.0
        market_iv = 0.30
        realized = 0.45
        result = compare_implied_vs_realized_ev(S, K, T, r, q, market_iv, realized, "call")

        expected_market_premium = black_scholes_price(S, K, T, r, q, market_iv, "call")
        assert result.market_premium == pytest.approx(expected_market_premium)

        # risk_neutral_ev must be priced AND probability-assigned with market_iv throughout.
        assert result.risk_neutral_ev.premium == pytest.approx(expected_market_premium)
        assert result.risk_neutral_ev.p_win_risk_neutral == pytest.approx(itm_probability(S, K, T, r, q,
market_iv, "call"))

        # edge_ev must keep the MARKET premium (that's what you actually pay)
        # but assign probability using the REALIZED/forecast vol.
        assert result.edge_ev.premium == pytest.approx(expected_market_premium)
        assert result.edge_ev.p_win_risk_neutral == pytest.approx(itm_probability(S, K, T, r, q, realized,
"call"))

```

```
    assert result.vol_spread == pytest.approx(realized - market_iv)

    def test_vol_spread_sign_matches_realized_minus_implied(self):
        result_higher = compare_implied_vs_realized_ev(S, 105.0, T, r, q, market_iv=0.30,
        realized_or_forecast_vol=0.45, option_type="call")
        result_lower = compare_implied_vs_realized_ev(S, 105.0, T, r, q, market_iv=0.30,
        realized_or_forecast_vol=0.15, option_type="call")
        assert result_higher.vol_spread > 0
        assert result_lower.vol_spread < 0
```

tests/test_performance.py

Exercises the FIFO round-trip matcher end to end: a simple long round trip, a simple short round trip, a losing trade correctly flagged as not a win, FIFO matching the oldest lot first when multiple entries exist, a partial fill correctly splitting into multiple separate round trips, the 100x option contract multiplier being applied, and different symbols being matched completely independently of each other.

```
"""Correctness tests for app.performance -- FIFO round-trip matching and
win-rate stats, on hand-constructed synthetic fill data (no live Robinhood
session needed)."""
import pandas as pd
import pytest

from app.performance import match_round_trips, round_trips_to_frame, win_rate_by_instrument_type,
win_rate_stats

def fill(date, symbol, side, qty, price, instrument_type="equity"):
    return {"date": pd.Timestamp(date, tz="UTC"), "symbol": symbol, "instrument_type": instrument_type,
"side": side, "quantity": qty, "price": price}

def frame(*fills):
    return pd.DataFrame(list(fills))

class TestMatchRoundTrips:
    def test_empty_history_returns_no_trips(self):
        assert match_round_trips(pd.DataFrame()) == []

    def test_simple_long_round_trip(self):
        hist = frame(
            fill("2026-01-01", "AAPL", "buy", 10, 100.0),
            fill("2026-01-05", "AAPL", "sell", 10, 110.0),
        )
        trips = match_round_trips(hist)
        assert len(trips) == 1
        t = trips[0]
        assert t.direction == "long"
        assert t.quantity == 10
        assert t.realized_pl == pytest.approx(100.0) # (110-100)*10
        assert t.is_win is True

    def test_simple_short_round_trip(self):
        hist = frame(
            fill("2026-01-01", "TSLA", "sell", 5, 200.0),
            fill("2026-01-05", "TSLA", "buy", 5, 180.0),
        )
        trips = match_round_trips(hist)
        assert len(trips) == 1
        t = trips[0]
        assert t.direction == "short"
        assert t.realized_pl == pytest.approx(100.0) # (200-180)*5
        assert t.is_win is True
```

```

def test_losing_trade_is_flagged_not_a_win(self):
    hist = frame(
        fill("2026-01-01", "AAPL", "buy", 10, 100.0),
        fill("2026-01-05", "AAPL", "sell", 10, 90.0),
    )
    trips = match_round_trips(hist)
    assert trips[0].realized_pl == pytest.approx(-100.0)
    assert trips[0].is_win is False

def test_fifo_matches_oldest_lot_first(self):
    hist = frame(
        fill("2026-01-01", "AAPL", "buy", 10, 100.0),
        fill("2026-01-02", "AAPL", "buy", 10, 120.0),
        fill("2026-01-05", "AAPL", "sell", 10, 130.0),
    )
    trips = match_round_trips(hist)
    assert len(trips) == 1
    assert trips[0].entry_price == pytest.approx(100.0) # matched the FIRST lot, not the second
    assert trips[0].quantity == 10

def test_partial_fill_splits_into_multiple_trips(self):
    hist = frame(
        fill("2026-01-01", "AAPL", "buy", 10, 100.0),
        fill("2026-01-02", "AAPL", "buy", 10, 120.0),
        fill("2026-01-05", "AAPL", "sell", 15, 130.0),
    )
    trips = match_round_trips(hist)
    assert len(trips) == 2
    assert trips[0].quantity == 10 and trips[0].entry_price == pytest.approx(100.0)
    assert trips[1].quantity == 5 and trips[1].entry_price == pytest.approx(120.0)

def test_option_trade_applies_100x_multiplier(self):
    hist = frame(
        fill("2026-01-01", "SPY", "buy", 2, 5.0, instrument_type="option"),
        fill("2026-01-05", "SPY", "sell", 2, 7.0, instrument_type="option"),
    )
    trips = match_round_trips(hist)
    assert trips[0].realized_pl == pytest.approx((7.0 - 5.0) * 2 * 100)

def test_different_symbols_are_matched_independently(self):
    hist = frame(
        fill("2026-01-01", "AAPL", "buy", 10, 100.0),
        fill("2026-01-01", "TSLA", "buy", 5, 200.0),
        fill("2026-01-05", "AAPL", "sell", 10, 110.0),
    )
    trips = match_round_trips(hist)
    assert len(trips) == 1
    assert trips[0].symbol == "AAPL" # TSLA still open, not closed, so not a round trip yet

def test_unmatched_open_position_produces_no_trip(self):
    hist = frame(fill("2026-01-01", "AAPL", "buy", 10, 100.0))
    assert match_round_trips(hist) == []

class TestWinRateStats:
    def test_empty_trips_returns_none_fields_not_zero(self):
        stats = win_rate_stats(round_trips_to_frame([]))
        assert stats["total_trades"] == 0
        assert stats["win_rate"] is None

```

```

    assert stats["avg_win"] is None

def test_mixed_wins_and_losses(self):
    hist = frame(
        fill("2026-01-01", "AAPL", "buy", 10, 100.0),
        fill("2026-01-02", "AAPL", "sell", 10, 110.0), # win: +100
        fill("2026-01-03", "TSLA", "buy", 10, 200.0),
        fill("2026-01-04", "TSLA", "sell", 10, 190.0), # loss: -100
    )
    trips_df = round_trips_to_frame(match_round_trips(hist))
    stats = win_rate_stats(trips_df)
    assert stats["total_trades"] == 2
    assert stats["wins"] == 1
    assert stats["losses"] == 1
    assert stats["win_rate"] == pytest.approx(0.5)
    assert stats["avg_win"] == pytest.approx(100.0)
    assert stats["avg_loss"] == pytest.approx(-100.0)
    assert stats["profit_factor"] == pytest.approx(1.0)

def test_all_wins_has_no_defined_profit_factor_denominator_but_returns_none_for_no_losses(self):
    hist = frame(
        fill("2026-01-01", "AAPL", "buy", 10, 100.0),
        fill("2026-01-02", "AAPL", "sell", 10, 110.0),
    )
    stats = win_rate_stats(round_trips_to_frame(match_round_trips(hist)))
    assert stats["win_rate"] == 1.0
    assert stats["profit_factor"] is None # zero gross loss -- undefined, not infinity

class TestWinRateByInstrumentType:
    def test_splits_stats_per_instrument_type(self):
        hist = frame(
            fill("2026-01-01", "AAPL", "buy", 10, 100.0),
            fill("2026-01-02", "AAPL", "sell", 10, 110.0, instrument_type="equity"),
            fill("2026-01-03", "SPY", "buy", 1, 5.0, instrument_type="option"),
            fill("2026-01-04", "SPY", "sell", 1, 4.0, instrument_type="option"),
        )
        by_type = win_rate_by_instrument_type(round_trips_to_frame(match_round_trips(hist)))
        row = {r["instrument_type"]: r for _, r in by_type.iterrows()}
        assert row["equity"]["win_rate"] == pytest.approx(1.0)
        assert row["option"]["win_rate"] == pytest.approx(0.0)

```

tests/test_cli.py

End-to-end tests of the CLI's run() function — the full pipeline from command-line flags through strike ranking and sizing to printed output — both with and without the realized-vol edge check enabled, plus a check that the --full-kelly flag actually changes the sizing output relative to the half-Kelly default.

```
"""End-to-end smoke tests for the CLI – verifies the whole pipeline (strike
generation -> EV ranking -> sizing -> entry/exit levels) wires together and
produces sane output for both the market-IV-only and realized-vs-implied
code paths."""
from risk_tool.cli import run

def test_cli_runs_end_to_end_with_market_iv_only(capsys):
    exit_code = run([
        "--ticker", "AAPL",
        "--spot", "190",
        "--iv", "0.32",
        "--dte", "30",
        "--direction", "call",
        "--account", "50000",
        "--strike-increment", "5",
    ])
    captured = capsys.readouterr()
    assert exit_code == 0
    assert "Recommended strike" in captured.out
    assert "Position sizing" in captured.out
    assert "Pre-committed entry/exit levels" in captured.out

def test_cli_runs_end_to_end_with_realized_vol_edge_check(capsys):
    exit_code = run([
        "--ticker", "AAPL",
        "--spot", "190",
        "--iv", "0.32",
        "--dte", "30",
        "--direction", "put",
        "--account", "50000",
        "--strike-increment", "5",
        "--realized-vol", "0.45",
    ])
    captured = capsys.readouterr()
    assert exit_code == 0
    assert "Realized-vs-implied edge check" in captured.out
    assert "EDGE EV" in captured.out

def test_cli_full_kelly_flag_changes_output(capsys):
    run([
        "--ticker", "AAPL", "--spot", "190", "--iv", "0.32", "--dte", "30",
        "--direction", "call", "--account", "50000", "--strike-increment", "5", "--full-kelly",
    ])
    captured = capsys.readouterr()
    assert "Recommended strike" in captured.out
    assert "Position sizing" in captured.out
    assert "Pre-committed entry/exit levels" in captured.out
    assert "Realized-vs-implied edge check" in captured.out
    assert "EDGE EV" in captured.out
    assert "Full Kelly" in captured.out
```

```
captured = capsys.readouterr()
assert "full-Kelly" in captured.out
```

tests/test_ibkr_client.py

Tests MockIBKRClient's connect/disconnect state tracking, returning pre-configured quotes and positions, raising a clear error for an unconfigured symbol rather than failing silently, and defaulting to empty when constructed with nothing — the contract the real IBKRClient has to satisfy too, without ever needing a live IBKR connection to verify it.

```
"""MockIBKRClient tests – this is the fully testable half of ibkr_client.py.
The real IBKRClient requires a live TWS/Gateway connection and isn't
exercised here; its methods are thin and import ib_insync lazily precisely
so this module stays importable and testable without one."""
import pytest

from risk_tool.ibkr_client import LiveOptionPosition, LiveQuote, MockIBKRClient


def test_mock_client_connect_disconnect_tracks_state():
    client = MockIBKRClient()
    assert client.connected is False
    client.connect()
    assert client.connected is True
    client.disconnect()
    assert client.connected is False


def test_mock_client_returns_configured_quote():
    quote = LiveQuote(symbol="AAPL", bid=150.0, ask=150.2, last=150.1, mark=150.1)
    client = MockIBKRClient(quotes={"AAPL": quote})
    assert client.get_quote("AAPL") == quote


def test_mock_client_raises_for_unconfigured_symbol():
    client = MockIBKRClient()
    with pytest.raises(KeyError):
        client.get_quote("MSFT")


def test_mock_client_returns_configured_positions():
    pos = LiveOptionPosition(
        symbol="AAPL", option_type="call", strike=150.0, expiration="2026-08-21",
        quantity=2, avg_cost=3.50, current_price=4.00, delta=0.45, implied_vol=0.28,
    )
    client = MockIBKRClient(positions=[pos])
    assert client.get_option_positions() == [pos]


def test_mock_client_defaults_to_empty():
    client = MockIBKRClient()
    assert client.get_option_positions() == []
```

tests/test_hedge.py

Covers `estimate_beta()` recovering a known slope exactly from noise-free synthetic returns and correctly using only the overlapping index when two return series don't perfectly align; the exposure helpers' sign conventions (a long put's exposure is negative — short-equivalent); `size_hedge()` reproducing the exact XOM/USO worked example from this project's own design conversation, and futures with a 100x contract multiplier correctly coming out to 1/100th the units of an equivalent ETF hedge; and the three P&L functions (`option_intrinsic_pl`, `share_position_pl`, `hedge_instrument_pl`) checked against hand-computed values including the specific put-expires-worthless-losing-exactly-the-premium case.

```
"""Correctness tests for risk_tool.hedge – direction sign-flipping and the
exposure/beta/hedge math are the properties most worth pinning down."""
import numpy as np
import pandas as pd
import pytest

from risk_tool.hedge import (
    estimate_beta,
    hedge_instrument_pl,
    option_intrinsic_pl,
    option_position_exposure_shares,
    share_position_exposure_shares,
    share_position_pl,
    simple_returns,
    size_hedge,
)

def test_simple_returns_known_values():
    prices = pd.Series([100.0, 110.0, 99.0])
    ret = simple_returns(prices)
    assert list(ret.round(4)) == [0.1, -0.1]

class TestEstimateBeta:
    def test_recovers_known_slope(self):
        # Construct hedge_returns and underlying_returns = 0.5 * hedge_returns exactly (no noise).
        rng = np.random.default_rng(0)
        hedge_ret = pd.Series(rng.normal(0, 0.01, 200))
        underlying_ret = 0.5 * hedge_ret
        est = estimate_beta(underlying_ret, hedge_ret)
        assert est.beta == pytest.approx(0.5, abs=1e-9)
        assert est.r_squared == pytest.approx(1.0, abs=1e-9)

    def test_aligns_on_index_and_uses_only_overlap(self):
        hedge_ret = pd.Series([0.01, 0.02, -0.01, 0.03, 0.01], index=[0, 1, 2, 3, 4])
        underlying_ret = pd.Series([0.02, 0.04, -0.02, 0.06], index=[0, 1, 2, 3]) # missing index 4
        est = estimate_beta(underlying_ret, hedge_ret)
        assert est.n_obs == 4
        assert est.beta == pytest.approx(2.0, abs=1e-9)

    def test_rejects_too_few_observations(self):
        with pytest.raises(ValueError):
```

```

        estimate_beta(pd.Series([0.01, 0.02]), pd.Series([0.01, 0.02]))

    def test_rejects_zero_variance_hedge_returns(self):
        with pytest.raises(ValueError):
            estimate_beta(pd.Series([0.01, 0.02, 0.03]), pd.Series([0.0, 0.0, 0.0]))

class TestExposureHelpers:
    def test_option_position_exposure_shares_long_put_is_negative(self):
        # 5 contracts, delta -0.46 (long put), 100 shares/contract -> -230 shares equivalent
        exposure = option_position_exposure_shares(delta=-0.46, contracts=5, shares_per_contract=100)
        assert exposure == pytest.approx(-230.0)

    def test_share_position_exposure_shares_signs(self):
        assert share_position_exposure_shares(100, "long") == 100.0
        assert share_position_exposure_shares(100, "short") == -100.0

    def test_share_position_exposure_shares_rejects_bad_side(self):
        with pytest.raises(ValueError):
            share_position_exposure_shares(100, "sideways")

    def test_share_position_exposure_shares_rejects_negative_quantity(self):
        with pytest.raises(ValueError):
            share_position_exposure_shares(-10, "long")

class TestSizeHedge:
    def test_short_position_positive_beta_hedges_long(self):
        # Matches the XOM/USO worked example from the risk_tool README-adjacent chat:
        # short 100 shares @ $165, beta 0.3447 -> hedge dollars positive (buy), ~43 USO shares @ $130.44.
        exposure = share_position_exposure_shares(100, "short")
        result = size_hedge(exposure_shares=exposure, underlying_price=165.00, beta=0.3447,
hedge_price=130.44)
        assert result.exposure_dollars == pytest.approx(-16_500.0)
        assert result.hedge_dollars > 0
        assert result.direction == "long"
        assert result.hedge_units == pytest.approx(43.3, abs=0.5)

    def test_long_position_positive_beta_hedges_short(self):
        exposure = share_position_exposure_shares(100, "long")
        result = size_hedge(exposure_shares=exposure, underlying_price=165.00, beta=0.3447,
hedge_price=130.44)
        assert result.hedge_dollars < 0
        assert result.direction == "short"

    def test_negative_beta_flips_direction(self):
        exposure = share_position_exposure_shares(100, "short")
        result = size_hedge(exposure_shares=exposure, underlying_price=165.00, beta=-0.5, hedge_price=130.44)
        assert result.direction == "short"

    def test_futures_multiplier_divides_units_down(self):
        exposure = option_position_exposure_shares(delta=-0.4576, contracts=5, shares_per_contract=100)
        etf_result = size_hedge(exposure_shares=exposure, underlying_price=165.00, beta=0.3447,
hedge_price=65.00, hedge_contract_multiplier=1.0)
        mcl_result = size_hedge(exposure_shares=exposure, underlying_price=165.00, beta=0.3447,
hedge_price=65.00, hedge_contract_multiplier=100.0)
        assert mcl_result.hedge_units == pytest.approx(etf_result.hedge_units / 100.0)

    def test_rejects_non_positive_prices_and_multiplier(self):
        with pytest.raises(ValueError):

```

```

        size_hedge(exposure_shares=-100, underlying_price=0, beta=0.3, hedge_price=100)
    with pytest.raises(ValueError):
        size_hedge(exposure_shares=-100, underlying_price=100, beta=0.3, hedge_price=0)
    with pytest.raises(ValueError):
        size_hedge(exposure_shares=-100, underlying_price=100, beta=0.3, hedge_price=100,
hedge_contract_multiplier=0)

class TestOptionIntrinsicPl:
    def test_matches_the_worked_xom_put_example(self):
        # 12x $162.50 put, entry $1.99, XOM falls 10% to $149.37 at expiration.
        spot_at_scenario = 165.965 * 0.90
        pl = option_intrinsic_pl(spot_at_scenario, strike=162.50, premium=1.99, option_type="put",
contracts=12)
        intrinsic = 162.50 - spot_at_scenario
        assert pl == pytest.approx((intrinsic - 1.99) * 12 * 100)
        assert pl > 0 # deep enough ITM to be profitable net of premium

    def test_otm_at_expiration_loses_exactly_the_premium(self):
        # Put expires worthless when spot stays above strike.
        pl = option_intrinsic_pl(200.0, strike=162.50, premium=1.99, option_type="put", contracts=12)
        assert pl == pytest.approx(-1.99 * 12 * 100)

    def test_call_intrinsic_direction(self):
        pl_up = option_intrinsic_pl(220.0, strike=200.0, premium=5.0, option_type="call", contracts=1)
        pl_down = option_intrinsic_pl(180.0, strike=200.0, premium=5.0, option_type="call", contracts=1)
        assert pl_up > 0
        assert pl_down == pytest.approx(-5.0 * 100)

    def test_rejects_invalid_option_type(self):
        with pytest.raises(ValueError):
            option_intrinsic_pl(100.0, strike=100.0, premium=1.0, option_type="straddle", contracts=1)

class TestSharePositionPl:
    def test_long_gains_when_price_rises(self):
        assert share_position_pl(price_at_scenario=110.0, entry_price=100.0, exposure_shares=100) ==
pytest.approx(1000.0)

    def test_short_gains_when_price_falls(self):
        assert share_position_pl(price_at_scenario=90.0, entry_price=100.0, exposure_shares=-100) ==
pytest.approx(1000.0)

class TestHedgeInstrumentPl:
    def test_long_hedge_gains_when_hedge_price_rises(self):
        assert hedge_instrument_pl(hedge_price_at_scenario=140.0, hedge_price_now=130.0, hedge_units=163) ==
pytest.approx(1630.0)

    def test_short_hedge_gains_when_hedge_price_falls(self):
        assert hedge_instrument_pl(hedge_price_at_scenario=120.0, hedge_price_now=130.0, hedge_units=-50) ==
pytest.approx(500.0)

```

`tests/__init__.py`

Empty — just marks tests as a package.

Project configuration

requirements.txt

```
robin_stocks>=3.4.0
streamlit>=1.38.0
pandas>=2.2.0
plotly>=5.24.0
python-dotenv>=1.0.0
pyotp>=2.9.0
numpy>=1.26.0
scipy>=1.13.0
pytest>=8.0.0
ib_insync>=0.9.86
watchdog>=4.0.0
```

pytest.ini

```
[pytest]
pythonpath = .
testpaths = tests
```

.env.example

The template for local .env — never committed with real values (see .gitignore). DEMO_MODE here is what switches the whole app between the real Robinhood-connected version and the public-safe synthetic version, without changing a single line of application code.

```
# Copy this file to .env and fill in your own values. Never commit .env.
ROBINHOOD_USERNAME=you@example.com
ROBINHOOD_PASSWORD=your-password

# Optional: if you have TOTP-based 2FA (Authy/Google Authenticator) set up on
# Robinhood, put the base32 secret here so login can be fully automated.
# Leave blank to be prompted for the SMS/app code interactively instead.
ROBINHOOD_TOTP_SECRET=

# Comma-separated list of tickers you consider "vol products" for the
# dedicated VIX/vol-ETP exposure view (edit to match what you actually trade).
VOL_TICKERS=VXX,UVXY,SVXY,VIXY,UVIX,SVIX

# Set to "true" to run entirely on synthetic data (app/demo_data.py) with no
# Robinhood connection and no login gate — this is what a public deployment
# (e.g. Streamlit Community Cloud) should use. Leave unset/false for your own
# real-money local use. See README's "Demo mode" section.
DEMO_MODE=false
```

.gitignore

Keeps .env, the Python virtualenv, cached pickled data, CSV exports, and pytest's cache out of version control — the actual mechanism that made it safe to push this repo publicly with zero credentials in it.

```
.env
data/*.pickle
exports/*.csv
__pycache__/
*.pyc
.venv/
.pytest_cache/
```